Tampere University

Roni Ahola

IMAGE GENERATIVE AI APPROACHES FOR DIFFERENTIAL MOBILITY SPECTROMETRY DATA ANALYSIS

Master of Science (Tech.) Thesis
Faculty of Medicine and Health Technology
Examiners: Associate Professor Antti Vehkaoja
DSc (Tech.) Anton Kontunen
October 2024

ABSTRACT

Roni Ahola: Image Generative AI Approaches for Differential Mobility Spectrometry Data Analysis
Master of Science (Tech.) Thesis
Tampere University
Master's Programme in Biotechnology and Biomedical Engineering
October 2024

Privacy, scarcity, and acquisition challenges of medical data have motivated researchers to investigate the generation and use of synthetic data with the recently hyped generative AI models. Synthetic data can be used instead of or in addition to real data when training machine learning models to classify medical data.

In this Master of Science thesis, the feasibility of deep generative models was examined for the synthesis and analysis of differential mobility spectrometry (DMS) images. Through literature guidance and experimentation, a suitable model was identified and implemented. Its performance was evaluated on a previously acquired DMS dataset of 352 images measured from cancerous brain tissue. The dataset is split into two classes based on an enzyme mutation.

StyleGAN2, a generative adversarial network (GAN), was trained to generate the synthetic images. Once trained, the GAN produced new samples that visually and statistically resembled the original DMS images. These synthetic images were further evaluated for their utility in a downstream classification task, where they either augmented or replaced the original data when training a machine learning classifier.

Two classifiers were used in the classification task: linear discriminant analysis (LDA) and convolutional neural networks (CNN). With LDA, the use of synthetic images increased classification accuracy by up to 4%. With CNN, the use of synthetic images did not improve classification accuracy. The reasons for these results are discussed further in the thesis.

The generative model was also trained using a specific technique called StylEx to explain the data. StylEx created attributes in the generator that controlled the appearance of class-specific features in the generated images. A Plotly-Dash-based front-end tool was developed to visualize and analyze these features, and their semantic value as DMS features was further validated through a discussion with an expert. Finding and visualizing class-specific DMS features may aid in understanding what features are significant for class differentiation.

Deep generative models can produce meaningful and useful synthetic DMS images when trained on a small dataset. No definitive conclusions were drawn regarding the extent of the gains achieved through GAN data augmentation. The explainable AI method StylEx found attributes in the generator that were further analyzed to represent fundamental concepts in the DMS images, explaining AI mechanisms and providing insight into the data.

Keywords: differential mobility spectrometry, generative adversarial networks, StyleGAN2, StylEx, synthetic data

TIIVISTELMÄ

Roni Ahola: Kuvia luovan generatiivisen tekoälyn soveltuvuus differentiaalisen liikkuvuusspektrometrin datan analysoinnissa
Diplomityö
Tampereen yliopisto
Bioteknologian ja biolääketieteen tekniikan maisteriohjelma
Lokakuu 2024

Lääketieteen alan tutkimusta haastaa tutkittavan tiedon niukkuus ja saanti. Pöhinää herättäneet generatiiviset tekoälyt ja niiden luoma synteettinen data ovat olleet yksi tutkijoiden kehittämistä ratkaisuista tähän ongelmaan. Oikeaa dataa voi korvata tai sen käyttöä voi tehostaa synteettisellä datalla, etenkin jos käyttökohteena on luokittelevien koneoppimismallien kouluttaminen.

Tässä diplomityössä arvioitiin syvien generoivien mallien käyttökelpoisuutta etenkin differentiaalisen liikkuvuusspektrometrin (DMS, differential mobility spectrometry) mittaamalle kuvadatalle. Tämänkaltaiseen dataan soveltuvaa mallia haettiin sekä kirjallisuutta, että kokeellista mallivaihtoehtojen vertailua apuna käyttäen. Mallin kelpoisuutta arvioitiin käyttäen aiemmin kerättyä tietojoukkoa, joka sisältää yhteensä 352 aivosyöpäkudoksesta mitattua DMS-kuvaa. Kuvat on jaoteltu kahteen eri luokkaan entsyymimuutoksen perusteella.

Malliksi valikoitui StyleGAN2, joka on ns. kilpailuasetelmaa hyödyntävä generatiivinen malli (GAN, generative adversarial network). Koulutettu GAN-malli kykeni luomaan silmämääräisesti ja tilastollisesti alkuperäisten DMS-kuvien kaltaisia kuvia. Tarkemmassa arvioinnissa näitä kuvia hyödynnettiin koneoppimismallin kouluttamisessa joko ilman alkuperäisiä kuvia tai yhdessä niiden kanssa.

Vertailu tehtiin käyttäen kahta luokittelijaa: lineaarista diskriminanttianalyysiä (LDA) sekä konvoluutioneuroverkkoja (CNN, convolutional neural network). LDA-luokittelijalla synteettisten kuvien käyttö paransi luokittelutarkkuutta parhaimmillaan 4%, kun taas CNN-luokittelijalla tarkkuus ei parantunut.

Työssä tutkittiin lisäksi generatiivisen mallin kykyä ymmärtää ja selittää näkemäänsä dataa. Tähän sovellettiin StylEx-menetelmää. Tämän avulla generatiivisesta mallista voidaan löytää säätöjä, joilla kuviin pystyy erikseen lisäämään ja korostamaan luokkakohtaisia piirteitä. Web-pohjainen kojelauta koodattiin piirteiden havainnointia ja havainnollistamista varten. Piirteiden merkityksellisyys DMS-kuville ominaisina erityispiirteinä vahvistettiin ioniliikkuvuusspektrometrian asiantuntijan kanssa. DMS-kuvien havainnointi hyödyntäen tätä menetelmää voi auttaa ymmärtämään, mitkä piirteet erottavat tietojoukon kaksi luokkaa toisistaan.

Syvät generatiiviset mallit kykenevät luomaan merkityksellisiä ja hyödyllisiä DMS-kuvia, myös silloin kun koulutusmateriaalin koko on suppea. Selviä johtopäätöksiä synteettisen datan hyödyn laajuudesta koneoppimisluokittelijan kouluttamisessa ei kyetty luomaan. Selitettävän tekoälyn metodin StylEx avulla löydettiin syvästä generatiivisesta mallista säätöjä, joilla voidaan muokata DMS-kuvien luokkakohtaisia piirteitä. Tällä voidaan sekä selittää tekoälyn toimintaa että hyödyntää tekoälyä datan ymmärtämisessä.

Avainsanat: differentiaalinen liikkuvuusspektrometria, generatiivinen kilpaileva verkosto, StyleGAN2, StylEx, synteettinen data

USE OF AI IN THESIS

The AI tools used in my thesis, besides those used and developed in the technical work of the thesis, *and the purpose of their use has been described below:*

ChatGPT-3.5, ChatGPT-3.5 Turbo, ChatGPT-4, Microsoft Word Editor.

Purpose of use and the part in which it was used:

ChatGPT was used to assess if certain sentences were hard to understand. The rephrasing suggestions of the LLM were not directly used, as they often changed the meaning, but they helped in finding a new structure for some clunky sentences. Approximately five to ten sentences was modified together with the model.

Other similar small uses include: asking about grammar rules, as a replacement for a search engine, asking ChatGPT for words or concepts I had trouble recalling, as an advanced thesaurus for synonyms, or discussing some concepts in shallow detail.

The ChatGPT models were heavily utilized when writing Python code. The code provided by the AI was always manually verified, as it was never completely error-free. The jobs were trivial but laborious. Similarly, it was also used to find LaTeX commands when writing this thesis.

Some small random use cases may have been forgotten, but any AI bias in the thesis should mainly be the result of suggestion psychology rather than delegation of work.

Microsoft Word Editor was used for spellchecking and grammar.

I am aware that I am totally responsible for the entire content of the thesis, including the parts generated by AI, and accept the responsibility for any violations of the ethical standards of publications.

PREFACE

This thesis was the final part of the Master of Science Programme at Tampere University. I received funding from Emil Aaltosen Säätiö from a grant originally granted to Joonas Haapasalo, MD., for the development of brain tumor diagnostics and treatment with differential mobility spectrometry. I am grateful to Joonas for the funding as it gave motivation to the labor, and also for the discussion we had that provided me with more clinical context on the use of DMS equipment.

I owe gratitude to my main thesis advisor, Antti Vehkaoja, for helping me find this suitable topic and for being patient and helpful during the time it took to finish this work.

I also received help from all those who I spent time with talking about the topic either here in Tampere or remotely, especially from the Olfactomics team. Anton Kontunen advised me on my thesis and on the topic in general. Antti Roine provided much more context to the use of DMS and their work using it, and he was also responsible for the initial idea of using generative AI. Osmo Anttalainen was extremely helpful during a detailed discussion about the specifics of the sensor and helped me in the analysis of the DMS features.

This work was finally also supported by those who, although not directly involved, offered me motivating words of encouragement.

Tampere, 10th October 2024

Roni Ahola

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

α	Alpha curve or function of differential mobility
DMS	Differential mobility spectrometry
$\mathrm{C_V}$	Compensation voltage
$\mathrm{S_V}$	Separation voltage
AI	Artificial Intelligence
D	Discriminator of GAN
G	Generator of GAN
GAN	Generative adversarial network
GAN-train	An evaluation metric for GANs
StudioGAN	A code library with GAN implementations
StyleGAN2	A popular GAN model
StylEx	A special method to train a GAN
S-space	A parameter space in StyleGAN2
W-space	A parameter space in StyleGAN2
Z-space	A parameter space in StyleGAN2
Augmentation	Modifying a dataset by adding complexity to it
CNN	Convolutional neural network
DL	Deep learning
Evaluation metric	A scoring system for a model
LDA	Linear discriminant analysis
TNR	True negative rate
TPR	True positive rate

1. INTRODUCTION

Clinical data is generally scarce, expensive, heterogeneous, and also hard to share due to privacy [1], [2]. Although machine learning could be applied to much of the clinical data, the deployment is slow due to these issues. Modern techniques to tackle these issues in machine learning deployment are for example federated learning and synthetic data [1]. The goal of this thesis is to assess the feasibility of image generative AI used for synthetic data with a specific type of imaging data that is being researched for medical use.

The synthetic data is generated by a generative adversarial network (GAN). This type of synthetic data has been extensively researched in the medical imaging field [3], and it may alleviate the aforementioned problems when working with clinical data. The data is more private as it is no longer the exact same data measured from the patient. Synthetic data is also cheap to generate. The generation of the data may also be controlled to reduce the effect of the distribution shifts of heterogeneous datasets, for example because of equipment parameter or environmental changes. Synthetic data should also be useful in training a deep learning classifier, where overfitting due to memorization is a large issue.

The origin of the data is differential mobility spectrometry (DMS) measurements that function as images and the synthetic data will mimic them. The DMS technology uses low and high voltage electric fields to separate ions based on their electric mobility changes in the different fields. It can be utilized in a medical setting for example in bacterial detection, tissue detection from surgical smoke, or detection of cancer metabolites from urine. Classification of the images could help clinicians for example to have intra-surgery information about tissue-type without slow histopathological methods. [4]

The dataset used in this thesis comes from neurosurgery biopsies from cancer patients where a specific enzyme mutation differentiates between two cancerous brain tissues. This kind of data has all the common issues of clinical data. It is very scarce as the type of cancer is not common and the operations are not frequent, it is expensive, highly heterogeneous, and also hard to share. The DMS images themselves do not necessarily expose privacy issues but the data can be linked to other information in the hospital system. With synthetic data this link does not exist anymore. Intra-surgery classification of the isocitrate dehydrogenase (IDH) mutation, the dataset of the thesis, could lead to a different clinical decision-making process [5].

The thesis work consists of two tasks where the generative AI is used. In the first task the synthetic data is used to train a classifier. The classifier decides what class a DMS image belongs to, such as IDH-mutated or IDH wild-type. The task is to find out whether synthetic images could be used to train a classifier or improve a classifier trained without them. A deep learning classifier is completely dependent on the quality and quantity of the training data.

The second task aims to explore whether generative AI may be used to explain the decision of the classifier or reveal information about the data. The goal of the second task is to see if the generative AI can reveal information about the data or explain decisions made by an AI classifier. The intuition is that if the AI model can generate images of two different classes, can it explain the differences as well? This explainable AI would also help with the adaptation of AI methods to clinical settings, as the black-box nature of AI causes distrust in the technologies.

The two tasks form the two research questions for the thesis:

RQ1: *Could synthetic images be useful in training a classifier for DMS data?*

RQ2: *Can the generative AI be used to explain data or classifier decision making?*

The work first explains the theories behind differential mobility spectrometry and generative adversarial networks in Chapter 2. Then I will go through the research experience of finding the most suitable GAN model for this thesis in Chapter 3. In Chapter 4, the implementation details are explained. The results are shown in Chapter 5 and a discussion about the results and future prospects and finally some limitations and challenges are presented in Chapter 6. The thesis is concluded in Chapter 7.

2. THEORY

In this chapter, I will explain the main theory behind the techniques in this thesis. The first technique is the method of data acquisition while the second is for data analysis. The data acquisition is done by differential mobility spectrometry (DMS) that utilizes the differential mobility of ions in low and high voltage electric fields. The data analysis part is about using generative adversarial networks (GANs) to generate synthetic images that should be similar to the images output from the DMS sensor.

I will explain how the DMS sensor works and how it produces a 2D image output. I will also focus on typical features that appear in these images. Knowing how the features in the images are formed is necessary for the later assessment of the synthetic images.

As both subjects are rather broad and unrelated to one another, I will be brief in my explanations and shallow in technical details. The DMS is explained especially from the perspective of data analysis usage. The GAN is very high in the evolution of deep learning neural networks and an in-depth explanation of the mechanisms is out of scope for this thesis. For references to the GAN concepts, I suggest [6], [7] and also [3] to understand the medical point-of-view for synthetic images.

The practical workload of this thesis is in building, training, and using the GANs, as the DMS data was already acquired beforehand. It is necessary however to also understand the concepts of DMS to understand the features we are interested in. I will start by going over the theory of DMS and then move on to GANs. Certain implementation details are left for Chapter 4.

2.1 Differential Mobility Spectrometry

In this section, I will introduce differential mobility spectrometry (DMS). This is a technique used to differentiate ions in a gas by separating them based on their mobility difference in low and high voltage electric fields. The DMS sensor can capture data that is expressed in 2D image format with either one or two channels, corresponding to negative and positive ion images.

I will start by explaining the method in brief using [8] as my main reference. It is necessary to understand the physical and chemical phenomena underlying the 2D images to get intuition on how to analyze the images and to understand the results from the analysis methods applied. I will introduce some use cases of DMS to showcase common approaches and challenges. This section aims to bring the necessary understanding of the DMS sensor to understand how the data could be analyzed using a generative adversarial network (GAN).

2.1.1 DMS principles

The concept originated in the Soviet Union in the 1980s. It was found that a specific electric field setup with high voltages started to separate ions in a non-linear manner that is different from the linear separation effects in low field strengths. Both are based on the mobility coefficient of the ion. The name for the method started from "Gas Analyzer of Ions", later referred to as "High Field Asymmetric Waveform Ion Mobility Spectrometry (FAIMS)". A different design with differently shaped sensors was researched in parallel to FAIMS and referred to as a "Drift Spectrometer" and is the design now called DMS. In current use DMS and FAIMS and sometimes DIMS/IMIS are often used as synonyms although their mechanisms and thus also their analytical properties differ [8]–[10].

DMS is in the loose category of eNose technologies and methods [11]. eNose equipment are used to analyze gases for a wide variety of uses such as military gas analysis, diabetes screening, and indoor or outdoor air analysis. As an example, human breath contains an estimated 200 to 300 detectable volatile organic compounds (VOCs) [10], some of which indicate certain pathological processes. Traditionally this is done with chemical sensors such as metal oxide sensors (MOX/MOS) that can be equipped with different sensitive materials and doping to optimize for the specific application. Besides these, there are other gas detection-related sensors. Optical sensors use optical properties by shining light on the gas, mass spectrometry (MS) ionizes and separates ions based on their charge-to-mass ratio and chromatographic methods use the phase-states of the mixtures for the separation, notably as liquid or gas chromatography (L/GC). To increase the resolving power, these can be used in conjunction such as in the hyphenated techniques G/LC-MS or DMS-LC-MS [12]. A very close method similar to DMS is the ion

mobility spectrometry (IMS). DMS is sometimes referred to as an advanced method over IMS although both have different advantages [13], [14].

Comparisons between the sensors have been made, particularly with DMS and MS. DMS is said to be more robust compared to MS as the "individual instruments can retain their functionality without major maintenance or calibration for several years" [15] as well as having lower cost and complexity [16], [17] especially because MS requires a vacuum to operate. It is noted that the design phase of a DMS sensor requires an initial "complex engineering challenge" [18]. However, the MS should have a higher resolving power to better detect some more ions as compared to the DMS [17]. In a study on Chinese soy sauce [19], FAIMS was suggested to be more portable and faster compared to traditional compound analysis methods in detecting characteristic compounds. In a review study [20] it was also contrasted to GC-MS, a process that takes up to an hour to separate VOCs based on chemical volatility, whereas FAIMS separated ions in near real-time.

An important comparison between DMS and GC-MS is that while the mass spectrometry-based data is quantitative, DMS is not completely quantitative but it is instead with a more strict definition a qualitative method as is explained later. The main opposition to chemical-based sensors such as the MOS/MOX is that they tend to drift very strongly due to a buildup of chemical substances and have high variance between measurements [21]. The way DMS operates makes it more robust against drift and variances between batches [10].

Especially in medical settings MS and DMS are compared against each other. Arguments are based on the cost and complexity. The vacuum is required for MS and the measuring time is much longer [17]. A more developed method named rapid evaporative ionization mass spectrometry (REIMS) is based on MS analysis of originally surgical smoke and has shown promising results [22], [23]. REIMS can differentiate between benign and cancerous tissues at high accuracy within seconds during surgery by differences in relative abundances of triglycerides and especially certain phospholipids that differ in cancerous tissue proposedly due to the Warburg effect [24]. Although fast, the equipment is still large and costly, limiting clinical use. The advantages of DMS are that it is light to operate and after initial challenges it is easy to use. It has high sensitivity in the ppb to ppt level, low power use, and inexpensive manufacturing and also has use in the hyphenated techniques. Most comparisons end up concluding that while they operate in similar regions, they also have their own areas of operation where they surpass the other in applicability, cost, or strength.

As mentioned before, the DMS is thought of as a descendant of IMS, ion mobility spectrometry. Due to its simpler design, the IMS was commercialized much before the DMS [13]. IMS has low demand for power, it is small in size and weight, and has high sensitivity in its operating range. It is therefore a very suitable measurement technology to be

applied to handheld instrumentation for applications in medical use such as acute bacterial infections from wounds [25] and also other applications including military use for the detection of chemical weapons [13].

To understand DMS it is good to start with a typical example implementation of IMS taken from [13]. In it, a neutral sample is first ionized typically by a radioactive source. Then, a small amount of gas is introduced into a drift region such as a tube. In this drift region there is typically a constant electric field such that the gradient is in the direction of the tube. This gradient will move the ions. As they move, they collide with the neutral molecules of the gas, dissipating energy. This gradient-collision event that happens in a low field will define the time taken in the drift tube to move from the start to the end. This *drift time* is used to separate ions. The relationship between gradient and collisions is different for different ions due to their shape and size [13], typically described by the statistical correlation between size and the chance for collision. The separable parameter that characterizes the differences between ions is written as *mobility* K. We can then write the relationship between the mobility, velocity v_d and electric field strength E as

$$K = \frac{v_d}{E}. \tag{2.1}$$

The K then represents mobility in a given gas with given gas parameters and changes for each ion with specific size, shape, and charge parameters defined by the ion-molecule properties. With a known field strength E, and by controlling the time of the measurement, we can get the drift time converted to velocity using the known tube length and therefore we can separate ions by their mobility K.

Although other implementations exist, the main theory of IMS is explained in the simple Equation 2.1, where the separability is the relationship between mobility, velocity, and the electric field.

The DMS uses the fact that K behaves differently in high values of E as it will start to show new non-linear behavior. This new separation method may have more resolving power compared to IMS especially when the shapes of ions are too close to each other and cluster into similar-sized clusters. The nonlinear effects in DMS that are caused by a dynamic clustering environment in the ion-molecule interactions [26] give a new way of separation. This new separation property is dependent on different properties of the ions compared to those of IMS. There is more emphasis in the polarizing parameters of the analyte compared to the shape and size parameters and less signal overlap with similar drift velocities. When the field strength is in the high setting (E_H), the nonlinear effects change K to either decrease or increase, typically depending on whether the ion became lighter or heavier [8].

To account for the non-linear effect of the field strength we rewrite the Equation 2.1 as

$$v = K(\frac{E}{N}) \times E, \tag{2.2}$$

where E/N is the gas-density normalized value for E. Accordingly, the gas density N could be used for the separation, but due to practical reasons the systems use E instead and keep N constant using carrier gases such as ambient air or N_2 [9]. E/N is later also noted simply as E.

In IMS, the ions survive if they have enough velocity to pass the tube in the given drift time. The velocity of an ion in a low field suitable for IMS is around $1 - 5$ m/s. To use the nonlinear effects for separation, the field must be high enough that the ions will speed up to velocities even two magnitudes higher, up to $300+$ m/s, while the mobility K only changes some percentages [27]. The very fast speed of the ions requires modifications to the sensor mechanisms.

DMS sensors are built using asymmetrical waveforms of the electric field. This means that their integral over applied field over time equal each other, but they have opposite polarities,

$$E_1 t_1 = E_2 t_2. \tag{2.3}$$

In Figure 2.1, we can see a simple setup of how this is applied. In the first field strength, the polarity may be positive or negative, and in the second it is the opposite. A step-like waveform is applied such that the Equation 2.3 is satisfied and the total electric field strength over time is 0. This means that equal amounts of drag are applied to the ion towards the ceiling and the floor directions of the filter. Ion species without differential field dependence in the given fields have 0 movement in the perpendicular y-axis with behavior following Equations 2.1 and 2.3. Ion species with differential field dependence in the given fields will however move towards the ceiling or the floor, as there is now the new non-linear behavior in v. For the ion with differential field dependence, there is then a displacement when the total electric field satisfies 2.3. This displacement is seen also in Figure 2.1 in the bottom part.

As we use the low-field mobility and high-field mobility together in different polarities, the *difference* in mobilities is what decides whether the ion survives the tube or not. The floor and ceiling of the tube neutralizes the ion without detection while the end of the tube neutralizes the ion but also detects the charge caused to the electrode by the ion. The goal therefore is to survive the filter region to be counted at the end. When both high and low field strengths are small enough to not cause any non-linear effects, most if not all ions survive as no directed movement toward the floor or ceiling happens. No separability is possible in this situation. When the field strengths are increased in both polarities the asymmetric waveform causes the saw-tooth-like pattern of movement seen in Figure 2.1,

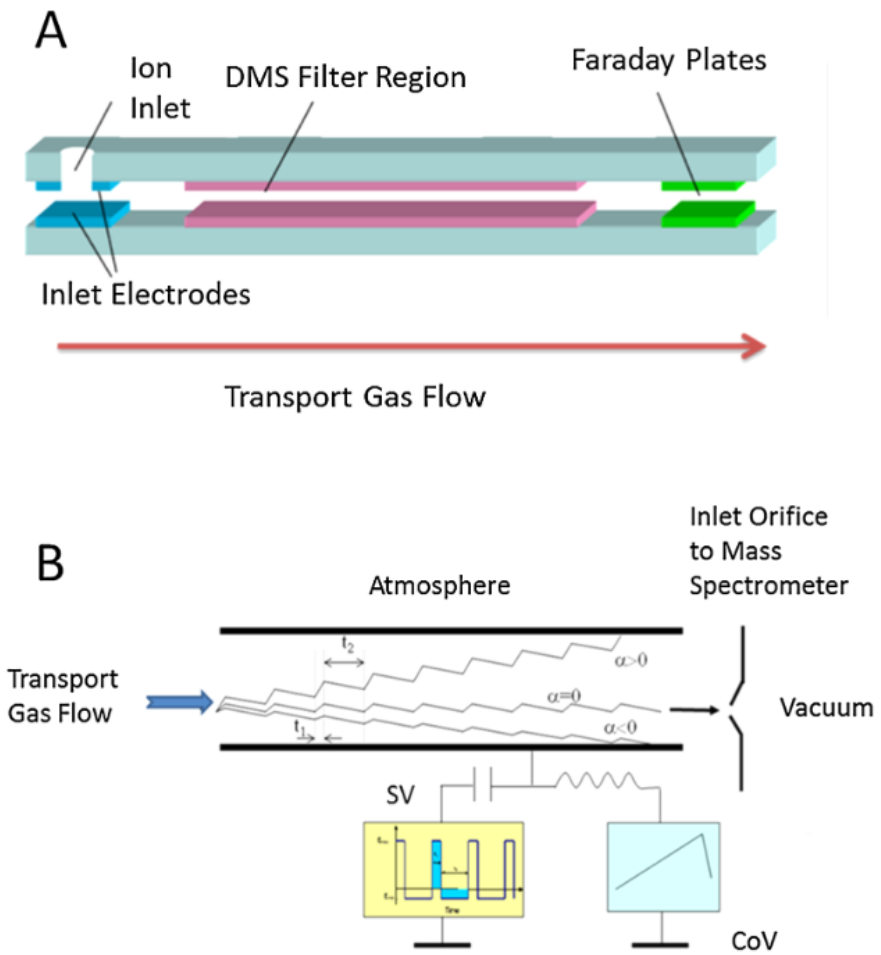

Figure 2.1. *An example overview of a DMS sensor. The gas with ionic species flows through the tubular section. The filter is adjusted using the compensation voltage value. Increasing the separation voltage brings more energy to the sensor, giving more separability but also increasing the speed of the ions. In A the complete sensor is shown, while B focuses on the filter region. Image taken from [8].*

where only those with a specific mobility difference survive the filter.

Depending on the set polarity of the higher field in the instrumentation, the ion will have a displacement d towards either the ceiling or the floor of the sensor. As this ion would get neutralized upon impact with a filter plate when the d is too high, compensation must be applied if we want to capture this ion with this high voltage value. Higher voltages are necessary to cause the separation and therefore this compensation is needed to capture those events. The *compensation voltage* C_V can thus be thought of as a bandpass filter setting to let exact ions pass.

We can write the total displacement d using the sum of displacements in both fields as

$$d = K_L E_L t_L + K_H(E_H) E_H t_H. \tag{2.4}$$

In order to make the single ion survive the filter, we need to have $d = 0$, or very low. We can write the above equation in that situation, clumping together all the constant variables such that

$$K_H(E_H) = k_1 \frac{E_L}{E_H}, \tag{2.5}$$

where k_1 includes all the constants, except the field strengths. If we apply a DC voltage, named the compensation voltage C_V, we get

$$K_H(E_H + C_V) = k_1 \frac{E_L + C_V}{E_H + C_V}. \tag{2.6}$$

As $C_V \ll E_H$, we can approximate that the high-field mobility K_H is dependent only on the ratios of the fields and it is controlled by the new DC voltage for the situation $d \approx 0$. Therefore, this DC voltage is a measurement of the differential mobility of the ion in the given measurement setup. The Equation 2.6 displays the filter parameter C_V that controls whether the ion passes the filter or not, and the ion that passes has the mobility given by the equation. We can see that a positive value for C_V will cause a larger K_H to be able to pass the filter, while a negative value will cause a smaller K_H to be able to pass.

Besides controlling the filter at one set of field values E_H and E_L, we can also change the value of those fields in a manner that Equation 2.3 holds by increasing or decreasing them proportionally. The waveform of E_H and E_L is denoted as S_V for *separation voltage* as it is the voltage responsible for bringing the separation energy to the system. With enough energy, the non-linear effects will start to happen. When this happens, an ion that had $d \approx 0$ before will now have $|d| > 0$, and the compensation voltage must be adjusted to capture the ion with the sensor. By adjusting S_V, we cause separability, and by adjusting C_V, we can measure a value for the differential mobility that caused them to separate. There are certain commonalities or types of how ions will behave that are explained later and can be seen in Figure 2.2.

The relationship between mobilities can be analytically characterized simply as the ratio of the two mobilities $K_H(E)/K_L$. The common way of expressing this in literature is in the normalized form where the low field mobility is used to normalize the value to be $\ll 1$ such that

$$\alpha(E/N) = \frac{K_H - K_L}{K_L}, \tag{2.7}$$

more commonly known as the *alpha function*. An example curve representing the function is shown in Figure 2.2, together with a DMS *dispersion plot* and a single row representing a single value of S_V with different values for the filter C_V. This alpha function is the separability information provided by the DMS sensor. If we raise the field value, mobility will change more. To capture the effect without neutralizing the ion under study, we need to adjust the filter with the DC voltage. Therefore, we have two parameters S_V and C_V that will form the 2 dimensions of the image. Each pixel therefore represents (C_V, S_V)

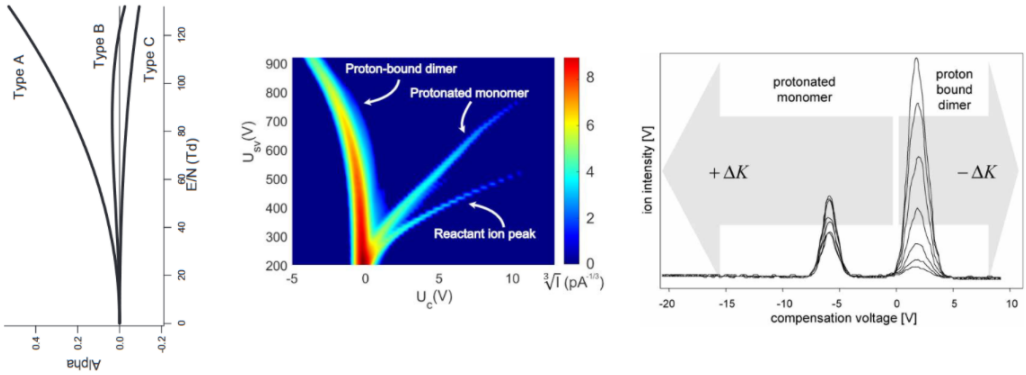

Figure 2.2. *Three representations of the DMS sensor output. On the left, theoretical a-curves are depicted for the three different types. In the middle, a real measurement is shown with some typical features highlighted. The features follow the typical curvatures. On the right, a single row representing a single value of S_V is shown to highlight how the ions are normally distributed in the row due to the sensor design. Image combined from [8], [13], [15].*

and the whole dispersion plot is defined in these dimensions. Any single pixel is the total current measured over a set time, consisting of all the ions that survived the filter and caused current at the electrode as they reached the end of the sensor as was seen in Figure 2.1.

The "root" seen in Figure 2.2 at $(0,200)$ represents the situation where the level of the higher field is too low to cause any non-linearity in the mobility. All ions will pass the filter and are counted near that pixel instead of any other in the same row, causing a very high-intensity reading and providing no information value. As we go higher in the image, the mobilities will start to be affected by the high energy field S_V. The adjustable DC voltage captures that curvature. The sensor can be set to scan any values of C_V that are interesting to "follow the curves" and capture their behavior. The finalized image will depend on how many different values of C_V and S_V are scanned over. Due to the instrumentation electronics, scanning over C_V is typically faster. The total time for one image to form may take anything from seconds to tens of minutes, depending on the resolution defined by the values scanned as well as, for example, the flow velocity of the gas [18].

Although the alpha function is a quantified value that depends on the shape and electrical properties of the analyte, we can't find the exact value for the α defined in Equation 2.7. We are missing the value for the low-field mobility K_L [9]. We could use an IMS to determine it based on Equation 2.1, highlighting how the methods complement each other. The DMS image is therefore a qualitative rather than a quantitative expression of the spectrum of differential mobilities found in any given gas giving us the *shape* of the alpha function, rather than its absolute values. This shape is commonly noted as the

alpha curve or α-curve. Another approach to gaining quantitative knowledge is through simulations of ion-molecule interactions [8], [26].

To summarize, the principle of differential mobility spectrometry is to separate ions by their electrical mobility behavior. Ions are bandpass filtered by their behavior in the combined field due to the differential mobility when alternating between low and high electric fields. These filtered ions can be fed for other analysis tools (LC, MS...) [12], or used as such to form an image that contains the alpha functions as curving features of the analytes present in the gas. These *alpha curves* can then be used to gain information on the gas, map them to specific analytes, or do simple classification based on the images. Ideally, one curve represents one analyte.

I have now explained briefly how the DMS dispersion plots, or images, such as those in Figures 2.2 or 2.3 are formed. A gas is fed into the sensor, ionized, and then fed into the filter. This happens very fast as the parameters of the sensor are swept to form the image, each pixel typically taking some milliseconds to form [18]. I will next briefly explain how the ionization is the basis for the detection and what features are present in a typical DMS image. This leads to the discussion of data analysis of DMS images.

2.1.2 DMS data analysis

Here I will go over some common features in a DMS image and how they form. The two most important features already seen in Figure 2.2 are the monomers and dimers and they come about from the ionization process of the DMS.

Typically, a given number of ionic species are available for the gas under study. These ionic species bind and re-bind constantly and the amount available for any alpha curve is limited. In the ionization phase of the sensor, it is typically always the carrier gas molecules such as N_2 and O_2 that get ionized first due to their extreme abundance. After a series of chemical reactions where the reactant ions change form [13], the important stable reactant ions in ambient air carrier gas are pronated water clusters $(H_2O)_nH^+$ and deprotonated water clusters $O_2^-(H_2O)_n$, named positive and negative reactant ions (RIP and RIN) [15] or alternatively also named together as the reactant ion peak (RIP+ and RIP-). These two then collide with the neutral analytes to form ionic analyte species that may be captured by the DMS filter. This event can be highly controlled by changes in gas and ionization if studying specific analytes [8]. Importantly, the amount of these reactants is limited, and they are competed for by any analyte molecules in the gas. The winners are chosen by their proton and electron affinities. This limits the informative area in ambient air gas to be above that of water, as water clusters will snatch most protons left for itself [28]. Similarly changes in relative concentrations of the molecules will have an effect, with extremes blocking each other out and humidity causing general issues. The surrounding humidity and temperature are common instrumentation parameters in

the manufactured sensors in hopes of normalizing the data with them [13].

The monomers and dimers are formed through reactions with these water clusters. The monomer is formed by displacing water to form the protonated monomer MH^+. In the event of a high concentration of M, two of them may bind together with the proton to form the proton bound dimer M_2H^+. The monomers and dimers have been found through empirical concentration research to have the typical curves seen in Figure 2.2, where the dimer with a negative value for the alpha function will curve to the left and the monomer with a positive value to the right. More theoretical models for the events that define the curvature express three types; A, B, and C for the curves, as seen in Figure 2.2. One type leans to the right and keeps going there, another to the left, and one goes right first and then curves to the left. Although it would be useful, these interactions are not yet completely understood [8], [26] and a deviation from the well-known A-C is also proposed as A-E [9].

The three curves depicted in Figure 2.2 are likely related to the hydrated clusters losing water molecules due to increased kinetic and thus vibrational energy of the cluster. As our analytes of interest M reside in these clusters, these curves are of high interest to us. It is a possibility that they may curve in the opposite direction when the energy field is high enough, as some other events affecting the molecules such as polarization might overcome the clustering and change mobility to the opposite direction. These effects produce four features: curve to the left, curve to the right, and change of direction for both. The curve to the left would generally not bend, but the curve to the right is observed in doing so quite often. The monomer curve to the right is of more interest as the dimer curves more slowly giving a narrow band for the analysis due to the high field limits of the sensor. A fifth feature is *fragmentation*. It occurs when the energy field is high enough to obliterate the molecule into pieces. This may reveal itself as a jump in the image, where a fragment may pick up its own curve higher up in the image if it manages to snatch its own reactants from the limited supply.

The DMS images have therefore a very complex dynamical ion-gas interaction behind each pixel formation. As data, however, it can be analyzed without all the knowledge about the chemical properties and events behind it. The five features explained above will be used as a basis to explain some results from the methods used in this thesis. I will go over some DMS data analysis examples in the next section.

2.1.3 DMS data analysis in medical applications

The DMS/FAIMS can be used in a range of applications. In a study on soy sauce [19], the authors did food control by detecting differences in the product to classify the region it was made in using wavelet packet decomposition (WPD) and principal component analysis (PCA) as feature extractors for a machine learning (ML) model. Another example [29]

includes detecting compounds harmful to humans in water vapor using a simple k-nearest neighbor (KNN) classifier. In a medical setting example [30], it was used to differentiate between the urine of a coeliac disease patient and an irritable bowel syndrome patient based on possible differences in microbiome using sparse logistic regression. Another medical example [31] includes detecting bad bacteria from stool samples using classical feature extractions tools, such as wavelets, for common ML classifiers including random forests (RF) and support vector machines (SVM). In a study on clinical wound infections [32], FAIMS images were also classified using more complex feature extraction techniques. In a recent study on acute rhinosinusitis [33], sinus secretions were analyzed for bacteria using DMS, and patients were classified using KNN and linear discriminant analysis (LDA). A systematic review [20] concluded that a micro-chip FAIMS used together with ML algorithms achieved promising results of roughly $80\%+$ sensitivity and specificity in point-of-care portable diagnosis of pancreatic cancer, diabetes, and gastroenterological studies.

In the research paper that uses the same data as this thesis [4], brain tumor samples were cryosectioned to be later burned with an electrosurgical knife to produce surgical smoke. The smoke was the basis for the DMS image, and it was classified using LDA and other methods. Similar work with surgical smoke was done *in vivo* to study the classification of breast tumors to aid in intrasurgical margin detection of breast cancer [23]. Some deep learning (DL) classification methods are also present in the literature [34]–[37], but classical ML algorithms still dominate in DMS data classification.

The use of DMS is motivated by positive results from a broad range of use cases where no other equipment could be used due to their expenses, time taken, or analytical power. Most of the aforementioned examples I presented are also completely non-invasive, not necessarily needing any tissue samples for biopsy, such as when doing normal removal of surgical smoke during electrosurgery. Most of the published research I found was done in close location to, or in collaboration with, a company that manufactures such a device, and in total used only a handful of different devices. The data analysis algorithms that are used in research differ, but they are generally based on first doing some feature extraction and then classifying instead of classifying the image without any preprocessing.

Most use cases I reviewed used classical machine learning (ML) algorithms. Feature extraction methods included wavelets, clustering, forward sequential feature selection (FSFS), principal component analysis (PCA), curve finding, and more [15], [36], [38], [39]. Most pixels of the DMS image are supposed to be devoid of activity and useless [18], which is why the whole image is not generally used and features are instead extracted. Classifiers included classical algorithms such as LDA, KNN, SVM and RF. The features are extracted from the full data (images) and fed into the classifier. Especially LDA (linear discriminant analysis) [40] has been very successful and it would make sense for the images to have linearly separable dimensions if the classes are completely explained by

different a-curves. The classifier learns how to classify from the training data and is then used to classify new, unseen test data [41].

A general machine learning classification example is a classifier that can tell between an image of a dog and a cat. That task was generally hard before deep learning methods [6], and computer vision relied on feature extractions such as color space features, edges in the image, and templates of the animals. Two examples of a classification problem of DMS data are illustrated in Figure 2.3. In the first example a human eye could differentiate between the classes but in the second one it couldn't. In both cases the machine must be taught some mechanism on how to do that.

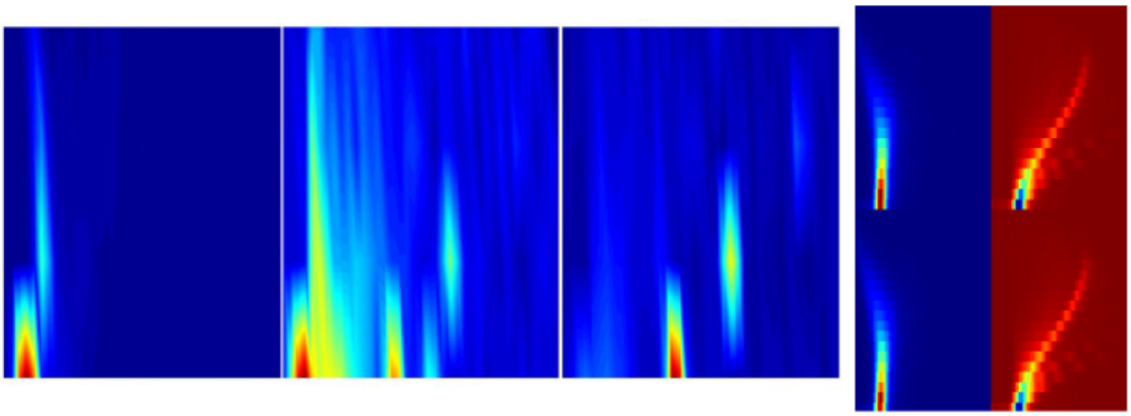

Figure 2.3. *Two different DMS datasets visualized. On the left, an easy three-class classification problem between liver, lung, and subcutaneous fat tissue with one ion dimension. On the right, the positive (left) and negative (right) dimensions for the IDH-mutated (upper) and IDH-normal (lower) classes that are used in this thesis. The three-class problem has much more distinct features and is discernible by the human eye, while the problem on the right has most of the intensity of the reactant ions spent on features that are shared between both classes making the differentiation impossible without contrast tools such as sigmoid or log-row. A machine may, however, see much more in the pixels as the hidden intensities reveal themselves in the data. The left image is taken from [15].*

Classification of data is a general problem, but classification of images is a specific computer vision problem. Using modern deep learning-based computer vision tools, such as convolutional neural networks, have shown comparable results to that of the LDA in DMS image classification, although the generalization issue has been present [15]. The neural network may overfit if the variance of the dataset is too low for the complexity of the network and it may simply memorize everything during learning [6]. This is a common problem in the medical domain where the data is scarce.

Although the theoretical alpha curves defined by Equation 2.7 are only functional curves, the curves in the image are formed by curves that are normally distributed around their peaks representing more a mountain range than a functional curve, where the peak gets smaller as separation voltage S_V gets larger. The geometrical and topological informa-

tion is not limited to these peaks because features such as fragmentation exist, and the complexity of the gas due to an unknown number of analytes creates a very heterogeneous setting for the sensor. Computer vision tools do not only succeed in finding curves and edges of images but also global and local complex and undefined formations. They may capture information in the image that is not explained by the alpha function such as changes related to concentrations of analytes and their relative water clustering as an example.

Now I will move on to the other topic of the thesis, modern deep learning neural networks and specifically the generative adversarial network. I will go over how it will be used specifically to tackle the two research questions set out in the introduction: *1) improving downstream classification* and *2) explaining data*. The covered DMS section is especially important when considering the latter one, as the domain knowledge will be necessary to understand the features in the data.

2.2 Generative Adversarial Networks

In this section, I will explain the basics of the generative adversarial network (GAN) and go more in depth with concepts that relate to the tasks set out for this thesis.

The generative adversarial network, GAN and its variations thereof, was famously called by one of the most prominent researchers in the field (Yann LeCun) "The most interesting idea in the field of machine learning in the last ten years" in 2016 [42]. This interest in GANs also appears in research literature where high research output from all fields is found, including medical [3].

The GAN is one of the modern generative deep learning network frameworks behind recent innovations in modern generative AI. Example tasks for generative AI include generation of natural text with text-generating models or synthesis of realistic images with vision-related models. While currently the most popular models are based on diffusion [43] (such as DALL-E), or generative pre-trained decoders and transformers [44] (such as ChatGPT), the GAN remains a popular choice in many tasks [45].

In this thesis GANs are used to examine the possibilities of synthetic DMS data. It has been well shown in literature [3] that GANs have a multitude of uses with medical imaging data. Some common tasks include reconstruction by reducing noise or artifacts due to measurement equipment, synthesis for completely new images or synthesis between modalities such as converting MRI images to CT, anomaly detection, classification, and more [3]. This thesis focuses on the synthesis of new images, where new images are formed based on training the GAN using the IDH dataset described later in Section 4.1. The idea to use the generated data to *improve* the dataset was popularized especially by a single publication [46] where the authors increased a very small CT image dataset size by adding the generated images into it. This new *augmented* dataset improved the downstream task of classification by a large margin. A lot more research followed. This is the basis for the RQ1 in this thesis, where I will generate synthetic DMS images to augment the dataset and compare classification results to see whether the synthetic images are of use for DMS image classification.

I will first briefly introduce how the modern GAN came to fruition and then some basics of how GANs are trained and used. Due to the depth and breadth of the subject, I will keep the technical parts short and focus on the key aspects of the GAN. After explaining the basics, I will describe the two key components of this thesis regarding the GAN. The StyleGAN2 [47], possibly the most popular GAN model, as well as StylEx [48], a counterfactual-based method to explain the inner representations of data. With these I aim to give the tools to answer the research questions put out in Chapter 1.

2.2.1 GAN basics

Here I will start by presenting a short history of generative adversarial networks (GANs) starting from around 2014 when the seminal paper [7] was published, springing life to the field of deep image generative AI. It is important to note that this did not happen in a vacuum, but instead was a part of the new "AI spring" with the exponential rise in the use of modern CNNs and other deep learning neural networks showing impressive capabilities in many settings.

Computer vision is a field of machine learning, where the goal of the computer is to have vision in terms of recognizing objects and patterns [6]. With large datasets, comprised of millions of images, algorithms compete on how well they can understand and explain these images. The most well-known dataset is the ImageNet [49], where thousands of classes such as a car or a dog are included in millions of images. The target for the computer is to learn to recognize an image and label it to a class that it belongs to, without ever seeing that particular image. In the field of deep learning, this is done by training a large neural network. In a subset challenge of ImageNet, ILSVRC, 1 000 classes were included. In 2010 the best top-5 error rate was 26.0%. In 2017 this was down to 2.3% with 5% being the human estimated accuracy. The contest was largely considered solved at that point.

The accelerated increase in accuracy started in 2012 with the introduction of a convolutional neural network model into the competition where the winner had a 10.8% run-up to the next competitor. The 2012 winner is considered as a starting point for the modern image classification hype, and it is one of the most cited machine learning papers with over 161 000 citations according to Google Scholar. The original GAN paper had over 81 000 thousand while the popularly-cited transformer paper 'Attention is all you need' had 129 000 thousand at the time of writing. The largest citation holder for deep learning is likely the residual image classification paper with over 236 000 citations in Google Scholar. Such comparisons do not necessarily imply the best or most interesting solutions, but they give hint as to what the researchers have found the most interesting deep learning techniques to focus on.

Although the convolutional neural networks used in the 2012 paper were already generally developed in the last century, it was the parallel computing using graphics processing units (GPUs) that ignited a fast spark that spread quickly through competitions with well curated datasets, such as the ImageNet. Parallel processing units are necessary for deep learning applications and GANs are also trained typically with either GPUs or in increasing amounts custom-made chips such as tensor processing units (TPUs).

The history of GANs and also deep generative models as a whole started only recently around 2014 with the mentioned paper by Ian Goodfellow [7], and as seen in Figure 2.4,

Figure 2.4. *Short history of image generative AI. It highlights the rapid evolution of image generation in the last ten years. While CNNs date back to the 1980s, adversarial learning is a new concept that has led to a lot of hype and research. Image from [51].*

the evolution has been very rapid. Besides good technical and research innovations, much progress has also been credited to the availability of public datasets. The human face and photos of animals are abundant in research datasets, which is why they are often used in research literature. Medical imaging has generally been lackluster in its representation though recently some large high-quality datasets have been published through competitions or collaborations [50].

According to Wikipedia definition [52], the GAN is "a class of machine learning frameworks and a prominent framework for approaching generative AI." A more refined definition given in the literature [7], [53], [54] is that the GAN is a data-model, where a minimax game between two parts leads the generative part of the model to Jensen-Shannon divergence between the generative and data distribution and therefore to the capability of the generator to sample the real distribution.

To achieve this, it consists of typically two neural networks that are built using typical neural network layers, such as fully connected layers or convolutional layers [6], [55]. The

two networks are set to be in what is called an adversarial process of competition. In the adversarial process the two networks compete in a game-theoretic minimax game [7]. They both try to get better, but getting better means the other gets worse. However, as the other gets worse, it subsequently will learn with more intensity. This learning is done in turns. The result of this game is a *Nash equilibrium*, where the system is in a stable state if both networks keep using the same strategies. This strategy is the *objective function* for the network and the two networks named Generator (**G**) and Discriminator (**D**) will reach this equilibrium guided by this objective function. A common objective function in mathematical expression for both G and D is given later in Equation 2.8.

The job of generator (G) is to generate *fake* images that the discriminator (D) is unable to distinguish from real images. When the G and D reach their equilibrium, the G is as good at fooling D as it possibly can be, when simultaneously the D is as good at discriminating between generated (fake) and training (real) images as it can be. This is due to the G outputting the same distribution as the training dataset in the theoretical situation. Unlike a typical neural network where the training is typically concluded when the loss functions reach a certain level, the loss functions of D and G typically oscillate after convergence to the equilibrium. As either G or D gets slightly better with continuous learning, the other will learn more and cause oscillation. This oscillation is seen as a search for new strategies. The objective function or *loss function* of G will cause it to learn new ways to fool D, while the loss function of D will teach D how to recognize the fakes of G. The end of training is usually decided by outside critics known as the *evaluation metrics*. The simplest metric is the human eye. If the image looks good, the training is complete. Training longer might make it look better, or different.

The *training* of GANs and neural networks in general is guided by a few general principles and large amounts of linear algebra. The loss function is the teacher for the network as it gives feedback on whether the network did a good job or not. The feedback is used to update the parameters known as the *weights* of the network. These weights are what make up the network. They are numbers that define the outcome of the linear algebraic equations such as in the simplest feedforward layer of $y = wx + b$, where w is a node weight and b is a bias weight parameter. This single node on a feedforward layer would then have two learnable parameters. By updating both, the outcome y for the input x changes towards the desired result. If the output y is bad, the loss function value will be high, and the weights will get updated with a larger swing. If the loss function value is very low, the weights will get updated less. Eventually, the loss function value is small enough that the weights barely get updated anymore. This means that the output was good, and training is complete. For the example above, the b and w will serve to approximate output y for x. If x is an image, output y could be the class label, such as 'cat'.

An important parameter to control the speed of learning is the *learning rate*. It is a multiplier to control how strongly the values for the weights change each time the loss function

value is calculated. With overall millions or billions of parameters all learning together from the *backpropagation* of the loss to the network, the training often requires finding an optimal learning rate that does not cause too large swings in any direction and result in unstable learning.

As explained above, the GAN has two loss functions for the two networks and instead of reaching a minima, they reach an equilibrium with each other. Although the training may take place forever, this general use of loss function to update weights still applies. If the G has a very high loss, it gets updated with a larger gradient toward new values for the weights. Eventually, this faster learning will result in G being better than D, tilting the scales and causing oscillation around the equilibrium of losses. It is important to realize when either side starts to overpower the other as it may indicate issues in the training. This typically happens when D starts to *overfit* by memorizing certain datapoints in the real images with very high precision, and artificial challenges must be made to sustain the equilibrium.

The losses for the generator and the discriminator are typically the modified minimax loss. The discriminator aims to maximize the difference between fake and real images. The generator aims to maximize the discriminator's output for fake images in the direction where the discriminator aims to maximize the output for the real images. Using the typical notation in a two-class setting this can be expressed as

$$\begin{cases} G_L &= ln(1+e^{-f}) \\ D_L &= mean(ln(1+e^{f}) + ln(1+e^{-r})) \\ f &= D(G(z)) \\ r &= D(x), \end{cases} \tag{2.8}$$

where f and r are *logits* for the generated image $G(z)$ and training image x, respectively. This loss is also known as the logistic loss. Logits are the numerical output of a neural network, defined as unnormalized probabilities. The loss for the G network is defined by the fake logits while the loss for the D network is defined by the logits of both fake and real images. The sign of the logit is important. Importantly, we can see that the goal of discriminator regarding real images is the same as the goal of the generator for the fake images. From the signs it is seen that the D will drive the training images to have values that are over zero and fake images to have values that are below zero, to minimize the loss D_L. Simultaneously G will drive the fake images to have values over zero, combatting the discriminator over the same images in this adversarial process. A simple diagram version of GAN training is given in Figure 2.5, where the dashed line is for the generator (G) and the solid line for the discriminator (D). The generator uses the discriminator, and the discriminator uses the generator, but they only update their own weights.

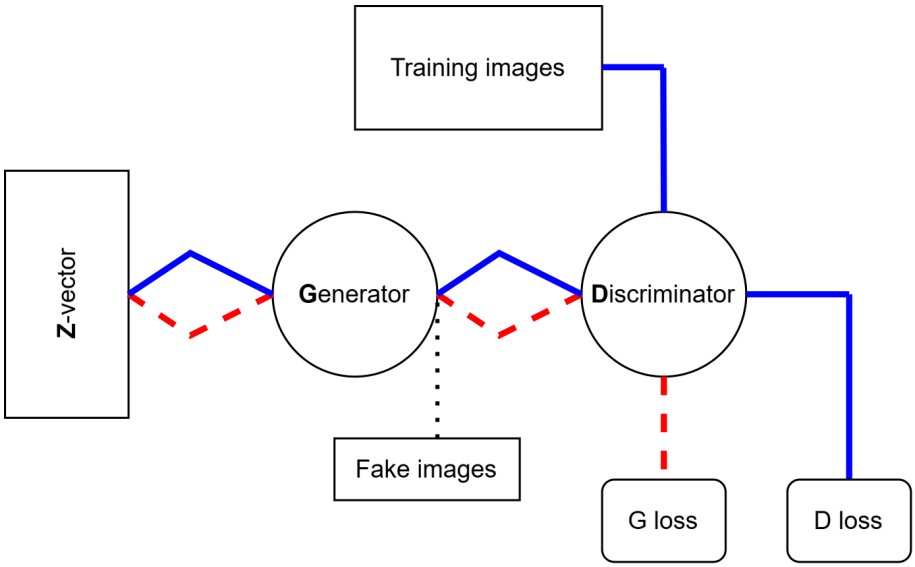

Figure 2.5. *A general diagram of GAN training. The blue solid line is for the discriminator while the dashed red line is for the generator training. Training G and D both use each other but have gradient updates only to themselves. They are trained in subsequent steps for as many steps as necessary. In many implementations they both start from a random latent vector Z to produce the fake images.*

At the very beginning of GAN training, the generator (G) has no idea what kind of image to output and generates images of pure noise. The discriminator (D) does not have any ability to discriminate either. However, as the discriminator is typically built like a classifier, it learns very quickly in the beginning. The differentiation between the training DMS images and images of complete noise is not hard to learn for a neural network classifier. The D will learn to map the fake images into negative signed logits and real images into positive signed logits as the loss function 2.8 will minimize when they are in those directions. The initial large gradients will change the weights in D towards the direction that will minimize the loss. This is the basis for the classification ability in any general deep learning classifier but also for the discriminator as it acts as a classifier here.

When D drives the fake logits into large negative numbers, the loss for G seen in Equation 2.8 will rise due to the sign in the exponent. This will in turn make the G have large gradients and update its weights with larger swings. It uses the logit-information provided by the D to find how large of a learning adjustment to make. If the initial output of the G is simply a noise image, the adjustments will move the weights toward a direction with less loss. As D has the least loss in a direction with similarity to real images, the G will learn to move its weights in a direction where it is able to produce them [7]. In what are called 'progressive' GANs [56], the G starts the image from a small resolution $4x4$ and grows into larger such as $512x512$ typically using upsampling in the convolution layers as opposed to downsampling that is used in the D. Updating these layer weights with the given losses is the basis of the synthetic image production as the fake images will start to resemble the original images to satisfy the minimax game.

The back and forth of G and D continues as long as necessary. In this thesis the discriminator learned in the first few hundred seen images with a very high confidence to differentiate between the fakes (noise) and reals. The generator caught up in about $20000-80000$ seen images (typically abbreviated as 20 kimgs and 80 kimgs) and they reached an equilibrium. As the training continued, the discriminator started to overpower the generator again by learning to look for only a very small group of pixels that it remembers from the training data. The D then with a very high confidence differentiated between real and fakes, unless the fakes had the exact same copy of those pixel values. This is a known issue in training when the discriminator starts to overfit and in general much of the optimization methods relate to making the discriminator dumber without affecting the quality of the generator outputs [57]. As the G learns based on what D focuses on, overfitting only on a few images or pixel areas provides the G also with bad learning signals and can eventually cause *mode collapse* where the generator is only producing a few modes of the data distribution as the D overfits on those. This poor guidance is one of the hardest obstacles of GAN training.

With proper constrainment, regularization, and optimization the generator will know the distribution of the data and can sample from it, and with added modifications to the generation, also around the data distribution. If the dimensions of the image are $2x32x32$, the 2048-dimensional distribution should now be in the generative parameters of the GAN and ready to be used. If the images are of cats, the entity *cat* defined by the 2048-dimensions is generated. It is later learned how the generator knows what to do when making a cat instead of a dog. The relation of the complex distribution to the concept of a cat is found by the visual intelligence of the neural network and is defined by the weights of the GAN where they can be found as representations. The representations in an X-ray image could be a "bone", "diseased area", or "background noise" and in the network, they would appear as single weight values or in groups that are updated in the way defined in this section. It has been found that certain GANs have very strong representational weight-spaces [58] which makes them an interesting subject of study. The weights can be searched for to find these representations and the generated images can be altered by changing the value of these weights.

Many important improvements since the original 2014 GAN publication have been made and a constant stream of publications still appear, especially regarding new improvements on regularization, optimization, or changes to model architecture and fitting into new specialized situations. In 2020 alone approximately 28 500 papers related to GANs were published [53]. The original GAN was completely unsupervised meaning that there was no control over the output and on the labels of images. The GAN might learn cats and dogs, but it would output them randomly. The conditional GAN (cGAN) [59] was quickly devised to include labels that control either or both the D and G. The conditioning can be a class label such as cat or dog, or a text that controls the generation according to natural

language orders as in the last example of Figure 2.4. Examples of other conditioning parameters include physical laws [60], disease information from literature to synthesize diseases in MRI images [61], or another imaging modality for translation between for example MRI and CT images [1]. Besides conditioning, much of the improvements have focused on the very finicky training of GANs [62]. Recent large models like BigGAN and StyleGAN often combine years of effort into a single robust, generalizable model, that is then adapted into research literature for standardized comparison between other models and methods [47], [63].

I will now go further into using GAN as a tool for data analysis. The first topic to explore is downstream classification, where the GAN is used to create images closely resembling the original training data so that the generated images may be used together or in place of the training data in a real-world setting when training a classifier. This task is related to RQ1. The second use of the GAN is for RQ2, and it relates to the topics of semantic learning, representation learning, data compression, disentanglement, latent space, dimensionality reduction, and especially explainable AI. For that, I will explain how the generative aspect of a GAN model can be explored for semantic understanding of data to explain questions such as what makes a cat a cat or what makes DMS IDH image wild-type or mutated.

2.2.2 GAN for downstream classification

After the first couple of years of GAN research, new research on how the generated images perform in classification against or with real images started to appear in the medical community [3], [46]. The early research showed very impressive gains in classifier accuracy of up to 30% though the variance in results was very high.

The premise is two-fold. First, the idea of any augmentation has proven to be a very good tactic to improve the accuracy of complex deep learning classifiers. In the traditional sense, this includes 64+ [64] methods such as simply rotating the image, scaling it, twisting it, and so on. Some example augmentation methods are shown in Figure 2.6. The improvements that this type of augmentation yields is based on the variance-complexity balance of the neural networks, where more complex networks require a larger amount of variance from the dataset, or else they will overfit. Augmenting will better match the variance to the complexity, reducing overfitting and increasing generalizing capabilities [6]. This type of augmentation is typically always used when training a modern vision-based neural network. The synthetic images provided by a GAN differ from the traditional methods as they are very much like the original images and not simply twisted or rotated versions of them.

The second premise special in GANs is that the learned distribution they sample from can be interpolated or traversed quite smoothly. In the simple case of a one-dimensional nor-

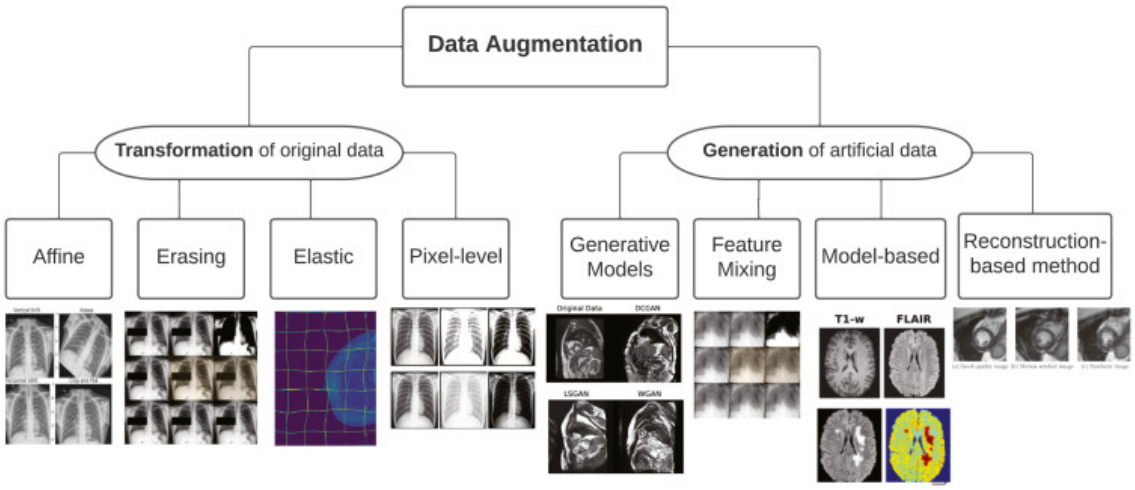

Figure 2.6. *Different data augmentation methods for medical images. Traditional methods rely on transformations of the image where the new image does not resemble a realistic version anymore. Generative models produce data that is similar to the original. Image from [65].*

mal distribution, this would mean that instead of having only a few discrete points that lie in the distribution to represent it, the GAN would be able to interpolate smoothly between the original unsmooth distribution with large discrete gaps in it and generate new points that lie in the distribution and thus provide more samples or training data. While a normal distribution could also be described analytically, for the complex case of images this is not possible. In the more complex situation where the image pixels forms thousands or millions of dimensions the generation of new samples is not easily visualized but the idea is the same. The complex distributions are smoothly interpolated, and the new datapoints should provide very useful value. Alternatively, certain distributions could be dampened while others could be emphasized. This is seen as a more sophisticated form of augmentation [3]. The task of augmenting a dataset using a GAN is also called downstream classification, as the GAN data is used for a downstream task. The exact methods such as how many images to use are not clear and vary highly in the literature [55], [66], [67]. As images are created within milliseconds, researchers often generate something between 10% to 10000% of the size of the training dataset when testing augmentation. Typically, a deep learning classifier is used in the downstream task.

A review of GAN augmentation in medical imaging that compared 105 publications between 2018 and 2021 commented in favor of GANs that "almost all papers with downstream application tasks explicitly stated that the dataset augmented by the proposed method could better improve the performance of downstream models compared with existing augmentation methods" [55]. A review on data augmentation in computer vision [67] complemented this view by seeing that synthetic data from a deep generator is especially useful in the case of exorbitantly costly data and when used together with traditional methods. They also emphasize the use of unlabeled data to train the generator as well

as the common problems when training deep generators, especially regarding the lack of good evaluation metrics. Evaluation metrics are outside evaluators of the quality of training and are further introduced in Section 4.3.

The data in the medical field is diverse, heterogeneous, scarce, expensive, and subject to privacy regulations. Using generative models for this is idealized to solve many issues, as the data is no longer linked to the patient and new synthetic data can be generated to normalize distribution shifts. The smooth traversals allow controlled augmentation and synthesizing especially very rare data could improve the quality of datasets largely. A GAN outputs an image in some milliseconds compared to a real scan of a patient with any modality which would take significantly longer. [1], [50], [65], [68]–[73].

The negative effects of limited data also affect GANs, and it has also been studied in the last few years [65], [74]–[76]. As the amount of data is low, the GAN faces similar issues to any other neural network, and the original implementations have relied on larger amounts of data. A specific problem is the mode collapse where the generator outputs only a couple of different images defined by their modes. This may happen either due to the generator finding a small set of images that always fool the discriminator, or the discriminator overfocusing on small parts of the image providing too large gradients towards these modes. Meaningless guidance and overfitting are the main challenges in limited data synthesis with deep generative models [75].

Other issues affecting GANs are also more pronounced when the amount of data is low. There are many strategies developed for GANs regarding this. In one popular method, augmentation within the GAN training is used. The generated and real images are augmented with certain traditional methods that are included in popular GAN augmentation algorithms such as DiffAug, ADA, and APA [75]. These augmentations do not leak into the generator, meaning that the generator does not take guidance from them and start to generate these augmentations [57]. Distance metrics between images are also used to maximize the information of the training data, contrastive learning is used to further separate either classes or real data instances and perhaps most importantly, knowledge transfer is utilized to use information contained in outside data [75].

Besides providing fake images, the GAN is also very interesting for its capability to represent the data in lower or latent dimensions. The weight parameters of the GAN have distilled knowledge in them that can be controlled by changing values of those weights. Research has shown how either single or groups of these parameters may represent semantically meaningful representations such as the size of the human nose, the color of the eye, the concentration of diseased tissue, or the weather season in a landscape photo [48], [58]. These control parameters can be studied to further understand how the training dataset that provided the knowledge distilled in these parameters is explained by them. Next, I will go further into this latent space of GANs.

2.2.3 GAN latent space

The GAN has valuable middle-model parameters that are typically called latent variables, latent space, noise vector, latent code, Z-space or W-space. These are not unique to GANs, and other generative models also have them, depending on the implementation. I will use numbers and names that relate to the specific GAN implementation called StyleGAN2 from now on, as that is the model used in this thesis.

The latent variables reside in the latent spaces of the generator of the GAN. Especially for StyleGAN2 the latent spaces are Z-space, W-space and S-space and also in special implementations a $W+$-space [47], [58]. In Figure 2.7 the different spaces are shown for the StyleGAN1. The generative pipeline starts from the Z-space, which is a 512-long vector of normally distributed random numbers. This vector is transformed into the W-space through multiple linear layers as this is argued to cause a stronger *disentanglement* between the parameters. The disentangled latent space means that the latent variables that are the representations act alone and do not interfere with each other. If a latent variable would control the color of an animal's fur, the rest of the image would be unchanged. Beyond W-space, the $W+$-space and S-spaces are studied to be possibly even more disentangled [63], [77]. The S-space is formed from the W-space through affine transformations.

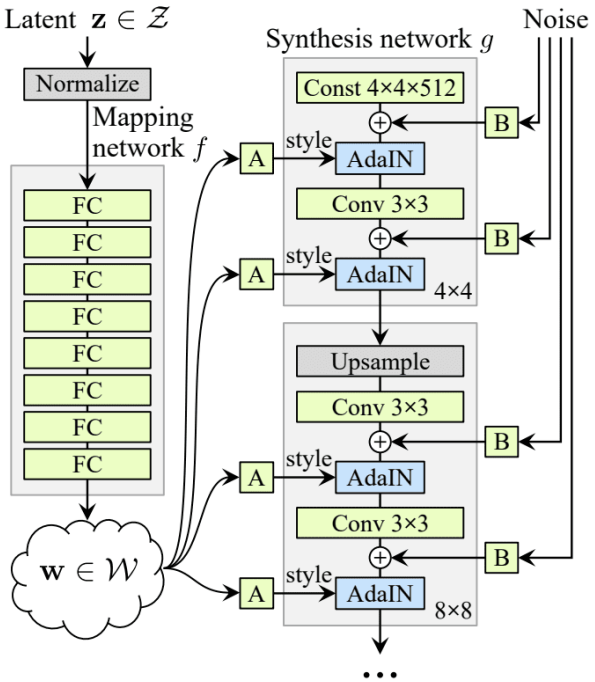

Figure 2.7. *The StyleGAN1 generator network graph. The operations inside the synthesis network are different in the StyleGAN2 version, but the Z, W, and S spaces are the same. The idea also stays the same: applying styles and noise at different resolution levels. The S-space is defined as W going through affine transformation A. All nodes in green are learnable parameters, including the addition of noise and the starting image, although those two are sometimes kept constant. Image from [78] ©2020 IEEE.*

The disentanglement and linear traversal capabilities of StyleGAN2 made it popular to use in image editing tools. Finding controllable and local representations such as the color of the eyes as latent variables through search algorithms was easy enough to have control knobs to edit images with. Tools based on similar methods are now a part of several commercial image editing software. Disentanglement is especially interesting in fields where it may be used for knowledge, such as in drug discovery [72]. Although rarely present in papers, there are evaluation metrics for discovery and disentanglement that could guide future development [79].

The simplest way of exploring the latent space of GANs is to change singular values of any of these parameter vectors. Starting from the 512-long Z-vector, we can change any value with ± 1 and see how the output changes. Often researchers will need to generate millions of images to find interesting representations. With certain techniques such as the StylEx used in this thesis, the number of generated images is highly reduced [48], [58]. Generally the latent exploration requires some specialized techniques or evaluation metrics, as millions of images are hard to go through. This latent exploration could also be used to build a more robust augmentation dataset for the task of downstream classification by finding disentangled latents that complete the distribution in meaningful metrics [77].

The whole network can also be built with the specific goal in mind of having controllable latents. The most prominent original work in GANs regarding this is the InfoGAN [80], where the authors found latents that controlled the output without any supervised labels. The dataset consisted of written numbers (MNIST), and the author had a continuous latent variable that would learn the most meaningful representation as defined by the information-theoretic concept of maximizing mutual information. In addition to the continuous variable, a discrete variable captured the different digits, such that when the value was changed $[0,1,..]$, the output of the generator changed representing a different digit. The continuous variable captured skew of the images in the dataset, while the discrete variable captured the writing pattern of different digits. The GAN not only learned the different numbers in a completely unsupervised way but also learned a variable that controls the tilt of the images, as there was a distributed tilt in the dataset. This concept is similar to clustering but much more semantically meaningful. The concept of maximizing mutual information has been applied to other generative models as well, but it is typically hard to use due to challenges in training. Other methods that also employ mutual information maximization have easier training with good results, for example, using variational autoencoders [81] or different GANs [82].

The technique named StylEx [48] used in this thesis to explore the latent space also builds the GAN in a special way, though to a lesser extent compared to InfoGAN. The basic building block is the StyleGAN2 network, with some modifications to the training setup. Especially some modification to the latent space W is required such that the exploration will be easier later without the need to generate millions of images. The goal

in most of these methods is similar: to find compressed forms of data in the middle of the model that the model itself uses to generate the final images. In addition, they should be meaningful to a human. If the model understands the distribution of the data very well, studying these parameters reveals insights into the data that might otherwise be hard to access due to the curse of dimensionality [41].

Next I will explain the StyleGAN in more detail and also how the StylEx or "Explaining in style" is used to train the StyleGAN in a specific way to form meaningful latents.

2.2.4 StyleGAN and StylEx

StyleGAN is a very popular model in the field of image generation introduced by the popular parallel computing manufacturer NVIDIA in 2018, with large improvements in 2020 and 2021 [47], named iterations V2 and V3. It combines many old and new innovations and found its popularity in research as well as general news [63], [83]. Typically, its abilities are displayed by generating very realistic human faces of people that do not exist when trained on a dataset consisting of 70 thousand human faces.

In Figure 2.5 we saw the general outline of a GAN network and in Figure 2.7 the generation for StyleGAN1 was presented. Typically, the StyleGAN2 generator starts with a 512-long vector of random variables called the Z or noise vector. Each value in the vector is drawn from the normal distribution. At the same time, for each image, a random label is drawn from the number of classes to condition the generator. The labels are mapped through a linear layer to cause an embedding for the class label [57], forming a 512-long vector from the single number. The noise and class vectors are concatenated at this point, forming a 1024-long vector. This is also mapped through multiple linear layers to strengthen the disentanglement. This part of the generator is named the *mapping network*, and after it we have again a 512-long vector now called W. The W-space is sometimes extended with $W+$-space to further increase disentanglement [58].

The image does not start from the W-vector but instead from a new $4x4x512$-sized vector. Importantly this can be either a constant or a learnable parameter vector. While the Z was always sampled from normal distribution, the $4x4$ start of the image is not. The image is formed within the *synthesis network* of the generator. In here, the forming image is changed by *styles* that are made from the W-space vector as they go through an affine transformation (linear layer). The S-space is seen after the affine transformation A in Figure 2.7.

In the synthesis network the image is first stylized in low resolution. Next, the resolution doubles from $4x4$ to $8x8$. Then it is stylized again with different styles, the resolution increases again and this repeats until we have the final resolution such as $32x32$ or $1024x1024$ or whatever is required. During this, other events such as filtering and noise

addition take place, but the basis is the progressively increasing size and stylizing in different resolutions. The styles applied in lower layers will have a more coarse effect on the final image as they are increased in resolution by upsampling the image.

The changes from $Z \rightarrow W \rightarrow S$ as well as the convolutional layers that upsample the image to higher resolution form the learnable parameters or weights of the generator. In addition, there are also *torgb* (to rgb, red, green, blue) layers that change the dimensionality from 512 to 3, although it is adjustable eg. to the value 2 that was used for this thesis. These parameters are what the generator adjusts when the loss in Equation 2.8 gets a value over zero for the generator. Eventually, after long training, the styles should represent semantical styles in the images, such that stylizing the image would only require changing the parameters of the W or S spaces to have a locally manipulatable feature named a "style". Typically, the styles in the lower resolutions are coarse blobs or brightness while the styles in the larger resolution should represent finer features such as fur color or a DMS image alpha curve. This is due to the upscaling effect of the upsampling layers. For $32x32$ images the StyleGAN generator network has 11 layers for styles and therefore there are $11 \times 512 = 5632$ unique styles that are added to each image with varying strengths.

The StyleGAN has demonstrated state-of-the-art results both in downstream classification [74], and representations [63]. The iterations StyleGAN2 and StyleGAN3 are preferred to the original, with StyleGAN2 often being a standard in research. For downstream classification, the StyleGAN2 is either trained using conditioning or a new model is trained for each class. The StyleGAN authors also released an augmentation method to train the GAN in a limited data situation named adaptive data augmentation (ADA) [57]. Although other methods have surpassed StyleGAN2-ADA in low-data comparisons in metrics such as FID and IS [75], it is still used in recent research due to the robustness and availability of the method [84]. It is noteworthy that the metrics used for comparison of low-data GANs do not necessarily correlate with the downstream task very well, as they typically only assess image quality using images of for example human faces and metrics that evaluate only perceptual quality, and not how they perform on the downstream tasks.

For representation learning, the common approach is to use unsupervised StyleGAN2 training where no conditioning is done. This is due to the idea of control-freedom trade-off, where exertion of control to the generation of the model causes loss in inversion capabilities. The model is more restricted in exploration of the semantic areas because of inherent bias in conditioning [77]. In our case, we are interested in the specific attributes that relate to a class, that is very similar to another class, but has small specific differences that are unknown and hard to visualize [18], [38]. Therefore for representations we wish to have class conditioning.

StylEx [48] is a method to train the StyleGAN2 in a specific manner to introduce new components to the network that can be used after the training to find specific styles that explain classes from the perspective of a classifier. The classes and styles depend on the dataset. Some examples of styles found by the method from a human facial image dataset include beard stubble, moustache, lipstick, and eyebrow thickness found to affect the classification of gender. In a dataset for retinal disease, the researchers found attributes for exudates, "cotton wool" and hemorrhages to be meaningful and controllable in the classification of disease.

The method is based on the idea of *counterfactual explanation*, where the statement of "had the input x been $\hat{x}$, then the classifier output would have been $\hat{y}$ instead of y" is studied. Given the example of the beard, this sentence could be "had the beard been more visible, the probability of the image being a female would decrease by 50%". To make the information interesting and useful, the change in x needs to have a semantical meaning that a human observer understands. The attributes that change a class according to a classifier might be completely invisible minuscule changes in the images, as those used in adversarial attacks, but these do not reveal the changes that explain the image to a human observer. Considering the five features we are looking for in the DMS images, we should expect to find these for the method to be successful, according to the above criteria.

The disentangled attributes that are found in the StyleGAN generator fit the criteria for the counterfactual explanation. From this basis, StylEx was formed. The classifier decision is added to the training loop of StyleGAN. A new loss function, that now also includes the classifier decision, ensures that the decision of the classifier is implicitly encoded into the training of the S-space parameters. This creates a latent space that is even more disentangled with regards to the attributes that have semantical meaning, but also are important from the classifier's perspective for class differentiation. The classifier-specific disentangled attributes will emerge into the S-space or stylespace as adjustable parameters.

The attributes are represented in the data only visually and not by any label besides the class. The StylEx method finds these *style attributes* by interpreting the labels and using the stylespace to map common styles to a label. If the styles are semantically meaningful, such as correlation between gender and the length of the beard, this information is used in training to control the generation in the stylespace in a way that these styles will especially be learned. This is the newly added attribution of the classifier to the network: the classifier gives probabilities on the image classes and exerts control on StyleGAN training in a way that the class-specific styles are more prominently learned. The generation of the image is still mostly the same as in the original StyleGAN, and only the W-vector is changed. To find these attributes after training the network, a search algorithm named AttFind is employed.

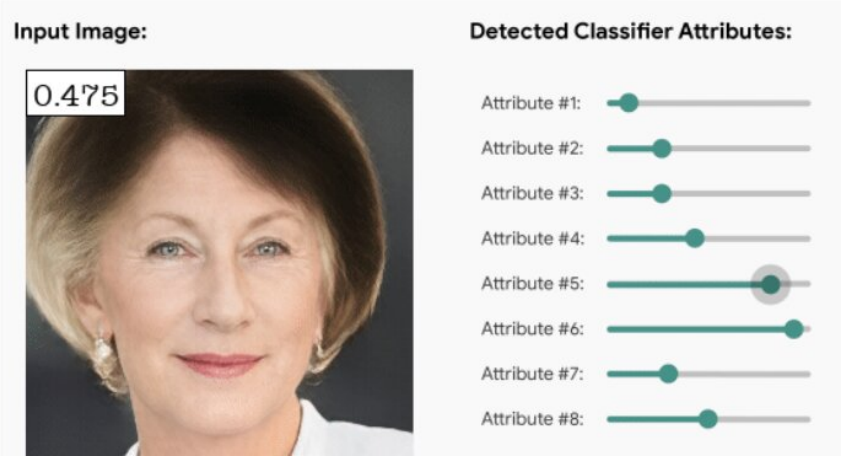

Figure 2.8. *Example of the outcome of the StylEx method. A visual tool for locally manipulatable and semantically meaningful attributes. The originally lightish white to gray hair has been turned brown by attribute #5, indicating that the brown hair is more correlated to younger age. Other attributes available in gif form at [48] are for example eyebrow thickness, smile, and face width.*

After finding these style attributes, it is possible to highlight and visualize them as the style is a parameter in the StyleGAN2 network that can be increased or decreased in value. For example, decreasing the value might reduce the presence of a beard, while increasing the value might enhance it. The original researchers provide a helpful visualization of this concept with found attributes [48], also seen in Figure 2.8. The new attributes can be used for knowledge, visualization, research, or study of biases in the dataset.

An article on GAN use for medical explainability describes the unique advantages of the StylEx method as follows: "This dataset-level explainability method offers a fundamentally different, yet complementary, approach to explainability than is typically used in medical imaging explications. Rather than answering the question "why has a certain prediction been made for a specific image?", this method helps answer the question "what image features are associated with each class?""[85]. Another article provides commentary on the possible negative aspect: "Though useful, an apparent drawback of the external module-based methods is that the module can inadvertently affect both the model performance and its explanations. For the latter case, the explanation can unintentionally be encoding the module's behavior instead of explaining the model." [86]. A state-of-the-art review into StyleGAN-based methods [63] states that the training scheme drives the latent space to capture classifier-specific attributes and the linear editing directions correspond to specific properties that the classifier searches for, such as the shape of the eye in the dogs versus cats case. In the case of DMS data, ideally this could be individual features such as individual a-curves or fragments.

The StylEx is generally a well-praised method for its uniqueness, although there are very few papers applying it besides the original. In machine learning literature this is a sign of

danger, as many computational methods fail to reproduce with different types of data. Besides StylEx there are other algorithms used for explainable AI (XAI) in computer vision. Methods similar to StylEx include saliency map-based methods such as LIME, SmoothGrad, GradCAM, and GANanalyzer, or those based on concept attribution for example ACE, PACE, ConceptSHAP, and TCAV [63], [87].

The implementation of StyleGAN2 had to be modified for both tasks in this thesis from the originals and the changes are explained in more detail in Chapter 4. In addition, the original literature provides more comprehensive explanations for the different modules and theoretical basis for the strength of the StyleGAN2 network [47], [48], [56], [57].

I have now explained the basics of both the DMS and the GAN: the data collection and data analysis methods used in this thesis. Before explaining the methods in detail, I will go through the search experience to find the right generative model for this thesis.

3. MODEL EXPLORATION JOURNEY

The exploratory process that was undertaken to find a suitable model for this thesis took a considerably large chunk of time at the beginning of the work. Researching the subject through combined effort both in literature search and experimentation was undertaken. I will go through my findings to explain and motivate the choice of methods. Given the broad scope of the topic, a complete search was not feasible. However, this chapter aims to provide the reader with an understanding of the depth and breadth of the conducted research and help them consider how this may have influenced the results.

For a long time at the beginning of this search I felt quite overwhelmed as I went through different model families and architectures. Eventually, as I learned more and understood what was missing from the models that produced bad results, and what was actually required by the model, I ended up finding the StyleGAN2 and it matched my requirements very well. Looking back, I should have started with the most popular models instead of meddling with those that fit a specific niche. The models were primarily searched through the literature reviews mentioned in this section, as well as from code libraries that included multiple models.

The requirements for the model were such that it should be able to recognize and generate very fine details in the image. For an additional challenge the images have very small differences between the classes as the DMS images in the IDH dataset are very similar. The model should also be able to be trained on a low amount of data. It should be available as code and the implementation should not be too specific such as being restricted to images with three color channels. It should be able to produce multiple classes of images that can be used for downstream classification, and finally, in the ideal situation, the same model would be able to be used for explainable AI purposes. Also preferably, the computation should not exceed absurd limits.

Initially I investigated broadly with different model families. Modern generative algorithms are divided into 2-5 model families, depending on the method of taxonomy [54], [62], [74]. Generally, the ones that stood out the most are Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Diffusion Models (DMs). In many domains as of 2024, the diffusion models are state-of-the-art, although GANs are more prevalent in medical research. As progress has been remarkably rapid in the last decade, I will also

mention the years of publication in this section.

A review from 2019 [3] suggested that most studies utilize a separate generator for each class. In a 2023 survey [74] for the case of generative model in data constraint environments (GM-DC), the authors found that the models employed were 85% GANs, although diffusion models had achieved similar results in metrics. They also noted that the methods developed for GANs may not be applicable in other models due to differences in the generation of images. A comprehensive survey into diffusion models in medical imaging from 2023 [43] suggests that a latent diffusion model was shown to outperform GAN-based methods in downstream classification task. Conversely, they also concluded that the representation space in VAEs and GANs is still superior due to the design around optimizing for it. This is an important consideration for the second task of the thesis, where the representations of the semantic space are studied for informative conclusions about the model or the data. They conclude that diffusion models are still in rapid development as compared to GANs that have had a few years of running advantage. A "critical assessment" into X-ray data of pneumonia from 2023 [66] found GANs to significantly outperform more modern diffusion-based models with small datasets in downstream classification task.

With provocatively titled papers such as "Diffusion Probabilistic Models beat GANs on Medical Images" in 2022 [88] or "Diffusion Models Beat GANs on Image Synthesis" in 2021 [89], the choice is not clear. The arguments are also against diffusion models with papers such as "Beware of diffusion models for synthesizing medical images" from 2023 and an update in 2024 [45], who show that diffusion models might remember training images raising the risk for privacy and GDRP issues. According to the 2023 survey on diffusion models in medical imaging [43], some specific problems are better tackled with diffusion models. The often-cited argument against GANs is the difficulty of training the adversarial process due to failures in convergence, whereas the diffusion has a very stable training objective with iterative sampling [90], [91]. The training of a diffusion model is considerably easier as the denoising process can be repeated infinitely until at least a hallucination of a signal is approached, while the GAN may fail to train completely and result in a broken model. This subsequently entails more computational cost with the diffusion, though modern implementations have improved from the original slow processes by focusing on improved strategies for the solvers and sampling as explained by a 2024 diffusion survey [91].

Both GAN and diffusion and other generative models try to sample from the implicit distribution, though the training objectives to achieve this are very different. Diffusion models are trained to adhere to maximum likelihood estimation (MLE), while the GANs minimax is a path to Jensen-Shannon (JS) divergence [54]. Most real models are approximations of these objectives, or hybrids such as Stable Diffusion that use adversarial training in one part of the model but are mainly diffusion-based. However, this difference in objective may have implications for the capability of distribution matching. JS divergence measures how

different two distributions are, while MLE is a method to find the best parameters for a model, so it fits the data with the closest likelihood. The end-result of these differences is not completely clear and is very specific to the dataset used to train the model.

The diffusion process starts from noise and iteratively changes the image until the estimation to the real data is very close (MLE). This iteration may be repeated infinitely for the same image. The GAN generates the sample in constant time in one go. The GAN learns two networks in an adversarial process such that the JS divergence is found, and the distribution of the generated images should equal the distribution of the real images. In the case of infinite samples, MLE and JS divergence are equal, but the statistical implications in a real-world setting might end up benefiting one method over the other depending on the case. MLE-based methods have been the main interest in the last decades as for example the LDA classifier used heavily with DMS data is based on MLE. This uniqueness that is achieved through adversarial training explains in part the interest in GANs.

The VAE has a probabilistic foundation and is more often used for latent representations than for reconstructions or synthesis [54]. The latent space of VAEs is very easy to explore as it is modeled in probabilistic terms. The latent space is also encouraged to have such representations that the input can be rebuilt from it. Initially this was very alluring for me, as the Gaussian-based latent representations should ideally fit the DMS curves very well. If we imagine each representation holding a single curve, the probability for a curve belonging to a class would be a very strong basis for classification. Then sampling combinations of those distributions could be used to synthesize images. Although VAEs were not ultimately chosen beyond initial explorations and testing, they are still very interesting in this regard with for example Attri-VAE [81] being a strong contender for the RQ2 of this thesis. However, it too is typically always trained with a large number of images with higher resolution.

In literature it is argued that VAEs are most useful for representations and not suitable for image synthesis, especially with limited data [75]. For diffusion, the literature is unclear whether they are capable for the data-constrained environment and handling issues in remembering training data [45], [75]. Remembering the data also affects GANs as the discriminator might be overtrained to remember the training data and might end up providing no guidance to the generator [75].

The VAE learns to make a latent representation that probabilistically represents the input image. DMs use a stochastic (random) process that transforms complete noise to the final image also using a probabilistic approach, as each denoising step involves calculating the probability of the data to be in that diffused state. The probability of a match is defined by the variance in the dataset, and with low variance, it might cause worse issues compared to the JS divergence method of estimation in GANs. Most benchmarks in recent review papers for the case of low-data image generation are dominated by GANs, with some

diffusion models appearing [74], [92]. The most popular models currently for general image generation are diffusion or hybrid based which combine parts of different models.

With the lack of clear literature on what models to go with, I concluded myself to try out the different models and see how they behave with the data.

I started with the VAEs using the PythAE library from 2022 and the latest updates in 2024 [93]. A very well written library with a large collection of models helped experimentation and debugging single models to not take the majority of my time. Although the specific requirements set in this thesis still required quite a lot of debugging and writing matching code, I was able to generate working images the same day I started. Many of the models in the library performed adequately, though not impressively. As is usual in VAEs, the lack of detail in the images caused them to not be useful in the downstream classification task [92]. The whole noise area in the fake DMS images had almost the exact same value for the pixels in it so minimal local fine details were ignored by the reconstruction phase in the model. The training went well, and the losses converged to values expected from literature. Results were still not impressive as the images seemed to lack fine details. It is somewhat unclear as to what the cause is, but this is a commonly seen result using VAEs, especially in the case of low resolution. After disappointing results even after some weeks of small improvements, parameter searching, and model optimization, I moved on to try diffusion models. A very good result of using only 50 MRI images for the basis of data augmentation has been achieved with VAEs in 2022 [94]. This same method did not yield good results for the DMS data as tested with the PythAE library implementation.

For diffusion models (DMs), I also started with a code library that included ready-made models. Using the MONAI library extensions from 2023 developed for generative AI applications in the medical field [95], I was able to get a denoising diffusion probabilistic model (DDPM) running in hours. The DDPM is the common choice for any diffusion task, although the library features other models as well. The experience was much better compared to VAEs, as I was able to generate images that had more detail on a test dataset included in the library. However, as was expected, the computation took significantly more time. As an estimate, training a single model took around 10 times more time for me using the DDPM compared to a VAE. This would mean that a single test training took 10 hours. I did not optimize this and likely could have reduced it. However, as the initial results for the downstream classification when using DMS data were also inadequate, likely due to the small amount of data, continuing was very discouraging. I tried a dataset with similar small resolution images, the MedMNIST v2 dataset made in 2023 [50], for its easy access. With MedMNIST, only after having over 3000 images the results were good, indicating that the diffusion model was not performing well due to the data constraint. It is likely that with enough effort the diffusion would have similar or better results as indicated by recent literature [75], but the intensive computational requirements and still an interest in exploration led me to move on during and after diffusion and VAE model exploration.

In the times between testing VAEs and DMs I also tested GANs. As with VAEs and DMs, I ended up using a library that had a collection of models. Many other libraries were tested in addition to the ones mentioned here. From this experience, I would conclude that it's best to pick something that: 1) has had a recent update, 2) delivers a paper explaining what the library includes, 3) the code is easy to read and 4) includes models that are approved in the literature. It is tempting to pick a model that a single paper indicates to be the best-of-the-best. Unfortunately, these may often disappoint when applied to data of different kinds and the time spent adapting that library to your data is time that would be better spent elsewhere. Adopting a specific model should come after trying with a more general one and learning what is missing.

The GAN library of choice, and the final library of choice, was StudioGAN with the latest update in 2022 [62], not to be confused with StyleGAN. StudioGAN provides seven GAN implementations with other adjustments relating to regularizations, optimizations, objective functions, conditioning, and so on. What made this the best choice for me was a combination of the previously mentioned factors. The library was well written: it was easy to understand and to learn from it. It included many models, mainly those that have been generally deemed to be state-of-the-art, and as mentioned, the GAN is still seen as the best choice in literature. The addition of different regularization and optimization methods proved very useful to try advanced improvements on the models. Multiple augmentation methods allowed me to test between APA, ADA, and DiffAug.

The final decision was made after the initial runs with StudioGAN running AC-GAN had better results with less time compared to any previous VAE or DM efforts. Although I still moved from AC-GAN to StyleGAN2, I was very happy to find proper results after a long investigation with many disappointing hours of waiting for the compute to finish. At this point I had spent some months only learning the basics and experimenting with different models. Each training session would teach me something, but they also took up to a day of waiting.

The choice between StyleGAN2 and other prominent models such as BigGAN, WGAN, or AC-GAN [76] came from experimental results. A large problem with many models was that they started introducing holes in the image at some point in training. This is often seen as a problem of bad sampling of the random variable and is fixed using the truncation trick. However, in my case this did not help. I also tried gradient clipping to see if it was a problem of exploding gradients, and while this helped, and the training went much better, it did not completely remove the problem. Even in good training runs the results showed poor variation and the training curves may have indicated a mode collapse. Commonly during the training, a large drop in discriminator loss was observed. In contrast, training with StyleGAN2 showed good convergence in learning curves, no clear visual problems in the generated images, and even the downstream classification task was successful for the very first time. StyleGAN3 was also included but the difference to StyleGAN2 is not

beneficial for the tasks in this thesis and it was not used beyond initial testing.

As I learned more about the issues in training, I went back to the other models to see if the earlier issues were simply caused by my inexperience. Even then every time StyleGAN2 still came on top even when applying tricks and tips learned during my research. The other models were ever barely useful in downstream classification. This could also be the model architecture, where the StyleGAN2 implementation might have been more accustoming to the small resolution of $32x32$, as most models expect larger images, but StyleGAN2 was also trained on datasets with smaller resolutions. At some point I decided to finally only focus on StyleGAN2 as the model for this thesis. Anecdotally, AC-GAN seemed most common in literature from 2016 to 2020, but from 2020 onwards StyleGAN and other models seemed to have taken over. AC-GAN was especially popular in the task of downstream classification. The classifier head part of AC-GAN can also be constructed into the StyleGAN2 architecture. While experiments showed that this hybrid had better capability than the AC-GAN architecture on its own, the results were not impressive enough on this dataset.

One final consideration was whether to train the model for each class separately or a single model for all classes. This is quite a large difference. A single model for each class does not need to be conditional and this could improve the capabilities of the model. However, from testing, it seemed that the classes were too similar to be trained as separate models. The inclusion of both classes helped train the single model toward more separate images for each class as compared to the two-model solution. This idea could have been tested further as much of the literature uses a different model for different classes [3]. However, the conditional single model is equally used especially with StyleGAN implementations. Using this ideology I also experimented with methods that emphasize intra-class similarity and inter-class distance even further using the ReACGAN included in the StudioGAN library, but the results did not improve the differentiation of the classes.

Some researchers also put less emphasis on the model compared to the dataset, as the dataset is the source of the information, and the model families may be raking in the hype too much. An engineer from the AI startup OpenAI, behind the popular generative model family ChatGPT, commented in 2023 that the "it" in AI models is the *dataset*. In his view, all the modern models already approximate the dataset to "an incredible degree" [96]. A 2022 review article [1] thought on the subject, especially with federated learning in mind, that "large, well-designed, well-labeled, diverse and multi-institutional datasets drive performance in real-world settings far more than model optimization." Therefore, too much time spent on model optimization might be better used on dataset curation or using more data from other sources.

Similar to this, no conclusive decision was reached on the best decision between all the models in this experimentative phase. Each model family has certain advantages, but

GANs were generally the best for this task and also appear most in literature. Diffusion models are newer and might overtake GANs, but recent literature shows that this might not necessarily be the case. The large effect of computational limitations on this phase of the thesis is discussed in Section 6.3.

The choice for conditional StyleGAN2 is still popular in literature for the downstream classification task [84], even though the field generally moves very rapidly, and it is a 4-year-old model [47]. The model is robust and competes favorably against more specific implementations on many GAN-related tasks [74], [75]. Choosing a robust model is generally a good idea. The inner workings of the model were further discussed in Section 2.2.4 and the implementation details are given in the next chapter.

4. METHODS

In this chapter I will go through the details of the GAN implementations. First I will briefly introduce the dataset used in this thesis. Then I will explain how this and other types of images are typically processed before they are used for GAN training. Then I will explain the detailed implementations that were used to train the models that will provide the synthetic images for quantitative and qualitative analysis. The analysis should answer the two research questions of the thesis: can we generate useful synthetic data, and can we explain the data with the AI?

4.1 IDH dataset

The dataset used in this thesis is related to a mutation in isocitrate dehydrogenase (IDH) enzymes 1 and 2 in cancerous brain tissue. IDH is a crucial enzyme in the Krebs cycle, and mutations in these enzymes are highly correlated with the overall survival of the cancer patient. The main theory behind this is that the mutation changes the activity of the enzyme in such a way that it now forms D-2-hydroxyglutarate (D2HG) instead of the supposed a-ketoglutarate (aKG), and D2HG is associated with hypermethylation of genetical molecules. [4], [5]

This dataset was originally produced to study the ability to discern between IDH wild-type (normal) and IDH-mutated brain tissue samples using the DMS [4]. This knowledge would support decision making in clinical setting as patients with IDH-mutated version have better prognosis independently of the type of cancer [4], and therefore a more invasive procedure would have a better outcome for survival-benefit. For IDH wild-type the prognosis is bad and the preferred way is to operate only minimally to minimize risk to current neurological function. An important aspect is also that even with frozen histopathological examinations the IDH is not well identified motivating for a new solution [4]. The dataset was produced in such a way that the DMS measurement would take only 13 seconds. This short measurement time could enable real-time guidance to the surgeon if the DMS sensor is used in the surgical operation room as a screen or a simple number-based system. An example image of this data is provided in Figure 2.3. With the human eye, there is no discernible difference between the IDH classes without any filters or contrast adjustments applied to the image, and a machine classifier is required.

The dataset consists of samples from 22 patients collected during neurosurgical operations between 2018 and 2021. The collection of data took such a long time due to the rarity of the IDH mutation. The mutation was labeled using immunohistochemistry. The samples were stored in -70℃. After storage and transport, they were cut into four equal sizes. Visible blood was carefully rinsed from the samples before placing them in well plates containing 0.18 mL of agar.

The samples were then incised with a custom-built computer-controlled CO_2 laser evaporator four times for each sample with 1 mm gaps between incisions. The vaporized sample was transported together with carrier gas to the DMS (IonVision, Olfactomics Oy, Finland) inlet. A total of 1200 data points were measured. The resolutions for separation and compensation voltages S_V and C_V were 20 and 60, respectively. The total measurement time for one incision was 13 seconds during which 250 laser pulses 2 ms long each were applied to the tissue specimen to provide a stable sample stream for the sensor. During this team all the 1200 points are measured. As the 22 samples from the patients were first cut into 4, then each part was also incised 4 times, the total number of DMS images in the dataset is $22 \times 4 \times 4 = 352$. Therefore, the final image dataset has the dimensions of $(352, 2, 20, 60)$ for (*sample, polarity dimension,* S_V*,* C_V). The two polarity dimensions are for the positively and negatively charged ions, as the sensor is capable of measuring both simultaneously. Put simply, the dataset consists of 352 images with 2 color channels and an image resolution of $20x60$.

4.2 Data processing

Most neural networks require some form of preprocessing of the data. In addition, data analysis typically involves exploring the data for any obvious signs that might help analysis. The data coming from the DMS sensor is in arbitrary units, resembling the current of the ions over the capture time for each pixel. In the IDH dataset, the minimum value was -309.3 while the maximum was $+412.3$, with negative ion channel having mostly negative values and positive ion channel positive values. A typical RGB image is limited to $[0,255]$ integers, while the IDH data is in float format. The distribution of the DMS data is also very heavily peaked around the median, as most of the image is in areas of very low activity, with background noise being the dominant signal. These factors create some unique challenges with respect to the numerical preprocessing of the data for a neural network.

The first step of my actual work was to process the data. The data comes from the equipment in a JSON format, where the measurement values are included with other sensor values and instrumentation parameters. The measurement parameters that define the 2D axis are the voltages used to control the separation voltage (energy) and the compensation voltage (filter), such that the image is $20x60$:

$$
\begin{aligned}
C_V &= [-2, 10] \\
S_V &= [200, 1000] \\
C_{steps} &= 60 \\
S_{steps} &= 20.
\end{aligned}
$$

Some interesting system parameters include temperature measurements, humidity measurements, pressure measurements, and failsafe indicators. In the original paper [4] it was suggested that some measurements had a linear rise in temperature, and they used *detrending* to reduce this. As this had a considerable effect on the results, it is further discussed in Section 6.1.1. Besides the temperature, other parameters were explored with visual exploration by plotting their change over the whole measurement sessions of the 352 samples that lasted around 6.5 hours in total. Although an individual measurement took only 13 seconds, the total from 352×13 is much less than 6.5 hours, as there were gaps between the measurements.

I explored the parameters for any clear trends either in specific wells used, or if there were clear correlations between system parameters and intra-subject values for the image intensity in the most intense points or the mean of the image. As the correlations were weak, I decided to not use the system parameters for anything after this. The most important findings were that the pressure and temperature increased over the whole set of measurements, while humidity decreased with a large amount of variation especially when changing the measurement wells. There was an especially large trend in the first 64 or so measurements that was also indicated as an operator comment in the JSON file.

The humidity is especially important, as the water clusters are highly relevant for the main features of the DMS. This bias in the dataset due to humidity trends is reduced as the measurement of the samples was done in random order, such that the classes were not measured in sequence, but overlappingly with four measurement sequences corresponding to the four slices of each tissue sample. The differences between the four measurements of a single tissue were also studied, and although no clear trends were found, it was found later by experimentation that averaging the four DMS images by dimension largely increased the accuracy of classification. The changes between successive measurements could be due to the filter not completely emptying, or temperature and humidity effects causing leftovers. A possible use case for a GAN is to train it conditionally on these environmental factors and then produce a dataset where these factors are normalized to reduce distributional shifts in datasets.

The JSONs were read in Python. Python is used in nearly all GAN implementations with the availability of very well developed libraries that are also backed by large companies such as the relationship between PyTorch [97] and Meta Platforms Inc. After initial failures in recreating the results from the original paper, I noticed that the JSONs were in reverse-

time order and reversing the measurements resolved my confusion. Although I could not replicate the exact results from the paper due to missing code about the implementation, the general proximity of accuracy gave a clear indication that the data was the same.

The values were saved in NumPy fp32 format. An important lesson learned was to keep control of the format. Even though the precision of this numerical format is enough for these numbers, this definition might carry on to later handling of the data where the precision is not enough and is not unfortunately automatically converted into fp64. Especially when passed on to the data analysis library SciKit-Learn [98], the numbers from np.fp32 do not get converted into np.fp64. At one point in the project this caused a lot of headaches as I received very poor and unexpected results. Although I later would have to convert them from NumPy to PyTorch, the NumPy format was useful and the conversion to Torch tensors was done before GAN training.

The actual preprocessing has two important steps that should be defined clearly: the resizing and normalization or standardization of the data. Although both transforms result in loss of information in the data, they can also increase accuracy because they act as filters that may behave nicely with the data.

Resizing refers to changing the dimensions of the image. The dimensions of the data are in format $CxHxW$ for channels, height, and width. As explained before the original values are therefore $2x20x60$. The motivation for resizing is in the convolutional kernels of the neural network. For a typical neural network with convolutional kernels, the square size is used for many reasons [6], mainly to simplify the process. Thus, the data was resized into $2x32x32$. Another option was to copy the image three times such that an intermediate $60x60$ was formed and then resized into $64x64$. Although this would have less distortion on the aspect ratio, it was deemed from experiments to have no effect on the results, and it had a larger computational time. Interestingly though the GAN also learned the triple-copied image with very clear borders as well.

Normalization or **standardization** in this context refers to setting the data to be in a fixed range such as $[0, 1]$ or $[-1, 1]$ for the former, and setting mean μ to 0 and standard deviation σ to 1 for the latter. This choice has broad implications for any downstream processes and is a subject of larger discussion in the machine learning literature [6]. Multiple preprocessing methods were tested and are also reported. The original implementation of StyleGAN2 normalized the data into $[-1, 1]$. The activation function of the convolutional layers does not necessarily need this normalization, however. For typical uint8 RGB images this is easily implemented by dividing by 127.5 and then subtracting 1 to transform $[0, 255]$ to $[-1, 1]$. For the DMS data this is not as simple, as each image has a different maximum and minimum. Strategies with normalization had repeatedly worse results, so it was excluded as a preprocessing method. By normalizing into $[-1, 1]$ the network was in fact harder to train compared to doing no preprocessing at all.

The hypothesis is that standardization should have the best results due to the nice numerical behavior that is typical in a neural network, where the weights lie on a similar distribution. After standardization we will typically have

$$\mu = 0$$
$$\sigma = 1$$

for each image and each channel separately. Then, for any image and for either channel the mean value of the $32x32 = 1024$ pixel values would be 0, and the standard deviation 1. This is done using the standardization equation

$$z = \frac{(x - \mu)}{\sigma}, \tag{4.1}$$

where the mean and std are for the original images and channels, x is the original image and z is the transformed image.

The other preprocessing methods used in this thesis include doing nothing and standardizing into the training set mean and std. Depending on the method the generated images may have less information about the original distribution. For example, a division operation with a fixed value in the preprocessing can be reversed on the generated images with multiplication. However, if each image is standardized using Equation 4.1, then the generated images will all have the same mean and standard deviation. Therefore, while normalization is reversible, standardization is necessarily not in the case of generated images. The choice of preprocessing is important when considering the downstream task. If the task is to simply use a classifier, having the generated images in standardized form might be useful.

A typical range of values in the data as a histogram is shown in Figure 4.1, where we can see that most values reside very near the mean 0. The two different peaks represent the background noise coming from the electronics for negative and positive ion channels separately, as differences between the operational amplifiers cause different values for background noise [27]. The most intense peaks in the images have values of 50 to 350 in intensity units.

Other preprocessing approaches such as using a histogram to find out normally distributed noise have been used earlier in DMS data handling [99]. However, the standardization method used here had the best experimental results in the convergence of the GAN.

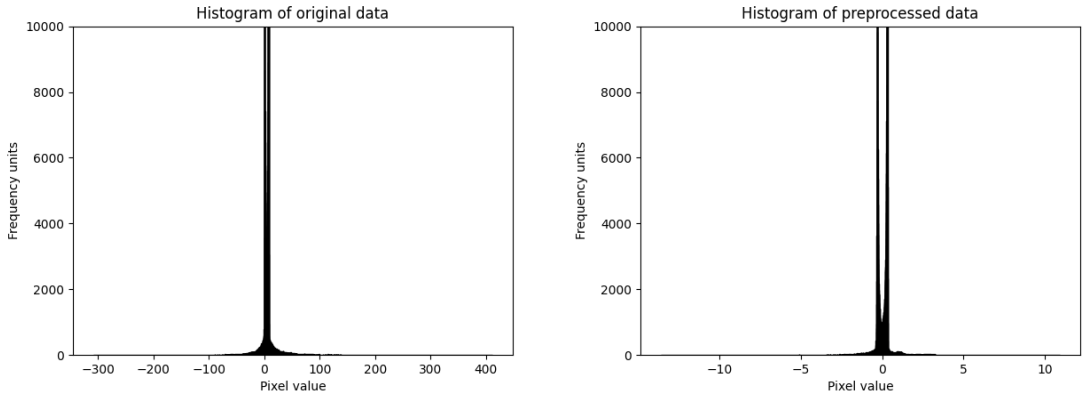

Figure 4.1. *Histograms of original and standardized data. We can observe that the standardized has much lower min and max values while both are mainly concentrated in a small area. The y-axis is cut off as the largest peak was around 60000. This highlights the difficulty of the DMS images as most of the information is concentrated on a very small area that is thought to be mainly useless.*

Alternative methods such as getting the logarithm for each row individually or contrasting the image with sigmoid were also explored on how they change the GAN training. Although training convergence was generally achieved with most of them, the evaluation metrics relating to downstream classification were consistently worse. It is likely the best idea to find a preprocessing method that minimizes information loss while providing data that behaves well in the network. Standardized data is also often no longer strictly necessary as modern neural networks include standardization layers within the model itself [47], [100].

At this point, the data is prepared for the model. The simplified data pipeline from tissue to results is presented in Figure 4.2. Next, I will go on to explain how RQ1 was answered.

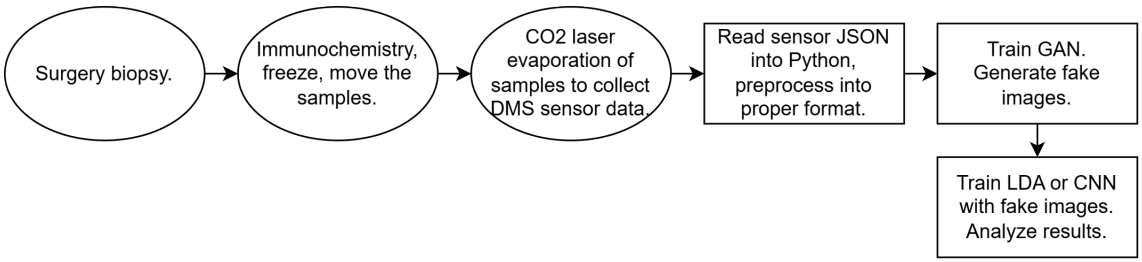

Figure 4.2. *The total data pipeline from the collection of the samples to getting the results. Each data processing step should be reproducible for the data analysis results to be meaningful.*

4.3 StyleGAN2 for downstream classification

Here I will explain the methods used to answer the RQ1: could synthetic images be useful in training a classifier for DMS data? I will explain some of the main model parameters considered and go over their importance to model training. I will also bring up the topic of metrics and their relevance in how they guide the decision on when to end the training. After introducing the metrics and explaining how the reported results were obtained, I will explain a method I used to change the generator output to have a more balanced generation regarding class balance using latent traversal.

The basis of augmenting data is in modeling the original data distribution and filling gaps or flavoring it in useful ways. The original dataset forms a distribution that the generator learns to generate as it tries to fool the discriminator by producing images that the discriminator wouldn't be able to differentiate between real and fake. Using the generated images to augment the original set of data would ideally result in a more complete set of original data, or alternatively having similar enough properties that they are useful in a downstream task. The challenge is to know when the augmentations are useful instead of harmful, and how to train a model that provides the former rather than the latter.

Optimization methods, loss function, regularizations, and hyperparameter values were chosen through a mix of experiments and grid-like search. As the number of hyperparameters in a very complex model such as StyleGAN2 is in the tens or hundreds and exploring each takes hours of computational time, the overall exploration is generally never conducted [57]. For example, only using two hyperparameters, the batch size and learning rate, the exploration of five values for each would take 25 training runs and approximately 50+ computing hours. With 10 hyperparameters, this number would be around 2230 years of computation for only five values per parameter. The process of finding final values is described as "graduate student descent" [101], where the exploration stops taking place when the results are adequate. A proper exploration into the hyperparameter optimization would use AutoML toolkits to finalize the modifiable parameters of the model, where the search is done using Bayesian optimization methods and heavily distributed computing. Although the parameters and the hyperparameters define the model training, generally only a few of them are properly explored, and the rest are kept the same as in previous literature.

Some of the main considered parameters and hyperparameters of the StyleGAN2 model are listed in Table 4.1. Batch size is how many images are used for each training step. A higher number generally means that the training is more averaged and single images don't swing the weights too much. Although StyleGAN2 implementations typically use a very high batch size, I did not see a significant difference between the values tested. Higher numbers resulted in memory issues and reduced training speed due to model weight movements in memory. Lower numbers were searched and 4, 8, and 16 had

Table 4.1. *Non-exhaustive list of the most important parameters considered for StyleGAN2 training. Generally, most parameter exploration had minimal impact while some proved a necessity for the training to work.*

Parameters	Final value	Values tested
Batch size	4.	2-64.
Learning rate	Default.	0.1, 0.5, 2, 5, 10 times default.
Mapping dimension	2.	1, 2, 8.
Generation noise	Default.	No noise on final layers.
LeCam regularization	Not used.	Use/don't.
ADA	Used in some runs.	Tested target p and kimgs.
DiffAug	Not used.	Tested with and without ADA.
APA	Not used.	Similar to DiffAug.
Lazy r1 regularization	Gamma = 0.01/0.1/5.0.	0.0001 to 100.
Training length	100k steps.	20k to 500k steps.
Loss function	Logistic, default.	Wasserstein, hinge, least square.
Truncation	Not used.	Between 0 and 1.

similar results. The learning rate was also tested. A higher learning rate speeds up convergence. In the final training, I however used the original one, as it still was more stable than the higher ones. Mapping network depth had a large effect on training and the guidance from [57] to reduce it was used. Generation noise is often taken out when producing the final images to provide cleaner images. This had a negative effect on downstream classification and therefore the noise was kept in. LeCam is a very strong tool for regularization of the discriminator in the case of low data. While it did show strong convergence in training, the downstream classification task did not necessarily improve when using it, and it was better to include fewer parameters. It was a very strong candidate however and could have possibly improved the results.

Adaptive data augmentation (ADA) was explored highly and was used in some of the final models. In addition, the augmentation methods DiffAug and APA were used. DiffAug and ADA were closely equal in results but eventually ADA was chosen. Perhaps the most important parameter was the R1 regularization. The value of γ in R1 highly guided the network training. With a higher value, the discriminator overlearned later, and this allowed for longer stability for the generator to learn new values. However, this also meant that the generator was much more restricted and less 'exploratory'. Training length was varied highly but was eventually set on 100k steps or 400 kimgs due to the balance between computational constraint and end-result quality. The loss function was varied but mostly when using different networks besides StyleGAN2 and logistic loss was chosen. Wasserstein loss was tried and is often used but was deemed unnecessary. It provides a more stable GAN training. Truncation was also explored highly. It is a method to "truncate"

the output of the generator. With no truncation, the generator is freer to explore and with higher truncation, it should provide a more stable stream of images with fewer features. Although it was interesting, its use in downstream classification was unclear. In total an estimated number of 100+ parameters and hyperparameters were explored, mostly to learn how they affect either the learning or the results. Most were eventually ignored as finding the optimal set of parameters was not the general goal of the thesis.

Code modifications were required. Large modifications in the StudioGAN training loop were done to visualize the results and losses during the training. Some intermittent values were extracted to follow the gradients and weights of the layers. This also helped in understanding training divergence, memorization of the discriminator, and exploding gradients. New functions were written for data handling, data visualization as well as for model saving. The original code structure provided in the StudioGAN library was very useful, as any changes were easy to implement on top of it. Implementations for evaluation metrics were also rewritten from other sources to be much more computationally efficient. Especially moving from CPU to GPU processing of the evaluation metrics made their computation up to five times faster. PyTorch profiler and cProfiler libraries were used to find bottlenecks in computation during the thesis, as the limit of computational power was always present. Although the code is in Python, all the low level instructions are written in C and CUDA when using deep learning libraries such as PyTorch. Therefore using Python does not generally make the training any slower, even though it is generally considered a slower language for computation applications.

A significant error was also found in the code, which also appeared in the original PyTorch code from the authors of StyleGAN2. They described in the paper [57] how the conditioning of the discriminator is done using $norm(embed(z))$. In the code, however, they erroneously had the embedded labels mapped through eight linear layers. This had quite a significant effect on the training when explored through model weight progression. The variable fused_modconv boolean flag was changed to True as I saw no reason for it to be false, and this was also questioned on the code repository by other people. The network was also changed from three color channels to two. In ADA, this caused issues, as most of the augmentation methods were based on 3D transformations and could not be used properly with two color channels. These and other similar small changes were also later found as issues in GitHub on the original repository of the code but generally never answered. Overall, no major changes were made, however, and the final model very closely resembles the architecture of the original StyleGAN2. The changes were thus mostly related to either parameters and optimization methods, or the way StudioGAN handled the training loop. The StyleGAN2 network remained very close to the original.

4.3.1 Evaluation metrics and training progress

In neural network training, the metrics are separate measurements from the loss for how well the network is behaving. While the loss governs the training, the metrics do not affect the progression of training at all. Instead, the metrics are ways of measuring the success of the training and are considered the goal of training [6].

In our case, the important metrics should then be related to downstream classification and image quality. These two do not necessarily correlate, however [66], as augmentation does not necessitate images that have perceptional quality. The simplest metric is visual inspection to simply see how the image looks. Although this is very useful for natural objects such as animals or objects, this metric provides barely any utility for DMS images. Typically, a research paper provides both: the quantitative metrics in numbers as well as showcasing some examples of generated images.

The best choice of metrics for training a GAN that produces visually good images is generally not clear, and for the downstream classification task less so [102]. The goal is to see how well the distribution of the fake images matches the distribution of the real images. The issue, however, is that even marginal differences to the original distribution may end up as the image being complete garbage or very useful, due to the high dimensionality of images.

Common choices for metrics in the literature are FID (Fréchet Inception Distance), IS (Inception Score), and KID (Kernel Inception Distance) and they are also used in StyleGAN2 research [57]. They are perceptional metrics and they rely on a pre-existing classifier trained on the ImageNet. This classifier finds *embeddings* of the images and compares the embeddings between real and fake images. These embeddings are thought to represent the image in a more abstract compressed form. These abstractions should then be equal or similar in fake and real images if they are perceived to be similar. As the classifier for the embeddings is traditionally the Inception-V3 trained with $299x299x3$ sized images from ImageNet [102], these metrics might not provide much information in a very different case such as ours. The base implementations for these metrics however already existed in StudioGAN. With little adaptation and improvement, they were quite fast to adapt to my training loop, and I therefore included them. Using FID, IS and KID with the already quite aged Inception-V3 is questionable and it is called out to be changed by recent research [103]. However, it is still the most common choice in literature mainly to have a fair comparison between implementations. The three metrics each act a bit differently and thus provide different advantages, with FID often being the "main" metric used for comparison. FID should have a high sample size, above 50k, making it even more questionable in this thesis.

Besides FID, IS, and KID I also computed precision, recall, density, and coverage (PRDC). Poor recall should mean that there is a mode-collapse and the whole training set is not present in the distribution. Poor precision should indicate bad quality with missing details in fake images. Density and coverage are argued to be improved metrics of these same concepts. While the former should inform about class diversity and image quality, the latter focuses on the trade-off between quality and coverage of the distribution. These four are also computed using the Inception-V3 logits and a nearest-neighborhood search between the logits of real and fake images is used to calculate the metric values. [102]

In addition to the mentioned metrics, the main metric to assess the models in this thesis is a modified version based on the GAN-train and GAN-test [104] also similar to Classification Score (CAS) [105]. In GAN-train, a classifier is trained on the generated data and evaluated on its performance on a validation set of the real data. For GAN-test, the real data is used to train a classifier to test classifying the synthetic data. If the classifier learns the distribution of the set, and the GAN outputs a similar distribution, these metrics should be somewhat equal. The GAN-train approximates the recall to catch the diversity of the samples. To assess the quality, GAN-test is indicated to provide an approximation of the precision. These were the most interesting metrics to me, as downstream classification is the task I am aiming for. Interestingly the authors found that the downstream classification metric might not necessarily correlate with the image quality found with FID and IS, and this finding was repeated by other papers as well [66].

In my version of GAN-train, I tested the metric using real images that were also used to train the GAN. Obviously, there is the danger of data leaks when the training data is classified using fake data that was produced with a generator trained with the training data. However, as I observed that an LDA trained on the DMS data only classified the data it was trained on with around 95% accuracy, I was confident this might still be useful. I did not want to have a validation-set of the training-set, as the training-set for the GAN was already very small at around 350 images. This would have also required cross-correlation analysis of the split, increasing the computational burden once again. Typically the GAN-test was much higher than GAN-train, indicating that the quality was good but diversity was lacking. While such observations were used during experimentation, they were not validated with results. The goal was clear: if the GAN-train metric could reach the 95% then the fake-data trained classifier would be equal to a real-data trained classifier.

Metrics for GANs is an active field of research as the GAN can be trained infinitely and knowing when to stop cannot be derived from the training losses. The metric acts as an outside evaluator for the quality of the model. In some cases, the human eye has been the gold standard to evaluate when the GAN has reached its peak ability to generate fake images [3]. Finding metrics for training a GAN in a limited data setting does not seem to exist beyond the usual ones. However, it is also seen with GANs that simply training for a set number of steps gives a good model instead of stopping at the best metric value.

Specifically, to calculate the GAN-train metric, I classify the training set with a classifier trained on fake images. Every k steps during training I generate n images, then train an LDA classifier with those images, and classify the training images with that classifier. I repeat the training multiple times to get an average with different generated images. The choice for k simply depends on how much I am willing to spend time on computation for this metric. I ended up having k at 100.

For the number of samples n, the choice is not very straightforward. The classifier for this metric was LDA for me. How many images should I generate and train the classifier with is one of the big questions in the topic, so choosing the proper number of images took some exploration. I did some trial runs to assess how the GAN-train reacted when LDA is trained with a differing number of images. I explored by increasing the images from 5 to 500, averaging 20 generated image sets for each run. Depending on the subset of training images and fake images, the best number of fake images varied, but a general peak was observed at around 100 synthetic images to train an LDA with. Therefore, I chose 100 as the number of images to use for the calculation of the GAN-train metric. The effect of the number of fake images used to train the LDA classifier is seen in Figure 4.3. However, it was also an interesting finding that the best number of fake images is unclear and the reason for this behavior could be explored further. Generally, there seemed to be an upper limit beyond which the model got worse. Typical implementations use a neural network-based classifier and these considerations could have been left out by having such a classifier. Alternatively, an ensemble of LDAs trained with differing number of images was considered.

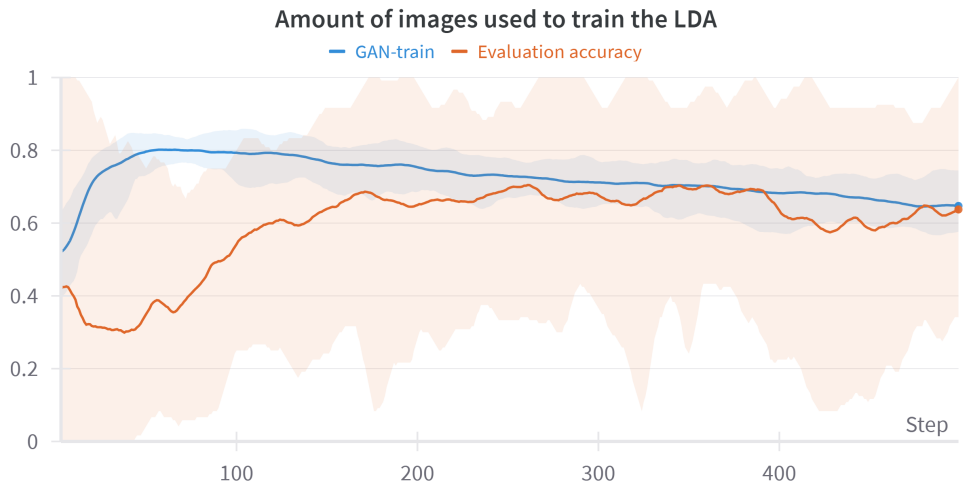

Figure 4.3. *How the GAN-train metric differs with an increasing number of images. The extra shape below it is the effect on the evaluation set. Although the evaluation set typically peaked at a very different point compared to the GAN-train, we can't use the effects on the evaluation set to make a decision. The color around the curves indicates the deviation from the average line. The evaluation accuracy varied highly on different sets of fake images.*

Every 100 steps during training I calculated the metrics. For the main metric GAN-train, I generated 500 images. I took a random subset of 100 images that had an even number of samples for both classes. I trained the LDA using this subset of 100 images and calculated the GAN-train metric. I took a new subset of 100 and repeated this procedure 20 times. This was to ensure that random sampling and the effect of randomness in generated images would be minimized. The result for the value of the metric was the average of the predicted probabilities for each sample over the 20 runs.

For FID, IS, KID, precision, recall, density, and coverage I used the pre-trained Inception-V3 for the logits. I upsampled the images from $32x32$ to $299x299$ using bilinear upsampling. As the model expects three color channels for images, I created a third channel from the average of the two, the positive and negative ion images. Modifying the Inception-V3 to accept two color channels broke the model and I had to make the third one. I also tried leaving the third channel empty, but the average of the two channels gave results that were closer to what should be expected. Training an embedding model to derive logits with medical images or DMS images could be an interesting approach to build a more useful model to use these logit-based metrics better. Progression of metrics over two training runs are shown in Figure 4.4. While all the metrics behave as they should, some are more informative than others. In this example there was clearly an event for the other run at around 60k steps where the training failed. While some metrics such as precision and density recovered quickly to even better values, others such as recall and KID had much worse values after the event. Failures in training are commonly seen in the metrics and can be analyzed to understand how the training failed and correct the training procedure.

Early saving was implemented using the modified GAN-train metric. Specifically, every time the metric improved the model was saved on disk overwriting the previous save. I started this only after 7500 steps to skip unnecessary disk writing at the initial phase of training. The average last saved step over all 22 evaluation sets was around 90k steps, depending on the run, with a total of 100k steps being the training length. The model from the last saved step was used for generating the images that the results are based on.

Observing losses as the metric is sometimes a good choice in neural network training. However, the losses for the generator and the discriminator generally converged quite fast and were not very indicative of the model's performance. They are expected to converge and then oscillate near the Nash equilibrium. However, the overtraining and memorization of the discriminator were often indicated in the loss curves and were useful to observe. Also, the effects of regularization could be observed and visually the trade-off between regularization and exploration was seen in the variance in the loss curves of the generator. These results were mainly visual observations and were not analyzed quantitatively or qualitatively. Observing them did guide the experimentation and reveal information, however, which highlights the usefulness of platforms such as Wandb to track the progress of

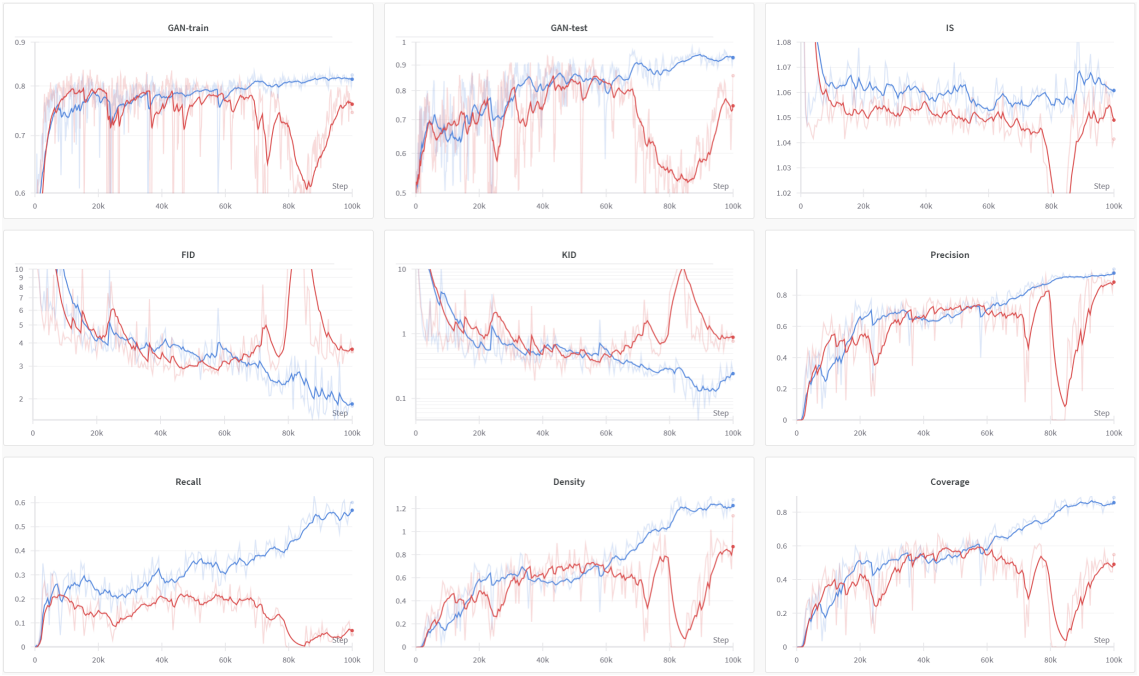

Figure 4.4. *An example of metrics progression over two training runs. The view is from the Wandb platform. Many metrics were included for comparative analysis. KID and FID should go down, while the rest should go up. The platform provided by Wandb allowed for quick visualization and making a dashboard of the most important metrics to follow. The training run in red failed somewhere after 60k steps while the training run in blue was of the normal kind.*

training. It was very useful throughout the whole thesis work to see from the loss curves whether some parameter change completely threw off the model or if everything went as it should have.

Besides the GAN-train and other GAN metrics, image statistics were explored as indicators of training progress. The mean, variance, minimum, and maximum pixel values showed convergence towards those of the dataset very quickly in training. Closer histogram analysis of specific pixel ranges revealed that most intensities converged to have similar histograms as in the training set. Only the pixels in the large median peaks shown in Figure 4.1 took longer for convergence in the training. It seems that the network quickly learns the statistical distributions of the images but takes much longer to work out the small differences in the largest peaks. The DMS images have a very high peak in the intensity distribution due to the large amount of area with very low activity. It would have been interesting to further study what the generative network is learning after the initial phase, where the main features of the images are learned, and as it moves on to focus more on the areas with minimal activity. No metrics were based on these statistical values, but their close correlation with other useful indicative metrics showed that they could have value.

I also investigated the weights of the models as well as the intermediate images of the

generator to assess the training. I analyzed intermediate images similar to the StyleGAN2-ADA paper [47], where they combined how different resolutions together made up the final image using relative standard deviations of the intermediate images $4x4$, $8x8$ and $16x16$. This is based on how the generator forms the image. Although I left it out from the final analysis, the general observation was a split of around 70/10/10/5 with 32/16/4/8 being the relative pixel sizes. For some reason the 8px image had consistently the lowest contribution to the final image. Conversely to the original paper, the 32px started to lose dominance as the training went on. It is possible that the model learned some fine-grained features too fast in the beginning. Observing the relative dominance of the intermediate images gives insight into how the final generated images are formed. Most of the image is formed by the high-grained details by the last layers, but the smaller images before that contribute with more coarse effects. Combining this with the histogram analysis and the logits of the losses could lead to further analysis on what the GAN is learning at which part of the training, and what parts take the longest to learn. This could also lead to insight as to how long the training should go on. Usually in the last 100 000 step most of these values were still converging.

The metrics, reporting, and saving are the only extra steps taken during training. Else it would be simply training the generator (G) and discriminator (D) in alternating turns, forever. To summarize the training loop: in each step, the loss is calculated and weights are adjusted for G and D, once for both. Every 100 steps the GAN-train metric is calculated, along with other statistics. If the GAN-train metric value is better than before, the model is saved to disk as the "best version". Every 1000 steps other metrics are calculated: FID/IS/KID/PRDC. The reason for doing it only every 1000 steps is that they take a lot of time to compute. Training stops at 100k steps. Overall, the calculations for the metrics added around 30% to the total training time.

All metrics were tracked during training using the Wandb platform [106]. The value tracking platform was very heavily utilized to track all the changing numbers during the training of the models in this thesis, with at least 2 200 tracked hours of computing logged. It provides automatic figures and visualization tools and stores values in the cloud for later analysis. There are also many MLOps alternatives, as the machine learning community has learned that tracking many numbers to observe the training is both useful and even a requirement.

4.3.2 Computing final evaluation results

The final images are generated using the model that is saved at the best value of the GAN-train metric. The last training step was 100 000 or 400 kimgs, as the batch size was 4. In the original paper [57], a similar model was trained for 25 000 kimgs. The implications of this training length are discussed in Chapter 6.

Evaluation of the methods is based on a similar approach as in the original IDH paper [4]. The whole GAN procedure is trained with the data of 21 patients, leaving one out for evaluation. The samples of this one patient are evaluated using fake data, real data, and combinations of them. Leaving one patient out during the whole training gives a realistic scenario of how the machine learning models would be used in a real-world setting, such that the new data does not affect the training of the model in any way. Therefore 22 GAN models had to be trained to evaluate the dataset.

To further validate the methods, averaging with randomness was utilized. This means that each evaluation is done by multiple sets of fake images to give an average output. The model might generate 100 samples that give a perfect evaluation accuracy due to luck. Repeating this generation multiple times until the average result with a small deviation is found gives a more realistic score. This can be employed in a real setting as an ensemble.

In addition, training multiple models with differing preprocessing steps gives validation to the overall methodology. Overall, six different models were trained for each patient to get evaluation results, the final list of GAN models is therefore $22 \times 6 = 132$ in length. The six different variations of GANs were:

Run 1: standardize to 0 mean and 1 std using individual image and channel-based standardization. R1 regularization = 0.01.

Run 2: similar, but use ADA and R1 = 0.1.

Run 3: no standardization is used; pixel values are original after resizing. R1 = 0.1 and ADA is used.

Run 4: standardization is done using the training set mean and std instead of individual image and channel values. R1 = 5, no ADA.

Run 5: similar preprocessing and parameter values to run 4, but detrending is added. Only every fourth sample is used, as the detrending averaged the 4 slices into a single image.

Run 6: no standardization, R1 = 5, no ADA. Only every first sample out of the incision sets of four is used, effectively ignoring 3/4 of the dataset.

This set of models was chosen to represent different parameters and preprocessing settings. Such comparison will provide more promise to the robustness of the GAN if the results are positive for all of them. I concluded that finding the best set of parameters to produce the best results would not answer the research question as well as validating with a larger set of models, even if the results were worse.

To get the LDA classification result I initially compared different methods including the ensemble, averaging, thresholded ensemble, and combining fakes with real images. In the end, I chose to use only fake images using an ensemble, as that had the most robust results with regard to the GAN-train metric. In addition, CNN classifier accuracy is

calculated using differing number of fake images. They both will compare the same objective: evaluation accuracy of the test patient. That is, each sample is classified as either IDH-normal or IDH-mutated.

The implementation details of the CNN classifier are not the focus of this thesis and are shortly presented in code in Appendix C. I used the ConvNeXt V2 [100] to obtain the results. A common choice in literature is Resnet-18 for this task [75]. However, using a more modern solution is more appropriate, and the Resnet-18 is not designed for small images. Interestingly, even the choice of the classifier might swing the results either in favor or against GAN data augmentation [66], making this another parameter to consider for proper validation in the future. ConvNeXt was also chosen to give a valid reference to a state-of-the-art result for the original dataset without using any fakes at all. The smallest 'atto' version was used. Larger versions had slightly better accuracies but the smallest was the fastest to train. Ensembles have been suggested between LDAs and CNNs for a DMS image [35], and though such work would be beneficial, it was not done here. I also tried vision transformers [107], but their longer training time with similar results made me lean towards ConvNeXt. It is notable to mention however that vision transformers performed very well with the DMS data. Due to their highly different mechanism compared to convolution, they might be even more appropriate with a proper setup.

Latent-traversal or **bias-traversal** relates to controlling the Z-latents during image generation to control the generation output. Specifically, I named this method 'bias-traversal', where I find specific latents that changed the GAN-train metric to be more balanced. I developed this method as an answer to an issue that was apparent and annoying during almost all training runs. An issue I noticed from early on during training of almost any kind of setup was that from the two classes, class 2 had much better results in the LDA GAN-train metric when evaluating each class separately. This means that the sensitivity and specificity were unbalanced. This caused me to put a lot of effort into finding out whether the binary classification setup was causing issues in the model.

I tried swapping classes, soft labels, balanced sampling of the GAN, modifying each class, exploring the inner workings of the network to find bugs, and other methods, but I could not really explain it. My conclusion after all this was that perhaps it was simply easier for the network to understand the features of that class and to generate images from it. I investigated the behavior of the discriminator logits for an answer to this and there was a slight indication that the positive class learned individual features while the logits of the negative class mostly responded to universal features that both classes reacted to.

It could be that one of the classes has easier features that the GAN learns faster, such that the features that define that class are much more distinct compared to the other class. It could also be that both classes share features that are more prominent in the positive class and the GAN overemphasizes those. As an example, the GAN-train would have

TPR (true positive rate or sensitivity) of 90% and TNR (true negative rate or specificity) of 70%. A balanced classifier trained with a balanced dataset of generated data from a GAN with a balanced training set should have only a small difference between classes such as 82% and 78%. Later I learned that this effect was the other way around for the CNN classifier, where the TNR was much higher than TPR. I call this 'bias' as it was a distortion to an unknown direction.

As I also observed this giving a bias to the evaluation set during evaluation, it seemed that this was a somewhat linear bias akin to the features found in the StylEx method explained later on. As explained before, the latent spaces of Z and W ought to be traversable. From experimentations I found out that changing either the mean or variance of the latent space Z had an almost linear relationship with this bias. I found out that by changing the mean of the latent space towards either the plus or minus direction I could directly alter the bias in a way that was linear or very smooth around the 0 mean. Further on, I learned that individual dimensions in Z could control this. Thus, I developed a simple algorithm to find a more optimal Z sampling, where the bias for the training metric was smaller. This was defined by the class-specific accuracies (TPR and TNR) being closer together without having a drop in the overall accuracy. The algorithm is as such:

1. Generate 10 000 images.
2. Calculate the GAN-train metric using 500 averages.
3. If the class differences are over threshold $t\%$ in accuracy, repeat 1-2, but change the next dimension of Z for ± 1. If this reduces the differential accuracy AND improves the combined accuracy, save the sign and dimension index.
4. Repeat 1-3 until enough dimensions from Z are found such that the class difference is below t.

With this algorithm, the overall GAN-train metric increased every time, and the difference between TPR and TNR got smaller. The idea behind this is that the latent space Z is well-defined for most of its 512 dimensions. With a well-defined latent space, the latent traversal should produce good results, but with some change in representation. The representable direction seemed here to affect the specificity and sensitivity trade-off directly. Typically, only a few dimensions were required for the change in classifier balance. As an example, the change for a single training could be such that

$$Z_{-1} = [1, 25, 35]$$
$$Z_{+1} = [3, 200],$$

where the numbers represent the dimensions of Z. They were chosen to be looped in random order.

My interpretation from this is that there are likely general features common to both classes, and traversal in the latent space will assign these features more commonly to one class than the other. How exactly this happens is something that I briefly explored but did not find the answer to. Certain augmentation methods focus on accomplishing interpolation at the feature level [75] such that this kind of traversal changes are used to control the synthetic dataset to have an ideally featured distribution. This type of augmentation would likely have better results if the control over augmentation can be made into a metric such as here using the TPR/TNR difference in the GAN-train. A GAN can be idealized as a general resampling method to generate a dataset that is well-balanced around the theoretical distribution of the data. This makes it an attractive option to generate maximally unbiased datasets for downstream classification using controllable latents by methods similar to this.

This also showcased the strength that GANs have in learning representations. The VAEs are another option when thinking about representations of the latent space, while the diffusion models do not have the same semantical separation, as discussed before. This strong representational space is of high interest in the exploration of new inventions such as drug discoveries, or new scientific findings [72], [108].

This concludes the methodological discussion of RQ1 as we now have fake images and methods to assess their utility. The representation portion leads right into the methods to tackle RQ2 for which I train the StyleGAN2 with a special technique to explain differences between the two classes in the data. The underlying idea is based on the assumption that the model can generate the two different types of classes. With a strong representational space, the attributes of that space should explain the separation of classes and therefore we should be able to learn how the classes differ.

4.4 StylEx for model explanation

The StylEx method [48] was generally described in Section 2.2.4. In this section, I will go through the implementation details, as well as some of the challenges I faced and adjustments I had to make to make the method work for this thesis.

This part was one of the biggest hurdles in my thesis, mainly as the initial research publication lacked certain crucial details. Towards the very end of my thesis, I found that the same team released another paper in 2024 [108], where they expanded the method towards a more general hypothesis and discovery investigator of especially medical imaging models and datasets. While this publication and the new supplementary details would have perfectly served me during the main work of my thesis, it was published after most of the work and I only found it at the latest stages, and therefore this section does not consider the new information provided in it. The new paper is briefly discussed in Section 6.1.2.

The latent control over Z that was found in the last section was already impressive. However, the StyleGAN is well known for its capabilities to find very strong semantic representations in the W-space, rather than the Z-space [47], [58]. Typical representations are for example parts of the human face, where during the generation of the human face a semantic representation could be the width of the nose or the color of the eyes [58]. This topic is called GAN inversion, where an input image could be partitioned into its representations, and it is used also in medical imaging, for example to generate mammograms with desired shape and texture [77]. The interpretability and controllability of the representations are the desired traits, as the controllable features should be easily understood and have good control in the image generation. Besides moving from Z to W-space, I also required a method that includes class-specific attributes, and not only general features of the dataset.

For the RQ2, explaining the inner workings of AI or data using AI, I also started with VAEs and other generative methods. I also tested InfoGAN, and while I managed to train the MNIST, I did not get good results for the DMS dataset. I quickly moved into methods that use StyleGAN2. The VAE had bad generating quality, and the representations would have to be generated with another model such as VAE-GAN. The stylespaces are generally explored using an extra latent space of $W+$ [58], but these provide no explanation for the differences between classes. A method I found in literature based on counterfactual examples called StylEx [48] fit my case very well.

In the original implementation of StylEx, the StyleGAN2 model is modified to include two new loss components: 1) reconstruction loss L_r and 2) classifier loss L_c. L_r is a loss defined between the reconstruction of an image and the original image. L_c is a loss defined by the new classifier. It is calculated between the classification results of original

images and their reconstructions. This reconstruction is made by the generator after an encoder (E) encodes and a classifier (C) classifies an original image. The W-space does not anymore start from the random noise vector Z, but instead from the concatenated encoding $E(x)$ and classifier results $C(x)$. This forms the new W-space and the rest of StyleGAN2 is original.

The reconstructed images from the encoding are the end-product of the generator. Therefore, each reconstruction could theoretically be the exact same image, if the generator is good enough. Note that this is generally not a desirable trait for a generator but is instead used in this implementation to provide information on how the generator behaves. It is minimized using the L_r loss.

The original code was very hard to read and lacked a lot of explanations of the details. There were significant differences between the code and the paper, and also new details that were not mentioned in the paper. Luckily, there were three re-implementations that were done to attest the "ML Reproducibility Challenge 2021", where the goal was to study whether a paper is reproducible. The three re-implementations [109]–[111] provide a wealth of new information. Some of them had contacted the original authors of StylEx for more details. However, even the re-implementations left open questions about specific details and best practices as well as well documented readable code.

I will describe some of the most important findings from the three re-implementations that were not present in the original paper. First, the classifier C is typically pre-trained and not trained during the training of the GAN. The encoder E they used has the same architecture as the discriminator, so it is a copy of it trained separately, with only the last layer changed to have 512 output nodes such that $E(x)$ and $C(x)$ will concatenate to 513-long W-space in the binary-class case. The generator (G) and discriminator (D) of StyleGAN2 remain as they were. They transform the images into $[-1,1]$ to use LPIPS (Learned Perceptual Image Patch Similarity) properly. The StylEx may be trained in alternating steps. That is, every other step is normal StyleGAN2 training, and every other step is StylEx-flavored training. According to the authors, this might provide a smoother W-space. Reconstruction and classifier losses L_r and L_c are only used during G training, not D. The pre-trained classifier was either MobileNetV2, DenseNet121, or ResNet-18, though in one of the reproductions the MobileNet did not succeed with one of the datasets of the original paper. More advanced nets should give more advanced clues. Discriminator filtering that skips encoded images that were too unrealistic was used in some implementations and not used in others. L_{rx} and L_{rw} were L1 distances. L_c is KL distance (Kullback-Leibler divergence) between the original classification logits and the reconstruction classification logits. The original authors had recommended default values of 0.1 for L_{rx} and L_{LPIPS}, and 1.0 for L_{rw}. They also altered the learning rates of the StyleGAN2 as well as optimizer parameters, though the values differed depending on the dataset. The original weights were in TensorFlow, and some authors transferred

them to PyTorch, as PyTorch was preferred. One of the authors used Wasserstein loss as the adversarial loss instead of the original softmax-based loss between D and G when training an alternating step that uses the encoder. Removing the addition of noise was not mentioned to my knowledge but proved helpful for my implementation.

Besides the mentioned differences and changes, the reproductions all had issues in using the method. Some challenges included the top attributes not being useful, training not converging and results not working for all the datasets of the original implementation. This is likely not due to problems in the method but rather due to missing implementation details even after correspondence with the original StylEx authors as well as being limited by computational cost. All of the reproductions mentioned long computing times as a limitation. One interesting observation from [111] was that the AttFind algorithm could work in any autoencoder not bound to GANs as a good approach to finding important attributes with counterfactuals. In fact, the authors in [110] found interesting attributes from a StyleGAN2 trained model without any StylEx type training by only applying the AttFind algorithm. All datasets used in these implementations had over 10 000 images. While this was worrying, it did not set me back as the general StyleGAN2 training was behaving well with a lower number of images.

My implementation follows the main algorithm outlined in the original paper using a combination of the parameters implemented in the re-implementations. The classifier used as the decision-maker was MobileNetV3. While this part of the work was quite discouraging due to the lack of clear information in the literature and lack of guidance from anywhere else, as well as even larger training time compared to base StyleGAN2, I eventually succeeded in converging to results that gave features similar in style to the original research, except with the DMS dataset. The simplified version of the total pipeline for the StylEx method can be seen in Figure 4.5.

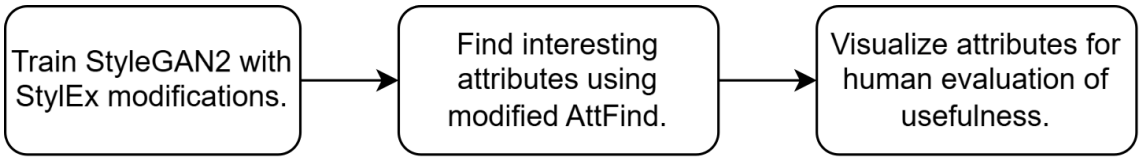

Figure 4.5. *The general workflow of the StylEx method as a diagram. First, we train a StyleGAN2 network largely in the original way but with some marked differences. Then, after training, we start to modify the generator with very small changes and see how the output changes from the perspective of a classifier. Finally, we visualize those differences from the most interesting attributes as the outcome is for human evaluation. The end result of this method is a visual representation tool for the attributes that define the classes that can be used for understanding a dataset or creating discoveries in it.*

AttFind is a search algorithm for good semantic dimension extraction given in the original research [48]. Generally, in GAN inversion, when finding dimensions that have good, semantically representational meaning, one needs to generate millions of images for each

dimension individually and find some way of measuring them. With 5 632 dimensions in the W-space, this is not ideal. AttFind works by modifying each dimension and seeing if the classification rate changed with a small modification for that dimension. If the average change for all images is over a threshold *t*, the dimension is chosen as a good dimension. After all the dimensions are tested like this, the dimensions can be then tested together or visually inspected for results. As the individual dimensions were already representative in my implementation, I did not try using them together even though this resulted in better attributes in the original research.

I had to make quite large necessary adjustments to the AttFind algorithm. Starting, I found that the ± 1 was not effective at all. This is likely due to my dataset size and possibly the size of the network. With their method I found dimensions that for example turned all images into black or white, but mostly complete nonsense. I found that the ± 1 was very crude and I had to look for smaller adjustments. Instead of using their thresholding method, I defined my own by searching for linearity within the classifier decision change region. To search for linearity, I went through smaller steps starting from 0.01. If the changes in the classifier decision were minimal, I would increase this in logarithmic steps. The decision change is defined by the logit output of the classifier. I stored the changed average classifier predictions when the change in decision is averaged over the whole dataset. I mapped out the range around the zero-point until the logits started to 'shoot-off' by going very high in value suddenly, indicating that the change made the image nonsensical as the classifier outputs a nonsensical prediction. This defined the limits of my area of change for each dimension. From this area I then searched for the best linear fit. The best fits were chosen to represent the attributes.

As my approach to AttFind was almost completely different, I also visualized some extra interesting information regarding the linearity around the zero-point. This approach should provide more stable attributes but is much more computationally costly when the dataset is large. It might also not be necessary with a larger dataset. In my case, with only a few hundred images, this was very feasible and useful. Implementing the work in batches provided a large boost in computational time.

The modified attribute search algorithm:

1. Make modifications in steps for each s dimension of S. Such as $[-1, -0.9...0.9, 1]$.
2. Calculate the average change in classification over the whole set of images $C(X)$.
3. Find those dimensions with linearity in the change, as defined by a good linear regression fit for the independent variable s and dependent value $C(X)$.

After the algorithm, I was left with interesting latent dimensions and a range of values that change a decision-maker's (the classifier) decision when traversed. The linear change region is plotted next to images such that I can easily visually confirm that they are sen-

sical. The decision changes are plotted for both an 'outsider' LDA and the 'insider' CNN. By controlling the attributes, I was able to visually see how certain attributes belong to certain classes. Enhancing those features in the image by increasing the change in the latent variable makes the classifier change its decision from one class to another. This change might be minimal and unnoticeable due to the visualization challenges of DMS data. Therefore, I boosted the changed values for better visualization to highlight what features the attributes present. The front face of the visualization tool is presented in the next chapter in Figure 5.2.

Ideal StylEx-attributes or *s-attributes* are such that they explain the features in the DMS images. The features should therefore be of a-curves or of fragments. They should explain both classes equally and as strongly. There should be some linear area of change such that it transitions smoothly between classes. The change in the images should be in local areas related to the features and not global. And the attribute should control a recognizable feature relating to features understood by humans. For the original research, the explanation between genders was for example a beard and the explanation for an ocular disease was the appearance of that disease in the images. For the DMS images the interesting attributes are the five explained in Section 2.1.2: the a-curves, or the fragments. These should ideally be in single dimensions of S, as the features should be disentangled in meaning. The combinations of dimensions were not explored, although the original research had a method for this. Compared to saliency maps, this method highlights very specific semantically meaningful representations instead of more general activation maps of layers.

This concludes the methods portion of this thesis. In the beginning, most of the time was spent on literature search, as simply different GAN architectures exist in the thousands. Specific methods for answering RQ1 and RQ2 are abundant. They also exist for different situations with specific implementations. However, code implementation and validation of each method for a new dataset would take weeks, and some time was thus better first spent in searching before experimenting. As the experimentation phase began, constant computing was done in the background to get some results or to test some parameters. From the start of the experimentation to the end of the thesis the work was constantly limited by computing power. During computations, the time was spent on debugging and understanding what was going on, while also still searching for new methods. The methods provided here produced the results that are presented in the next chapter.

5. RESULTS

Here I will present the results of using GAN-made synthetic images for the tasks of downstream classification (RQ1) as well as explainable AI (RQ2). The results for the first task are presented quantitatively in tables, while the results for the second are presented only qualitatively based on visual observation and discussion with an expert [27], with more discussion of the results in Chapter 6.

5.1 Classification results for data augmentation

The classification accuracies for the six different models explained in Section 4.3.2 are presented in Tables 5.1 for the LDA and 5.2 for the CNN. In Table 5.3 I have compared how use of different metrics might have changed the end results. In Figure 5.1 and Table 5.5 I have presented some results for the bias-traversal method. All values in the tables with the exception of Table 5.5 are the number of samples correctly classified from the number of samples in that set shown as percentage. For the whole set this is called accuracy. For class 1 and class 2 individually this is typically called the true positive rate (TPR) and true negative rate (TNR), or sensitivity and specificity. For individual patients this is typically called evaluation accuracy. If there are 8 samples and 7 are classified correctly, this value is $\frac{7}{8} = 87.5\%$. In the whole dataset there are 352 samples, with 176 for each class. For runs 5 and 6 this is divided by 4 and there are therefore 88 samples.

The LDA Table 5.1 includes values for both classes, a value over the whole set as well as individual patient values. The results for CNN in Table 5.2 includes varying number of fake images used to train the classifier and reports the class values for both classes for all six different GAN versions. The baseline comparison in the CNN table is $n_{fakes} = 0$ and it was computed as an average over 10 runs to get a stable value.

The general observation is that compared to the baselines in Tables 5.1 and 5.2, the accuracy improved using the fake images with LDA but did not increase when using a CNN.

Table 5.1. *Accuracy of LDA classifying each patient using only fake data. The first two columns are baseline comparisons with the original data. IDs 1-11 belong to class 1, while IDs 11-22 belong to class 2. Most patients were classified very similarly in the baseline and GAN-trained classifiers between all the different runs. D is for detrending.*

Evalset	Og.	Og. D	Run 1	Run 2	Run 3	Run 4	Run 5 D	Run 6
Class 1	71.6	75.0	75.0	73.9	71.6	76.7	77.3	79.5
Class 2	79.0	81.8	76.1	80.1	73.9	83.0	86.4	81.8
All	75.3	78.4	75.6	77.0	72.7	79.8	81.8	80.7
1	91.7	100	87.5	95.8	79.2	100	100	66.7
2	25.0	50.0	25.0	37.5	25.0	25.0	50.0	0.0
3	50.0	57.1	54.3	46.4	53.6	39.3	71.4	85.7
4	43.8	50.0	43.8	43.8	43.8	50.0	25.0	75.0
5	100	100	100	100	75.0	100	100	100
6	87.5	100	100	87.5	75.0	100	100	100
7	0.00	0.0	50.0	25.0	50.0	50.0	0.0	0.0
8	92.9	85.7	89.3	92.9	82.1	89.3	85.7	100
9	100	100	100	100	100	100	100	100
10	25.0	50.0	62.5	50.0	75.0	62.5	50.0	100
11	81.2	75.0	84.4	81.3	84.4	87.5	87.5	75.0
12	25.0	0.0	25.0	25.0	50.0	25.0	0.0	0.0
13	50.0	50.0	62.5	62.5	50.0	50.0	50.0	50.0
14	93.8	100	93.8	93.8	93.8	93.8	100	75.0
15	83.3	100	91.7	91.7	91.6	83.3	100	100
16	75.0	66.7	58.3	50.0	16.7	50.0	66.7	66.7
17	91.7	100	87.5	91.7	91.7	95.8	100	100
18	100	100	100	100	83.3	100	100	100
19	100	100	100	100	81.3	100	100	100
20	58.3	100	58.3	66.7	75.0	66.7	66.7	66.7
21	85.7	100	78.6	85.7	96.4	92.9	100	100
22	59.4	50.0	53.1	65.6	46.9	65.6	75.0	62.5

Table 5.2. *Table for CNN results with fake images for the six runs. The first row is the baseline, where 0 fake images are used. The last two rows represent the situation where only fake images are used, while in the rest all the training images together with some fake images were used to train a CNN classifier. The two values in each cell are the TPR and the TNR. The extra column with E is for the ensemble.*

n fakes	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 4 E.
n=0	81.1	81.1	81.5	82.1	88.0	66.4	82.4
	80.6	80.6	78.9	80.2	85.2	74.8	80.7
150	81.8	80.1	80.1	81.8	89.1	62.0	82.4
	78.4	81.3	79.0	81.8	79.0	76.8	80.7
300	82.4	80.1	78.4	81.0	86.8	69.4	81.8
	74.4	79.0	80.7	82.1	77.1	74.9	80.1
600	80.7	81.3	78.4	80.7	88.5	70.9	80.1
	77.8	76.1	77.3	79.6	84.3	74.9	80.1
500	80.7	82.4	71.6	78.4	84.8	67.1	79.0
	72.7	65.0	73.9	77.3	81.6	80.6	79.0
10 000	75.0	77.8	70.0	79.0	86.6	70.5	77.2
	70.5	75.6	76.7	74.4	79.4	82.9	75.6

In Table 5.2 the number of fake images to train the CNN was varied to see how it would affect the result. All 6 runs were computed to provide a comparison over different settings of the GAN as well as test the robustness of the generator to changes in parameters. Although there is some variance due to inherent randomness, the general sentiment is that adding fake images did not improve the classification rate. The fake augmented training set got worse as the number of fakes in it increased. The last column indicated by **E** is an ensemble of ten models. For that, ten CNN classifiers were used, and their logits were summed to form an average opinion. From that run it is clear that the baseline was the best and any fake images made the result only worse. The first row uses no fake images. The next three rows use a varying number of fake images together with the original images. The last two rows use only fake images to train the CNN.

The model was saved based on the value of the GAN-train evaluation metric. Values for other evaluation metrics were also saved during the training, as well as evaluation accuracy when computing those values. In Table 5.3 I show how the evaluation accuracy differed at the best values for each metric during the 100k steps of training. That is, the in-run computed evaluation accuracy was used at the best value for each metric. For example, the best value for FID is the lowest value it attained over the 100k steps. Although the results in Tables 5.1 and 5.2 are calculated in a different way from this value, it gives some indication that the used GAN-train metric was probably not optimal, as other metrics had better accuracy for the evaluation data in their best-value during training. In addition to this, it was also noted during result analysis that the in-run calculated GAN-train differed from the value found when re-calculating it later with higher accuracy using

Table 5.3. *Comparison of indicative results for other evaluation metrics. The values are the evaluation accuracies when each metric had its best value during the whole 100k steps training. The chosen metric to represent the final models in this thesis was GAN-train. From these results, however, it seems that other metrics could have given a better step to stop the training. The last step simply means taking the very last step. Random is the baseline: a random step for each id.*

Metric	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Average
FID	73.6	76.4	75.6	**79.5**	80.7	79.5	77.6
IS	78.1	73.0	75.9	79.3	80.7	79.5	**77.8**
KID	74.4	75.6	74.1	77.3	83.0	73.9	76.4
Precision	75.9	76.1	76.7	79.3	84.1	70.5	77.1
Recall	73.9	76.1	71.6	75.6	83.0	76.1	76.0
Density	**79.6**	76.7	73.6	76.4	81.8	77.3	77.6
Coverage	77.0	**77.6**	73.6	75.9	80.7	**80.7**	77.6
Last step	77.0	73.9	**77.3**	76.7	**85.2**	75.0	77.5
GAN-train	75.3	**77.6**	71.9	75.6	79.5	78.4	76.4
Random	73.7	73.9	72.4	75.7	80.1	75.8	75.3

Table 5.4. *GAN-train and GAN-test values at the saved steps for both LDA and CNN. The GAN-train value was the main metric. It is seen from the table that only the CNN GAN-train value indicated run 5 as having the best diversity while both agreed that run 3 had the worst diversity in the fake images. GAN-train is calculated with fake images classifying real images, while GAN-test is calculated with real images classifying fake images. Ideally both should be high.*

Metric	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6
CNN GAN-train	90.2	90.8	83.0	88.2	98.0	97.2
CNN GAN-test	93.1	92.7	91.8	92.6	97.2	96.1
LDA GAN-train	89.1	89.9	86.0	87.7	89.8	90.8
LDA GAN-test	91.4	90.3	83.4	91.6	90.0	83.9

more sets of fake images. On average over the six runs the in-run LDA GAN-train was 1.5% higher (90.4% vs. 88.9%) than the more precise computation. This was due to the number of averages used to calculate the value. Therefore, the best saving point was also due to swings and not the true best value for the GAN-train. The more precise GAN-train and GAN-test values are given in Table 5.4 for the LDA as well as for the CNN calculated in the same step using 10 000 fake images.

The effect of traversal for bias fixing is shown in Table 5.5 for some LDA runs and in Figure 5.1 for a CNN model. The difference in GAN-train classification rate between the classes changed significantly and the evaluation accuracies followed. The value for the diff-delta in the table is the difference between the class differences before and after the traversal

search. The differences become less while simultaneously moving the evaluation class in the same direction. In other words, this moves false and true positives towards negatives, with a higher proportion of false positives changing. This effect happens both in the training set as well as the evaluation set.

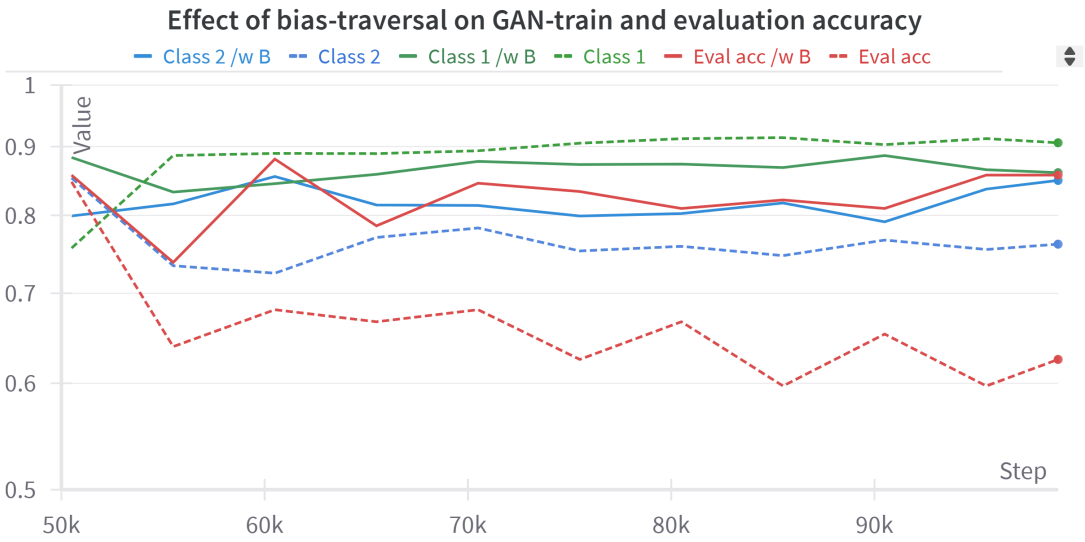

Figure 5.1. *Illustration of how bias-traversal balances the generated dataset during one CNN model training. In dashed are the original values for TPR, TNR, and evaluation accuracy. The difference between TPR and TNR was* 14.4%*. After changing a single value in* Z*-space, the difference was only* 1.2%*. In red, the evaluation accuracy is also seen to increase, as it belonged to class 2 in this case. If it were in class 1, it would have had a drop in value. Controlling the generation to produce a balanced dataset could have increased the CNN accuracy, but finding the good latent dimensions requires a very large amount of computing.*

As an example, if the classes have class-specific accuracy values of 0.90 and 0.70 and the evaluation set belongs to the latter, the values after the method might be 0.82 and 0.79 and the evaluation set would improve in a similar direction. In that example, the diff-delta value would be $0.2-0.03 = 0.17$ and the sign in the table would depend on the evaluation set class. If the evaluation set was part of the first group that had a classification rate much above the second class, the classification rate of the evaluation set would drop as well. The method essentially brings more balance to the output of the generator by controlling the latent variables that independently control the ratio of positive and negative recalls.

Table 5.5. *The effects of bias-traversal on the classification balance. The signs indicate the favorable direction. For 'run 1' all results with threshold=0.05 are shown. For the rest, only the largest difference was computed using threshold=0.02. The correlation between the signs is a clear indication that the method is working as intended: it reduces biased results in both directions balancing the dataset further. The last result highlights how much the skewness in the generator may affect the output of the downstream classifier. Producing a more balanced generator should reduce the overall bias of the data.*

Evalset	GAN-train diff-delta	Change on test set
Run 1: 1	+0.103	+0.054
2	+0.028	+0.028
3	+0.024	+0.029
4	+0.008	+0.001
5	+0.031	+0.010
11	+0.042	+0.021
21	-0.015	-0.010
23	-0.031	-0.017
34	-0.008	-0.008
Run 2: 25	-0.100	-0.053
Run 3: 35	+0.162	+0.100
Run 4: 21	-0.044	-0.030
Run 5: 33	-0.091	-0.019
Run 6: 21	+0.166	+0.222

5.2 Results for the explainability of the model

For the second task the final visualization tool is the main result. As it is not presentable in a static PDF, Figure 5.2 presents only the key concepts, with more images in Appendix A and discussion in Section 6.1.2. The website (explaining-in-style.github.io) provided with the original paper [48] gives a great intuitive idea of the method on multiple datasets.

All the attributes I found were in the "torgb" layers of the generator, none in the synthesis layers. This is the complete opposite of [58], where another S-space method found the torgb layers unusable. The StylEx papers do not comment on this subject. Modification of the styles in $\pm$ direction in the synthesis layers resulted in symmetrical classification change in both directions. The symmetry around zero for every attribute of the synthesis layers was surprising. Due to this, only torgb layers were used for analysis. This made the computations much faster, as only $3x512$ instead of $11x512$ dimensions were studied.

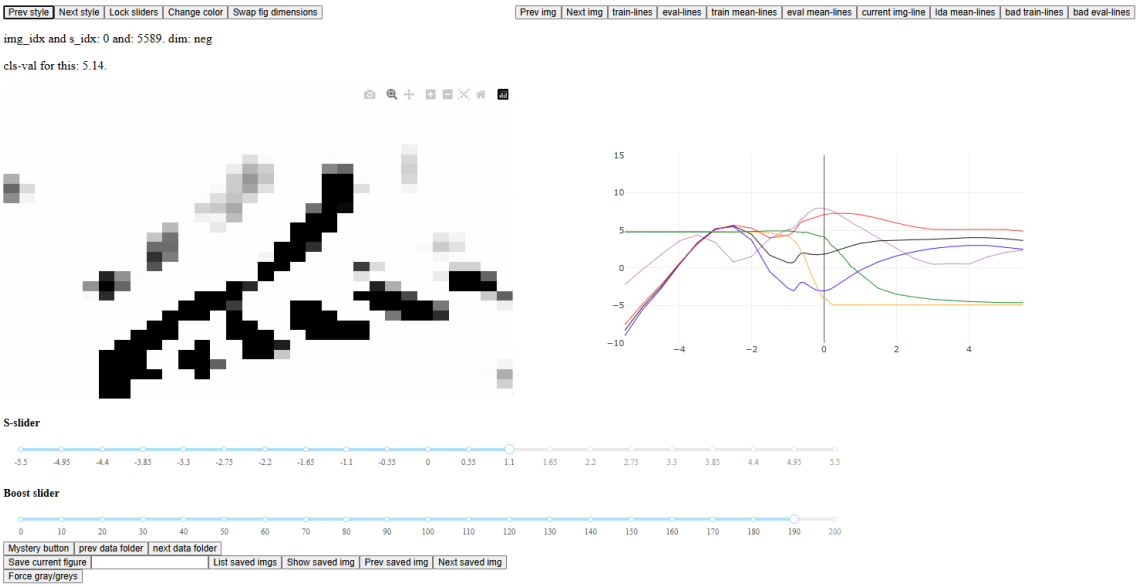

Figure 5.2. *The Plotly-dash front-end for the visualization of the features. On the left of the two figures is the feature that is highly boosted using the sliders. On the right is the graph where the change in classification is shown when the s-value is changed. In this example, the LDA and CNN classifiers do not agree on the direction of change for the image. Buttons are included to help handle data. The Python script for this code is 1062 lines long in the final version. A similar tool was used with the original research as the slider to change the feature is a very intuitive way to see the change in the image. The graph on the right is the result of my changes to the AttFind algorithm and it was not included in the original method.*

The results are further discussed in the next chapter. I will go over the implications of the results, future works and related possibilities, challenges and limitations as well as a general outlook of the project.

6. DISCUSSION

The thesis topics are discussed in this chapter. I will start by going over the results in more detail. I will then go on to discuss how future approaches should improve on what was learned from the results and also other approaches, as well as briefly discuss privacy and AI aspects in medical data in general. Finally, I will go over some challenges and limitations in this thesis, especially regarding computational constraints.

6.1 Results discussion

The results are split into the two tasks, downstream classification and AI explainability. First, I will go through the results of the data augmentation and downstream classification. I will explain the importance of the results as well as what they taught about using GANs for DMS data. Then I will go into more depth regarding the explainability of the model as seen through the results for the RQ2 defined task, where I used the StylEx method to highlight how a classifier makes the decision for a class based on semantically meaningful attributes in the image.

6.1.1 Downstream classification

For downstream classification the results are presented in Tables 5.1 and 5.2. We can see that the accuracy increases slightly when using fake images as compared to the baseline with LDA. The general observation from the reported results and unreported results is that the accuracy changed up to $+5\%$ using fake images. The gains from using the fake images are robust as they appear for most of the different test run setups for the LDA. For the CNN the accuracy did not improve. For the unreported results the CNN accuracy also improved, but these could have been due to chance. The unreported results are those that came about during the experimentation phase when the methods were still being researched and were not confirmed as robustly.

The LDA and CNN were the classifiers I used for the result analysis. Linear discriminant analysis (LDA) is a method that maximizes the component axes that separate classes in the training data by maximizing inter-class variance and minimizing intra-class variance with the goal of finding the linear discriminants that will act as linear axes that maximize

the ratio between inter-class and intra-class variance [40]. Convolutional neural networks (CNNs) are modern deep learning models and have been used in most if not all visual computation tasks in the last ten years. They work by applying the convolution operation that produces feature maps from images, combining local and global information together, as the convolution kernel is used in downsampled images or with pooled layers [6]. Beyond these, I tested many other traditional and modern neural net-based classifiers, but settled on these two as they are the ones most used in DMS literature to represent both a simple and also a deep learning method. Visual transformers (ViTs) worked well and had similar results but took longer to train.

The LDA classification rate improved using only fake images compared to the baseline. Using an ensemble and the summed probabilities instead of accuracies achieved superior performance compared to using a single classifier or averaging over accuracies. The posterior probability scores were used in another DMS work [15] as well, although to make visualizations of activation maps. The ensembling method that had the best 20 classifiers as defined by the GAN-metric achieved superior scores. This can be seen as an example of controlling the sampling of a GAN to achieve wanted results, similar to latent traversing, but the samples are thresholded instead of controlling the generation. Although even the best result with the detrending operation was 82% as compared to 86% in the original paper, the fake image trained LDA surpassed the baselines as seen in Table 5.1. It is unclear what the results would have been with the same preprocessing method and parameter search of the original research.

The computational time of the downstream task, whether using CNN, LDA, or an ensemble, is still in milliseconds. The heavy computational workload to produce the classifier is done outside of the timeframe of the real-world classification task, and the inference of the classifier models is still close to real-time. Applicability to real operations is then no different from using different methods of training the classifiers. The heavy computational workload of using a GAN to augment the data is done prior to the real-world scenario.

ConvNeXt CNN [100] did not benefit from the synthetic images. Whether using the synthetic images alongside the original training images, or alone as with the LDA, the accuracy was generally worse as seen in Table 5.2. Although the results are still subject to randomness in neural network training, as well as issues in GAN training, the agreement over six models provides validation to the result that the synthetic image augmentation did not provide extra benefit. The results with ConvNeXt were best for the original data with heavy augmentation using only traditional augmentation methods that are based on simple transformation of the images. While the generated fake images are impressive and are able to train a neural network to a closely similar accuracy as the training set, they were not useful in increasing the accuracy further, as was the hypothesis. General image augmentation methods were enough to train the dataset for this CNN. A further study should be done to compare these results, but they seem to align with the literature [67],

[112], [113], where the GAN might have a slight advantage or disadvantage, but no clear winner is found. One big factor is the dataset balance. The most promising results and indicated use of GANs for data augmentation is for the case of an unbalanced dataset. The IDH dataset had an equal number of samples for both classes, making it perfectly balanced with the exception of the take-out evaluation set.

Using only synthetic images without traditional augmentation did not work. Without any traditional augmentation, the network overfit on the training set very fast, even when tested with up to 100 000 fake images. It is likely that the DMS images are too static to train on a CNN without any traditional augmentation. The traditional augmentation strategy is shown in Appendix C. While the idea of expanding a dataset from the underlying distribution (GAN) is very different from the manipulation of images to avoid overfitting (traditional augmentation), they both should complement each other rather than be set in competition against one another. Both were used in all the runs.

The synthetic images were further tested on their capability to pre-train a CNN classifier to be used for a completely different DMS dataset, a new "B" set. For comparison, this ConvNeXt model was trained with either 1) only the B dataset, 2) pre-trained with the original-IDH dataset and then B set, 3) pre-trained with the synthetic data from the IDH-GAN and then the B set. The model was always launched from the pre-trained ImageNet as is usual. Pre-training with either the IDH-original data or IDH-GAN data both improved the training accuracy as well as the evaluation accuracy. The training accuracy increased from 75% to 82% or 84% and evaluation accuracy from 82% to 86% or 87% with the latter increase being the GAN-pre-trained value. This value, while only indicative, shows how much pre-training with similar data increases the accuracy and also how the GAN-based data was similar and even better in this pre-training scheme. GAN-made images therefore have value in pre-training schemes of out-of-distribution datasets and can be utilized for transfer learning, which is helpful, especially in the case of medical data privacy issues.

Looking at single test subjects we can see from Table 5.1 that most methods had similar values for each subject. In real-world scenarios, the main advantage of data augmentation should be to boost especially those situations where the accuracy is poor. The LDA methodology did not provide this. The class-specific features in those subjects might be too hidden and only very finely controlled feature-generation might improve their classification rate. The results with CNN, although not presented, were very similar, where instead of boosting those with very low accuracy, the differences were more spread evenly among all the patients. Many of the large swings are also due to low sample size, for example patient ID 7 had only four samples.

Standardization methods did not largely affect the results in meaningful ways. The training of the GANs was however highly affected, as the standardization made convergence to the plateau much more robust and faster. This was seen in the network loss values

as well as the evaluation metrics. Although modern neural networks do not necessarily need any standardization operations at all, the results for 'run 3' were worse for the LDA, the CNN as well as the metric values when compared to runs 1, 2, and 4. Run 3 had no preprocessing done besides resizing and the images had the original intensity range. As the normalization was unsuccessful, and run 3 had comparatively worse values, the standardization operation seemed to be the most valuable method.

Evaluation metrics generally improved with longer training. One longer run is presented in Figure 6.1, where some evaluation metrics for a model trained for 500 000 steps are shown. In the figure, we can see that the last step (100 000), where the training stopped, might have been premature. An interesting unreported finding was that the CNN classifier had better results when using images that were generated at a later stage. Although further analysis and validation were left out, it seems that I might have had better results if I trained for longer. I did not have the computational means to use CNN as a GAN-train metric classifier or train all the sets for a long time. However, as also seen in Figure 6.1, it does seem that some of the features may take much longer to learn beyond the 100k, which was the absolute last step for my training. StyleGAN2 is often trained for 10 or 50 times more than what I did [57], though there is no clear lower or upper limit, and the training is stopped whenever the metrics look good. This could be the very next step to take in the future, to validate the method with longer training times, and datasets that split into only 2 evaluation sets and not 22.

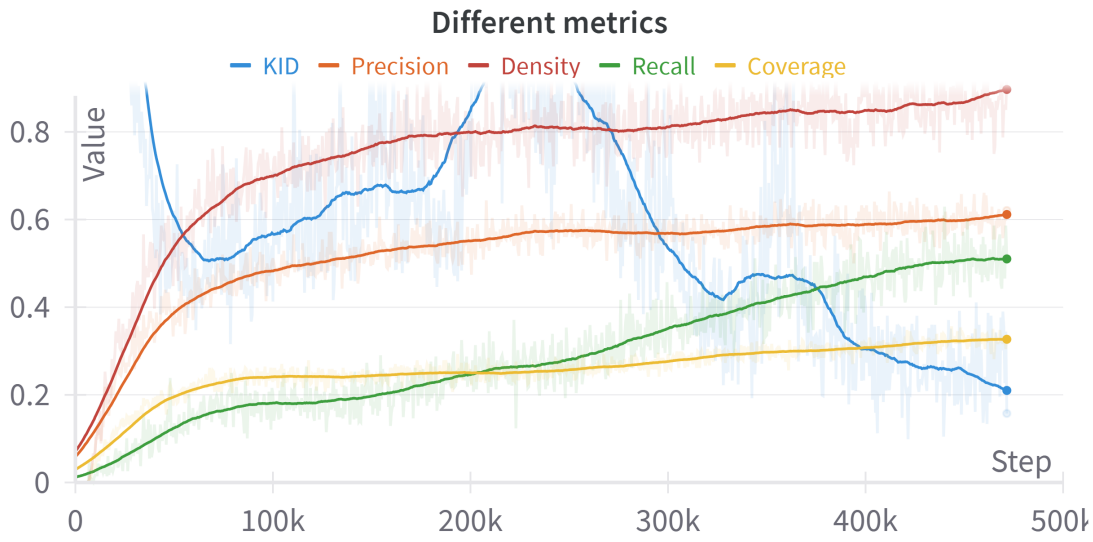

Figure 6.1. *A longer training session showing the evolution of some of the metrics. In this example all of them improved, as KID is supposed to go down. This specific training session reached the GAN-train metric highpoint already at 91k steps, revealing that maybe the implemented GAN-train guides the training to end prematurely. The training of the models in this thesis ended at 100 000 steps, while this model was trained for 500 000 steps, five times longer. There also seem to be different phases in the training. This model took 25 hours to train. Some more metric figures are presented in Appendix B.*

Metrics analysis in Table 5.3 reveals that the LDA-based GAN-train was not necessarily optimal to guide model saving as other metrics were consistently better than the GAN-train, although with only a slight difference. Even though these values are not representative of the final accuracies and only indicative of the final results, it seems that the modified GAN-train was not the best choice for the metric. The embedding-based methods were better even with the upsampling and dimension-adding operations. The logit model Inception-V3 is rather old and it is trained on natural images with the ImageNet dataset. A more modern logit model trained with more relevant data should therefore improve the applicability of these metrics. Alternatively doing the GAN-train with a CNN and a validation set might work best but it is computationally very intense. Consistently beating the baseline in Table 5.3 provides validation that the metrics are useful in general, but the small differences reveal that a better metric is still required, and their importance to the results was less than hypothesized. Although the 'last step' metric beat out the LDA-based GAN-train, testing the 100k saved sets with a CNN revealed that it was flawed. Using the 100k-step models resulted in very bad accuracies for a few patients, which brought the accuracy down for all six runs by 2% to 6%. The conclusion is that some metrics are better than the last step, but the modified LDA-based GAN-train was flawed.

The GAN-train was also calculated using the CNN at the saving step to test whether it would have been useful or indicative of the evaluation accuracy. This was done by training the CNN with 10 000 fake images and inferring with the original training set. The values in Table 5.4 show this. The CNN is mostly in agreement with the LDA except for runs 5 and 6, which used only a fourth of the dataset. It seems that for these the CNN model learned the whole set with only fake images. Run 6 was the only one of the six runs where the CNN beat the baseline using fake images, and for run 5 the gap was very small. It seems that the GAN-train and GAN-test values for the CNN might translate to better downstream accuracy, and it should be used in the future instead of the LDA-computed version. Correlation between the training metrics and evaluation accuracy can be assessed, but the validation would have to be done on a completely separate dataset to not introduce validation bias to the method.

The different fake images used to train the CNN caused the class-specific GAN-train values to vary for each run. This was used to calculate a correlation value between the TPR or TNR and the evaluation accuracy. An average of 10 CNN models were used with run 4 to calculate the correlation. The correlation between evaluation set loss and the GAN-train value for each class was on average -0.39, negative due to loss being best at 0. This indicative result shows that the evaluation set accuracy is correlative with the CNN computed GAN-train, as the test result is dependent on the class-specific accuracy (TPR or TNR). This was also found when doing bias-traversal, as some patients had very skewed class-specific GAN-train values, and doing bias-traversal improved their evaluation accuracy.

Bias-traversal was working towards the direction it was intended as is shown in Table 5.5 and Figure 5.1. When the generator was skewed towards one class as indicated by the TPR/TNR difference, the skewness affected the evaluation set as well. Using this method corrected the generator that had too much skew as proven by the result on the evaluation set. The effort to balance the TPR and TNR also increased the value of the GAN-train accuracy. Although this indicates that it would increase the accuracy of the evaluation set, this was not validated as the search for latents was too computationally intense.

By traversing latents to negate the bias in generation output we get a more balanced training set for the downstream classifier. With longer computing time, the whole set of latents could be searched with a low threshold such as 0.01, to essentially produce a perfectly balanced dataset. Further exploration might also be commenced on the found latents that balance out the dataset, as these might be common for both classes and indicate features that are not necessarily useful but instead mainly cause bias in the dataset. They could also reveal areas that classifiers should put less emphasis on.

Overall, this method improved the balance of the dataset with the LDA and the CNN classifiers without reducing the quality metric of the generation. As the search was too slow to do, better search algorithms similar to AttFind should be used to find the controllable latents that help the downstream classification task. It is also notable that the bias is different in the LDA and CNN classifiers. As seen from Tables 5.1 and 5.2, in the LDA classifier the class-specific accuracy is generally much better for class 2, while for the CNN the situation is the opposite. Therefore, the downstream classifier must be considered when generating images with latent controls. This method showcased how the generation can be controlled to make a dataset with desirable features. In this case the desired feature was a balanced downstream classifier that also improved the evaluation accuracy. By neutralizing skewness, the fake images might have been more useful to the CNN training. However, finding the proper latents for this would be an impossible task without a search algorithm.

Changes to model architecture were not explored quantifiably. In the exploration phase of the thesis, different classifier heads were tried with a considerable amount of effort. Their accuracy for the evaluation data was similar to the CNN, though slightly worse in the trials, and therefore further analysis was left out. Differing the number of mapping layers was also tested, as well as using the skip-layers or residual layers in the discriminator. Using dropout in the discriminator worked well but was not added to the final analysis due to the unnecessary need to validate multiple hyperparameters. It was decided that although some improvements were definitely attainable, these few percentages are not necessarily interesting, as they are model and case specific, and I wanted to focus more on general results in this thesis. One distinct change I tried was using the InfoGAN methodology in StyleGAN2, as this was easily accessible in the StudioGAN code library. The continuous variable it learned controlled the brightness of the image while the dis-

crete variable was trained conditionally using the class labels. The idea was to use this to control the generation of the output, but this was also eventually ignored as it was another parameter to consider, and it increased the training time considerably.

Statistical comparisons of images revealed that the statistical parameters of minimum, maximum, average, standard deviation, and histograms were similar in the original and fake images. A nearest neighbor search was implemented to see whether the fakes were similar to only a subset of training images. As the distribution of nearest neighbors of fake images to training images was very even, this was concluded to not be the case. The best metric for mode collapse is typically the human eye, but it is not usable in this case. Although the fake images generally might have a reduced set of features compared to the training data, different methods would be required to reveal what limitations they have and if some training data are more represented than others. The PRDC (precision, recall, density, coverage) metrics also aim to achieve this, but besides observing them in heuristic ways, they were not used for further analysis. They were generally increasing throughout the training as was seen in Figure 6.1.

Visual inspection revealed very little because DMS images are hard to evaluate visually. One general observation was that only with very long training (200k+ steps), certain features would start to appear. As an example, a couple of images in the training dataset had a much more distinct water peak compared to others. This feature appeared in the generated images only after training for a very long time. It did not appear in all images, but instead very rarely, similar to the training set. That image was compared to the one in the training set and while the water peaks were similar, other features were different. This shows how the network can find individual features (or *styles*) and the final image is a combination of them. The combination should resemble those in the training data as the generator outputs a similar data distribution, and is therefore not a random set of features, but instead a learned one. The set of features should act according to the rules of the data but also vary in ways that they necessarily do not in the original data. For example, fake images of a human might have a differently sized head, but never two heads, unless the training set had images of conjoined twins. For DMS images this could be validated by training a GAN on data that should never have two distinct curves in the same image and seeing if the generator would generate such data.

Often latent traversals are visualized to find features, but I did not focus on this beyond brief testing as this was the theme for RQ2. The brief testing was done by smoothly changing the conditioning from the integers 0 and 1 to a float value between 0 and 1 for example to $[0.1, 0.2...0.9]$. In this way, the produced image is idealized as for example 10% class 1 and 90% class 2. The observation did not reveal anything interesting about the data. This transition is sometimes used to understand a gradual change between two classes.

Detrended data comes from preprocessing similar to what has been done in the original research. In the original analysis of the data the authors noted a "temperature rise, which caused a baseline drift" and they "remove a dimension-wise linear trend" from some parts of the data [4]. By contacting the original authors, I found out that this method was a pixel-wise trend removal using SciPy [114] detrend function. Runs with "D" in Table 5.1 and later the "run 5" shows my results with a similar approach. The original code was not available so the exact implementation and therefore the results differ slightly. I used the 'constant' type of detrending for all 4-measurement sets. Essentially, this takes the pixel average for each 1200 pixels for each channel individually, and averages that pixel, reducing the dataset size by 4. This increased the accuracy of LDA classification before the GAN part and increased the classification done with fake images when the GAN was trained using the detrended set. It is unclear what the improvements were exactly in the original research using this detrending, as the original values before detrending were not reported.

In the original research classification algorithms for the IDH data were compared using grid search for parameters and nested cross-validation to validate the best-found methods. Comparison between KNN, XGBoost, Decision Tree, Random Forest, Support Vector Machines, and Linear Discriminant Analysis yielded LDA as having the best accuracy, specificity, and sensitivity of 86% with the detrending [4]. I did not search for optimal parameters and instead used LDA with the default setting in SciKit-Learn with priors $[0.5, 0.5]$. My result seen in Table 5.1 was 78% compared to the 86% presented in the original paper. This discrepancy is due to different detrending details as well as lack of a parameter search. The exact methodology of the research paper was not available, and I was not able to repeat the results despite some effort by trying out similar preprocessing and parameter search as described in the supplementary material of the paper. The best accuracy I was able to reach was 81%.

Taking the average of the 4 measurements done in successions is intuitively useful. By measuring the same tissue 4 times the effects of randomness and environment are reduced. This approach would of course take 4 times as long to do, increasing the measurement time from 13 seconds to 52 seconds. The better results could also be attributed to luck as the smaller dataset has a higher margin for error and should therefore be validated with other datasets using this approach. This is also somewhat confirmed by the 'run 6' where I only used the first measurement of the 4, reducing the dataset size again to 88 and achieving improved accuracy with the LDA, though not with the CNN. Considering that the GAN was trained with only around 88 images, it is impressive that the network is able to produce fake images that obtain better results compared to the originals with LDA classification. The training converged faster as well. It does seem that even for a very low amount of data the GAN can learn representations from the data to be able to produce similar, but not the exact same images from the data. It likely helps in this case

that the images are static. Further analysis into the smallest trainable amount would be warranted and this analysis should include better methods to analyze the mode collapse event to ensure that the images represent the total distribution instead of the distribution of a few samples.

By analyzing pixel values during the temperature rise I found no clear trends even when analyzing each sample separately. While there was a clear temperature rise as noted in the paper, its effects on the data were unclear. Whether detrending is generally beneficial or related to the temperature rise remains unknown, though it does appear in other DMS literature as well to 'detrend linear drift' in data [115]. Using the information between subsequent measurements could also be utilized. GANs are also used for sequential data such as T-GAN for video [116], and using approaches that use the sequential measurement information like the detrending averaging could provide us with methods that are better suited for the setting where the measurements are done sequentially, as in for example surgical operations. The effects of drift due to environmental variations could be an interesting study using the latent traversing capabilities of GANs.

The advantage of GAN-based augmentations is that they are representative of real data. The advantage of the traditional methods is that they have a strong mathematical basis for many machine learning operations. Both augmentations can add complexity to the dataset without impairing the classifier, but both can also impair the classifier if used improperly. Beyond these simple interpretations, the GAN may be used in more complex ways to control the augmentation. Using style mixing, latent transversals, or physics-guided methods [60] to control the output of the generator, the GAN can produce augmentations of very specific and exact kinds. In an example paper [117], the authors trained the StyleGAN2 in a manner where it could produce equal number of eyeglass users for both genders, even though the eyeglass users in the training data was heavily skewed towards one gender. With similar methods, both DMS classes could be equalized to have an equal number of common features in them. This kind of data would train more balanced classifiers that do not rely on common features and instead focus more on class-specific features. Controlled generation in this thesis included using the bias-traversal as well as thresholding the generated images according to an evaluation metric. Further controls could have been exerted on the noise level, randomizing the initial image, truncation, and latent traversal on other spaces beyond Z. In addition, further conditioning could have been tried, a simple example being conditioning with the temperature or humidity values of the DMS instrumentation. The quality of the generated images can be validated either using evaluation metrics or validation data. Validation data is a part of the training data that is not used for training but instead to validate the results. The test or evaluation data is what would come by in a real-world setting and it can't therefore be used to validate the training.

The traditional methods for data augmentation are based on random transformations and image manipulations, such as brightness, scaling, rotations, and 60+ other methods common in medical imaging [64]. They alter the image in ways that it does not necessarily resemble the original form anymore, as was seen in Figure 2.6. The traditional augmentation used in this thesis was already very strong with the DMS data and the GAN augmented data did not complement it. When tested with other DMS datasets, the same traditional augmentation pipeline trained all the other datasets very well, revealing that it was very strong on its own. Improvements attained by GANs in other research may have had a worse set of traditional augmentations used, as typically the comparison is not examined thoroughly.

The main advantage of GANs based on literature is for very unbalanced datasets, where only a few labeled data are available for some classes, while there is an abundant amount for the rest. The dataset in this thesis was balanced and this could also explain the lack of accuracy improvement when using the CNN classifier. It could also be due to the parameter setup in the GAN or the CNN classifier. Due to the large number of controllable parameters, the results are very hard to validate, as any result could be due to specifics of the implementation and might not generalize to other data. A systematic literature review on medical data augmentation [65] notes that generative techniques are useful especially when controlling the generation to provide specific complements to the data. Another recent paper [66] compares the downstream classification task between different generative models and downstream classifiers. They find that even the change of downstream classifier from one CNN architecture to another tipped the scale from negative to positive value for the augmentation. The general sentiment from literature [55], [67], [70], [118] is that deep generative models are best used together with real data, and in combination with traditional augmentation techniques. It is often also noted that due to the simplicity of the traditional techniques, and the very finicky nature of generative models, it is best to use generative models only after comprehensive testing without them. Other advantages of GANs often overshadow the possible improvement in naive augmentation. These include especially normalization of distribution shifts, improved fairness, increased privacy, and the possibility of controlled generation.

Why would generative data help instead of simply training a discriminator is an interesting question. In the end, intuitively it feels that we are making something out of nothing, which should be impossible. We are training a generative model with the same data that we could train a discriminative model. We are then using that generative model to generate data based on the original data to train the discriminative model. There is a view that generative models tend to generalize better than discriminative models when the training data is limited. The task dynamics of the adversarial process expose a different set of knowledge compared to the discriminative perspective. The method of learning that the GAN uses is different enough from what a classifier uses that, especially in the low

data setting, the adversarial advantage might be more pronounced. Another viewpoint is that in training a classifier there is generally an endpoint. We can train a GAN forever, essentially learning new views of the data for as long as we want, if we just avoid mode collapse and other training issues. A classifier might be happy with some features, but the adversarial learning objective is never satisfied. The generative model might also put less emphasis on outliers that confuse classifiers. These and other views are still argued over in the literature of generative and discriminative modeling, and there is no consensus on why GAN-based data would be useful for a downstream task in the first place, or if it is simply a very powerful augmentation method for specific situations but it also has its limits. Although the results for this task were negative in this thesis, based on the overall experience and literature results it seems to be as likely that the results could have been positive as well, by only a small change in some parameter values. As synthetic data was used and improved some classifiers and also was successful in another pre-training task, the answer to RQ1 should be 'yes, probably'.

6.1.2 StylEx

In the StylEx results all common visible features present in the original data are also present in the attributes found by the algorithm. The most intense curves of the DMS representing the biggest dimers and monomers appear as unique features, but they are also together correlated with other shapes to form more complex features. Features that are not visually detectable or appear only in some images of the training data are also present. All the features discussed here are shown in Appendix A. One feature was illustrated in the dashboard app in Figure 5.2 and a few other examples are shown in Figure 6.2.

Figure 6.2. *Image of three different features found with StylEx. On the left, the monomer curving to the right together with* $(0,0)$ *artefact. In the middle, the dimer together with some pixels on the water curve. On the right, a more crude multi-feature with both the start of the dimer and a crude version of the monomer. These images are also in Appendix A for a better view.*

The changes to the AttFind algorithm were successful, as the modified algorithm found many attributes that generated features that are well understood by visual inspection and have a linear relationship with change in class classification. The other implementations have not compared multiple classifiers. This is an important study to consider, as a worry is that the classifier input in the training part of StylEx would have effects on the training

and results [86]. The relationship between the two classifiers (MobileNetV3, LDA) indicates that for this dataset the effect of the classifier in StylEx did not control the model's behavior for explanations. Instead, the explanations seem to be more general changes in the features of the image that represent the class. Proper statistical analysis was left out and agreement between different classifiers should be quantified for further validation of the method.

The lengthy discussion with an expert [27] and the main designer of this specific sensor used for the dataset confirmed that these features are very sensible and not random. He confirmed many observations and added his insights. Some of the features found in the attributes are indicative also of the data collection process or biases. As an example, the $(0,0)$ pixel has a value of 0 for every original image. Due to the preprocessing steps, the nonzero value was replaced, and the pixel value was more intense in one class compared to the other. The algorithm also found this nonsensible attribute that was a result of preprocessing instead of anything meaningful in the data itself. This kind of information is useful when determining biases in the dataset and features that might exist due to unknown reasons, such as issues in electronics or certain environmental factors.

The general sentiment from the discussion was that these features are clearly representative of the events behind the DMS alpha curve formation, as they can be visually confirmed to represent the curves. Both monomers and dimers were present and in the correct polar dimensions. High intensity in one dimension was seen in the other dimension in the features as well, such as if a high concentration of positive ions is present in the gas, then that feature is also seen in the negative dimension. We also observed features that presented the other typical a-curve behaviors such as first leaning to the right and then curving to the left. Possible fragmentations were also present as features, but they are quite rare and should have small intensities, and instead they may have also been some mystery or bad features from the method. While fragmentation events are not explicitly excluded, the DMS data collected using IonVision equipment should not contain them in meaningful observable amounts, as they typically occur at very high field strengths that the equipment cannot reliably achieve.

Blob features that are more general pixel areas were found. This is also due to how the generator works, as these belonged to the lower level torgb-layers, when the image is only $4x4$ or $8x8$. Interestingly these fell in areas where the curves typically started to have more curvature or changed direction. Some of them also appeared in areas where no activity should be present. They could be indicative of areas where splits in mobility happen, areas of some fragmentation events, or areas of "ghost" features. The blobs appeared in multiple different attributes and the classifier had agreement between them, meaning that these areas are important for classification between the two classes. The most interesting and understandable features were in the two last layers, the $16x16$ and $32x32$ torgb layers. The blob-like features indicated areas of interest, but it was hard to

make deductions about them besides single pixel values in a general area. Although this could represent a probable area where a split happens in that specific class more often compared to the other, the features found in the higher layers were often more meaningful and had a similar or better informatic value.

Small variations in the tangential axis of the curve of the RIP-feature and the main monomer-feature were observed in the found features. It is hypothesized that the amount of water in the water cluster is responsible for this, as the mobility changes with the cluster size and the size is dependent on the air humidity. The conclusion from this would be that the attribute is found because some sized water clusters are more apparent in one class compared to the other, or alternatively there is a humidity bias in the data. This was also apparent in what were likely ammonium-curves as they were below the RIP-curves. They also had a distribution-like spread. One interesting feature regarding this was also a distinct normally distributed feature on the right end of the image where the main RIP peak curves to. This could represent the whole distribution that is due to the size differences in the water cluster. An alternative explanation is the jump to the next S_v level as the sensor electronics make some noise.

Electronics related artefacts and "ghost" features were also discussed. The most apparent was the very large blob at $(0,0)$. The attribute that the StylEx picked in this case related to the preprocessing resizing operation, and not to the DMS data itself. No knowledge of the process behind the formation of the dispersion plots is used to find these attributes. Therefore, these kinds of features representing biases in the data, artefacts from preprocessing, or differences due to electronics or environmental factors are found as features to support one class over the other. Such an outlier tends to get picked very easily as it is a statistical leverage point for classifiers for being far away from the rest of the distribution [41].

The non-interesting findings were interesting as they reveal how the classifier has a bias towards features that are meaningless in the true dimension of the data but meaningful in the data dimensions used to classify the data. Other interesting ghost features were related to areas where there should be no activity. Especially on the bottom right, but also the left side of the image left to the dimer-curve. The bottom-right area should be devoid of any activity due to the proton affinity of the water clusters and the sensitivity of the sensor, and that area is generally considered useless [18]. Features appeared even below the possible ammonium curves as well. I did not analyze these features. However, taking into consideration that the classifier uses them to make the decision between classes, this fact would warrant some inspection into possible biases affecting the classifier. They might be the result of similar processing as the $(0,0)$-feature, but they also correlated with each other, meaning that some attributes had them all together. One possibility is that the classifier never learns those areas in the images, so they get assigned randomly to one class over another.

Further analysis of the features could be commenced in a similar manner to the $(0,0)$-artefact feature. Doing statistical analysis of the datasets for each class or similarity searching using templates built from these features. This would be interesting and was something I intended to explore, but did not have the time for. Especially those features that appear with very low intensity but still distinctively, such as the curvature of the main monomer to the left, with another feature going to the right at the same time. This could mean that two analytes traveled together until the separation field got high enough that their mobilities started to differ from one another. This type of information could be used to gain new knowledge about the dataset. As the feature extraction of DMS data is notably hard, but useful [15], it is interesting to note how the completely algorithm-driven method provides visually informative results. Extracting quantitative features would require further development of the method, as it currently presents itself in visual form for a human evaluator and is also initially very labor demanding.

A quantitative exploration of these results could be completed by exploring the dataset, especially at or around the pixels found in these features. Perhaps using a "template" figure that only consists of the specific attribute and background noise. Such postprocessing would allow finding and confirming features in the original data and could lead to building a better classifier based on these features, and additionally help in understanding the data better. A larger number of images could also be averaged to find average differences between classes. Averaging the different classes of the training datasets revealed general areas of difference that were also found in StylEx attributes. While these could not alone be used to train a classifier, these features along with others could be used to train one.

The large blobs could also be analyzed using heatmaps. Images with multiple features could be analyzed using nearest neighbor search for all pixels above a certain threshold set by visual inspection or top-k % cut of the intensity distribution. The correlation between the features could also be inspected by simply picking the highest intensity points from an attribute with multiple features, and testing if those peaks correlate more with one class than the other. Besides technical analysis, it would be interesting to find why one class would support general features like water peaks or ammonium peaks more than the other. The feature-hypothesis for this would be that the samples have molecular differences that together with the CO_2 evaporation process cause these peaks. The bias-hypothesis would be that on average these carrier gas molecules were more present in the data during data acquisition, although the measurements were completed overlappingly and not in order. Whether they are real features of the data or biases in the data collection can be analyzed with further controlled data collection.

The distributed RIP-curve could be analyzed further together with the ammonium curve. The environmental factors could be the reason behind this, and they could be analyzed using the instrumental parameters provided in the sensor, as it saves humidity and temperature values during measurements. A dataset specifically done in an environment with

highly changing humidity could be utilized in this. Besides the analysis of these features, they could be used to normalize the dataset regarding the environmental factors or do a more controlled dataset augmentation to enrich the samples with synthetic changes to the humidity. The effects of environmental factors on DMS sensor output are not completely clear [15].

Explanations for the classes using features were generally left out of the analysis due to the complexity of the subject. What separates DMS images of IDH-normal and IDH-mutated tissue specimens is not well explained with conventional visual inspection tools. From the biomolecular perspective studying changes in tissue such as the Warburg effect in the case of cancerous tissue reveals some insight into how some parts of the cell structure should differ, and therefore the molecule analytes in the gas should also differ. However, the path from cellular differences to the output of the sensor includes a lot of steps that make the outcome hard to analyze. Ideally, each a-curve in a good quality DMS image relates to a single substance, and classifying only by finding curves in the image is an intuitively good idea and has also been successful [38]. With heterogeneous gases, however, the gas is a mix of unknown analytes, and the substances are hard to identify. Studies such as the research done on the IDH mutation might reveal what alpha curves are the most important to distinct between that enzyme mutation. There is still much data in other regions and features that are not well understood. Typical visualization tools to help understand the images are similar to those used in other medical images, where intensities are balanced using sigmoids or logarithms to highlight contrasts. Finding alpha curves that differentiate the classes would allow a measurement setup where the sensor scans only over those alpha curves, making the total measurement time magnitudes faster.

The results from the StylEx method differ from the typical contrast tools or from the usual neural net tools such as Grad-CAM. By using the fact that the machine can generate samples from two different classes and interjecting the classifier decision process into the generation learning process, we can manipulate the StyleGAN2 to learn attributes that the classifier uses to make its decisions. We find that the features are singular a-curves, combinations of them, or some other features that are quite hidden. We can visualize them and see how the DMS images would have to change for the classifier to flip its decision between classes. Using the analogy of cats and dogs, this method can add singular attributes such as the ears of the cat on a picture of a dog to fool the classifier. In the case of DMS, the features that the method revealed are interesting in that they are hard to extract with conventional tools and seem very locally edited constraining to single features. The next approach could be to find images that a classifier very confidently classifies wrong and see how the StylEx attributes change this class: what features in that image are particularly altered to flip the class from ’wrong’ to ’right’?

General sentiment about the method is that the attributes or styles found with StylEx showcase that the black box nature of neural networks can be opened for insight. This method specifically revealed that not only the GAN is explainable, but the decisions of the CNN classifier are also explained with these features. The correlation between CNN and LDA showcased that the inclusion of the CNN decision-making to the training of StylEx is not the reason for the change in the CNN decision-making but instead a real feature.

Further use of these features would start for example with feature extraction and training a classifier based on these features, making class templates or archetypes to build a prototype for any class, or using similarity distancing, for example cosine distance that has been used with DMS [99], to match the feature to a class. Additionally, these could be used to do a controlled augmentation to augment a dataset or use the tool for knowledge discovery to find features in the data. Automated pipelines would make it possible to gather these features from data without supervision and gain knowledge of DMS data to help build chemical "fingerprints" for diseases. Notably, the continuity of the alpha curves has not generally been used in classifiers, although recently some good results have appeared for example in [39], where using the knowledge that the alpha curves are continuous in the image improved the classification accuracy. More testing is however required to confirm quantitatively the meaningfulness of these features as representative features for the mutated or normal case of the IDH 1-2 enzymes.

The explainability of an AI system is inherently limited to the data and the models, and thus, the explainability portion of this thesis notes that the explaining is related both to the action of the AI (the classifier) as well as the data used (DMS data). Therefore, biases in the AI and biases in the data are intertwined in the results. Representations besides the biases proved useful in explaining both the AI and the data. Generally latent discontinuity often happens when training generative models with limited data [75], but in this thesis, a strong traversal was found both in Z and S-spaces with very smooth interpolations.

The StylEx and in general the methods for this thesis were examples of using modern generative algorithms for data analysis. The research in the last 10 years provides thousands of alternative methods, optimizations, tweaks, and experiences like mine. While the results did answer the research questions that were set out for this thesis, no definite answer was found whether these methods are the worst, the best, or something in the middle. Especially for explainable AI methods, a thorough validation is necessary if regulatory processes will start to require them. Therefore, the exploration of methods should consider also how common the method is, and perhaps focus on methods where explainability has been researched.

A follow-up StylEx publication [108] from the original team showcased the use of StylEx in a new setting and with more comprehensive analysis. Here the authors form a complete pipeline to use the StylEx method to first find interesting features and then create

hypotheses for what they mean. They present the final attributes to an expert panel that decides their usefulness and meaning. They use eight different medical datasets. They find medically relevant attributes that explain diseases in images, but also, they find confounding attributes such as makeup predicting low hemoglobin due to gender association. Dataset artefacts are also found, similar to the findings in this thesis. They also find plausible new scientific discoveries, for example certain fundus associated with self-reported gender.

Their new paper is a great improvement on the original and showcases how the StylEx method can be utilized further. With a complete pipeline, they create new hypotheses that can be validated via research. They also highlight that similarly to the features in this thesis, the method also finds biases and problems in the dataset that will motivate better dataset curation procedures. As the team is from a large tech company (Google), it is possible that the method is integrated into a large foundational model, such as the Med-Gemini, or other future AI medical models. Alternatively, it can be used by itself as a research tool to explore dataset features and link them to physiological mechanisms. The authors conclude that beyond the "where", the method helps to explain "what" changes, and "why" the change happens. The why is linked to the class-conditioning while the "what" is linked to the semantic feature.

I presented this paper here shortly as it is highly relevant to this topic and is a follow-up to the original research. It was published and I only found it after most of the research work of my thesis was done and I could not use its findings to my advantage. Especially the Supplementary Note 4 would have been helpful, as it gives more details about the StylEx training. Importantly all their datasets had over ten thousand images with $512x512$ resolution. They ignored coarse features altogether from the first few layers. They changed the AttFind mechanism, but it was still a lot of manual tuning for them to find good thresholds. Comparatively, the method in this thesis was based on automatic finding of areas with linear change. I contacted the main author regarding this. They responded that the line fitting was an interesting suggestion and agreed that it can be more robust to outliers in the S-value. They also agreed that their simple thresholding method could be improved. However, they also added that due to the need for human evaluators, and having only a handful of classifiers, they did not investigate the thresholding method as thoroughly and were happy with what they had.

6.2 Related future works

There were many methods that I tested and researched that did not end up getting used in the final methods in this thesis. Some of these are worth discussing as they strongly relate to the subject and could be useful with proper setup. The main focus is on knowledge transfer, such that we use knowledge beyond what we have in the one dataset to make decisions.

Transfer learning, meta learning, pre-training, unsupervised learning, and similar concepts are all related to the idea that a *knowledge transfer* should occur prior to or during the training with labeled data. In this thesis, I trained a GAN using only labeled images with either around 80 or 350 images. A GAN is typically trained from 10k images upwards. I also trained a CNN that was pre-trained on a very different type of dataset. Trying to train the ConvNeXt CNNs without ImageNet pre-training resulted in much worse results with much longer training that did not necessarily ever converge.

A deep neural net requires a lot of iterations to settle all of the parameters that are presented as weights in layers. A pre-training scheme is idealized as having these weights already established in a manner that is useful from the point of view of the training that we are interested in. The training is sometimes called fine-tuning, as we are only finely changing the mostly already established weights. The weights may have statistically pleasing behavior, such that the settling will take less time when fine-tuning with a new set. Alternatively, the weights may already have perceptive knowledge about the subject, for example when pre-training with another DMS dataset would lead the network to have knowledge of the general types of features in DMS data. There could also be some meta-tasks that give the object information about what kind of direction the learning should aim for, using prior knowledge about the goal. The basic idea of transfer learning is to "utilize information from one task to improve the learning for a second one" [119]. Use of previous knowledge is done in all state-of-the-art implementations, not only in computer vision, but also in textual generation and other AI applications. Lately especially the use of transformers has made transfer learning stronger [120].

Information transfer from previous knowledge can be achieved in a multitude of ways. The general approach however is to gather data that is either general or similar in domain and gain insight from that. General data could be anything from natural photos to medical images, while in-domain images would be preferably DMS spectral images, or something very close. The images may be labeled, but they don't necessarily have to be. The network can be taught to find general and repeating features in the data. When we then later want to train with our specific set, such as the 350 images, we fine-tune the already pre-trained weights. This speeds up the convergence to the final weights as well as improves the final accuracy if the pre-trained knowledge is useful to the network weights. This is commonly the case in the limited-data or the few-shot setting, where the

number of samples is rarely enough to provide all features and generalize the network enough and not overfit. The more complex the model, the more data is required to avoid overfitting. Not all weights need to be fine-tuned, and it is possible to freeze some parts of the network and then only train a small part. Typically the trained part is the classifier head in a classification task.

This approach is applicable to most generative AI implementations including StyleGAN2 [57], [121], where it has been shown that pre-training outperforms methods that do not use pre-training [122]. Retraining on the same weights is the simplest of the knowledge transfer methods, as for example the more complex meta-learning instead learns how to learn, and self-supervised methods label the data themselves using pseudo-labels. The StyleGAN2-ADA authors noted that a single run was reduced from around 25 000 kimgs to 1 000 kimgs by utilizing transfer learning.

Beyond focusing on the weights, any other information related to the data may also be used in cleverly set training schemes, even if all the data is not in the same format, such as images. These include data on chemical properties of the samples, physical laws underlying the events, sensor information, or knowledge from text-based literature, for example using captions of medical images to fuse in information to the model [44], [60], [68]. Recently a large foundational model used multiple different sources including images, electronic health records, voice notes from doctors, literature search information, and even chat-AI discussions with medical doctors and fused the information into embeddings using transformers, achieving state-of-the-art results in Medical-AI domains [123]. Similar methods are used in GANs to control the conditioning or embedding multiple sources of information into the same model.

Transferring knowledge is not without its challenges. It has been shown that using pre-trained GAN and then fine-tuning it might cause the final model to replicate the images more, risking privacy and even making the model worse [124]. For downstream classification, applying transfer learning to GANs has however been shown to give better augmentation results, even when the source and target distributions have large differences in image quality, such as using images of natural objects in the pre-training and then fine-tuning with medical images [121].

The only research found for this thesis that had used GANs with a similar type of data was a GC-IMS type situation that used self-supervision to improve the accuracy by utilizing unlabeled data along with the labeled data [37]. They also employed a classifier head in the GAN. I tried similar approaches of using a classifier head in the GAN with for example the AC-GAN implementation. The method of using a classifier head was satisfactory, but not impressive. Using the GAN as a classifier remains a popular choice instead of generating images for a downstream classification task [3], as adding only one extra layer to the model gives this new capability for classification. Another related study trained a classifier

using a pre-trained CNN to classify FAIMS data and found that accuracy improved while the training time was roughly halved when using the pre-trained model compared to a completely fresh model [125].

These results indicate that using knowledge that is out of the training dataset distribution should work very well with DMS data. While labeled data is scarce in medical settings, unlabeled data exists in more abundance. Using DMS images from different sensors and different image resolutions is therefore a useful strategy for pre-training schemes when training classifiers or deep generative models. The easiest way to test this is to simply gather any DMS data, even from ambient air, and see if pre-training with that helps the fine-tuned classification rate. In the more general context of supervised learning in healthcare, data scarcity, quality, and heterogeneity are argued as issues to move beyond supervised learning from an empirical risk minimization theory to improve machine learning results not only in image recognition but in other fields as well [2]. Essentially, using only labeled data is seen as a health risk, as most of the useful data ends up unused and unutilized.

Transformers have been mentioned multiple times so far in this thesis. They are often compared to CNNs. Visual transformers (ViTs) may require less amount of labeled data with their strong unlabeled learning capabilities such as masking [44]. There are GAN implementations that use attention mechanisms that surpass CNN-based models such as the StyleGAN2 in certain datasets [126], [127]. The long-range dependencies in the attention mechanism are alluring as the DMS data is very structured and certain features should be very related to each other, but with a long span for the relation in the image. For example, if two analytes have very similar monomers, but split up in the image in a way that both have a drop in the intensity, this dependency should relate to the total strength of the monomer a-curve before that split due to the limited number of reactive ions. The receptive fields of CNNs are built in a more stacking and global way and the features they capture tend to differ, as transformers have theoretically infinite receptive fields and can capture fine-grained details with long spans.

An interesting model I considered largely for this thesis is the PixelSNAIL [128]. In PixelSNAIL the autoregressive recurrent neural network is not exactly a transformer, but the memory mechanisms have a resemblance. The image is built one pixel at a time. If we look back at how the DMS equipment operates, it also builds the image one pixel at a time. A more interesting approach would have been to try to build the image one a-curve at a time such as in the approaches in [38], [39]. This kind of pixel-regressive generation was interesting, but eventually abandoned, as the models were old and they were not tested for limited data settings and are also not mentioned in recent literature. However, the single pixel-based methods are still very promising, especially with the strong pre-training using masks and an interesting modern approach could be the MaskGIT [129].

I tried the vision transformers for classification. Initial results were promising with accuracy being in the 80% range without any pre-training and with only small minimal adjustments to the parameters. The training took considerably longer compared to the ConvNeXt V2 CNN, which achieved similar if not better results, albeit it was pre-trained on the ImageNet. This small testing session however confirmed that ViTs have no trouble training with DMS data, even when the training samples are very limited. I also tried with another dataset of three different classes and around 150 total images, and the results were close to the LDA results reported in literature. These were done during the experimentation phase of the thesis and are not reported beyond this, but included as they did bring in new information. The strong unsupervised training and the intuitively strong idea of using masked image modeling (MiM) with DMS data make ViTs a good future study subject.

The representations learned during the pre-training with a highly variable DMS dataset should very strongly correspond to the alpha curve features and possibly other features of DMS data, as they generally present themselves in the dispersion plots in similar manner according to the types of curvature. It is noteworthy that almost any method mentioned to work with a neural network classifier also works to improve GANs. For example, the masked image modeling and the popular contrastive learning [68] have both shown good results in low-data settings of training GANs [75]. GANs are at the end of it typically just two neural networks battling against one another. However, testing such methods often still requires a lot of implementation work if the code is not provided.

Changing the conditioning is similar to changing the prompt of a chat-AI model. The StyleGAN2 trained in this thesis was conditioned on the class label. This was simply a number 0 or 1. That number was encoded by linear layers into a 512-long vector representing that class and this encoding is learned during training. Alternatively, we could encode any other information and use that for conditioning. We could include for example subclasses, environmental information, text, extracted features from other sensors, or any other prior knowledge. Text-based descriptions as conditions are mentioned as an important future approach [3] together with the physics-based conditioning [60] as well as in chemical sciences where the inverse method is applied with generative models to produce wanted features instead of solving for them [130]. Change in conditioning could help especially with humidity-based variation, as even subsequent measurements may have different results due to changes in the water environment of the gas. In [131] the author concluded that even in the simplest case of differentiating DMS images between skeletal muscle and intermuscular fat the system would not achieve a 100% accuracy due to variability of the external measurements conditions such as the humidity and temperature.

One interesting approach for DMS would be to focus on the sequence of the images rather than a specific sample. This was also highlighted by the results when averaging improved the classification rate. The change between measurements of the same class

could be used as conditioning information to generate samples that show variability due to uncontrollable factors.

I intended to use the WHO cancer grading groups of the patients as subclass conditioning. In the original paper, they detected in-group differences in accuracy with the cancer grading and this could have been interesting to study. This information was not available to me, unfortunately. If the model is trained with more broad conditioning, the generation could also be more useful as we would have more control. Then again, it is postulated that it is best to leave the machine to figure out the interesting controls, and then let the researcher search and look for them in the generator for later use. An author of a review on this topic concluded that "we believe that the real strength of GANs relies on their ability to learn in a semi-supervised or fully unsupervised manner" [132]. The original GAN was completely unsupervised [7].

Using synthetic data to pre-train is now a generally accepted pre-training method in recent large AI models [68], [123]. Generating data is fast and cheap and even if it is not perfect, using it for pre-training should still be useful, as even pre-training with completely different types of images (ImageNet) tends to improve later training. Controlling the generation of images for the pre-training scheme could especially fill in the "missing tails" of the long distributions that are feared to cause the models to deteriorate if trained on generated data with the incursion effect. This could for example mean that a GAN is trained on dataset A, and then images from that GAN are used to pre-train a new GAN that will be fine-tuned with dataset B. The middle-man GAN would help prevent private information leakage, while also improving the second GAN by providing pre-training improvements. As mentioned in Section 6.1.1 I did use the synthetic images to successfully pre-train a CNN for a different dataset, but I did not try pre-training a GAN. Pre-training with a large set of synthetic images would allow the network to settle most of the weights to helpful values and the fine-tuning would take a much shorter time.

Explainable AI had very promising results and other similar approaches could be researched and tried, as StylEx was only one example. There exists now a large literature based on finding the best representations that are semantically meaningful such as the results from the StylEx method in this thesis. Whether completely unsupervised like InfoGAN, or using other models such as VAEs, it is important to set the goal of either letting the machine find out what is interesting, or alternatively having more control on what kind of features should appear. As the features in the DMS are generally well known, the prior information could help produce a better explainer. In one example [81] the authors used a modified VAE model in a way that they were able to manually control the specific latent dimensions and input radiomics information there to control the synthesis. These controlled attributes were used to generate synthetic data that trained a classifier for cardiovascular images with great success for downstream tasks.

In medical diagnosis systems, the application of models has been greatly restricted due to the black-box nature of the models and regulatory issues. Using interpretable evidence of the nature of the models should then also help in the acceptance of using these models, as they act as 'proof' of how they work. An important consideration is that the method I used is especially for visualizing changes between classes. Alternatively, completely unsupervised methods would find general features that would reveal more insight into the inner workings of the sensor and not focus on class differences. Finding biologically relevant representations or reasoning from the data would be beneficial in the general acceptance of the technology.

Privacy and fairness relate to GANs in medical field very heavily [73], [118], [133]. Although the DMS images should not be traceable back to a person, they may still be classified as medical images by regulatory bodies. And due to these regulations, the handling of these images by AI models may be difficult. As synthetic images provide a privacy buffer, especially with a privacy-oriented model [134], they may be useful in sharing data between institutions. The lack of efficient metrics in GANs [1] however necessitates the expertise in both domains [72] to avoid possible leaks in private data if the progress and implementations are too rapid. It is often only years after the implementation when adversarial attacks find possible leaks in data. Besides privacy, the distribution shifts in ML are largely responsible for the slow deployment to the medical setting [1], and GANs are used to alleviate those issues [118]. Having control over distributions that relate to sensor, environment, or bias shifts is useful when the measurement settings are known to be heterogeneous.

6.3 Challenges and limitations

The transfer learning methods mentioned in the previous section increase the capabilities of the model, but also relate to another topic that was a constant challenge affecting the implementation of this study: computational power requirements. Besides that, some other factors that affected the results are also considered here.

Computational time is limited. Not only due to time itself, but to access to computational processing power. The final GAN models took somewhere from 4 to 6 hours to train. As I did one for each patient and had a total of six models, that time is then around 700 hours or just short of a month in computational time for only the final GAN models and not for any other computations, such as experimentation, StylEx, and the ConvNeXt training. In hindsight, choosing a dataset where I must test 22 times is not ideal. The experimentation phase was very limited by computation as no proper validation could ever be done and testing the methods in that phase relied on heuristic analysis.

I would have to verify each change in a parameter with multiple runs. The longest run I did for a single training was 25 hours when I tested what would happen if I trained the model for a longer time to produce Figure 6.1. This resulted in 500k steps or 2000 kimgs of training. With similarly sized datasets as mine, other researchers have completed these studies in the sub-500k kimgs range [84] but even 25 000 kimgs are not uncommon, though the dataset is generally larger in those studies.

The StyleGAN2-ADA research paper included some calculations on the computation time spent on producing the results and exploration for their paper [57]. The total single-GPU years they spent was around 135, or around 1.2 million hours totaling around 300 megawatt hours costing roughly 15 000 euros in 2020 energy costs. A single model with their "Human faces" dataset took around 0.7 megawatt hours. These numbers highlight the importance of computational power in AI research. The researchers of the most prominent papers are from non-academic institutions, where computational power is abundant, such as Nvidia for StyleGAN, OpenAI for ChatGPT, or Google for Med-Gemini. The same authors noted that a single model in one of their datasets generally required 25 000 kimgs of training with reasonable results at 5 000 kimgs. They also note that with transfer learning this dropped down to 1 000, highlighting the power of knowledge transfer. A lot of my trials were limited by computational capacity and I had to cut short many experiments.

The TCSC (Tampere Center for Scientific Computing) and CSC (Finnish IT Center for Science) offer computational power to the students at Tampere University. At the time of writing, the local TCSC and national CSC both have Nvidia P100 and V100 GPUs, the latter of which is superior and is the same GPU that was used in the StyleGAN2 research paper. The CSC offers 10 000 billing units (BUs) for students with more by application. An

important service that they also offer is the ePouta, which provides security for sensitive information, as is the case in medical patient data. By estimation, I would have run out of free hours in the first month or two of this thesis. I had the privilege to compute locally using RTX 3060 with 12GB RAM. In the winter months it provided nice warmth to the apartment, but in the last parts of the thesis during summer the extra sauna room was a suboptimal environment for thesis writing.

Computing scarcity in healthcare AI is noted to eventually affect the democracy of healthcare, as technological companies compete for the largest resources to build the best general artificial intelligence, and the energy resources are limited not only in materials to build the processing units, but also in electrical energy costs [135]. While research such as this thesis may confirm results from other papers, it is hard to make progress without access to the large computational power available only within the largest technological companies.

Besides computation, the disk space requirements might grow large. Saving so many models ended up taking some hundreds of gigabytes of disk space, but this was mostly due to having it free and therefore not stressing over it. A single StyleGAN2 model to save the discriminator and the generator took around 0.5GB of space for all their parameters.

Training a GAN is hard and is known to have troubles due to gradient vanishing, mode collapse, training divergence, and other problems related to the finicky nature of the two networks. Computational time spent to find proper parameters will be reduced in the future, but for now, a lot of exploration is generally required. The StyleGAN2 proved very good already in the beginning, and proved later that new advanced GANs are more robust compared to the fears of the early literature.

Software maturity is a concept that relates to computational time. While a lot of classical algorithms, such as LDA, are very mature and easy to use, the modern generative algorithms are mostly used within the machine learning community, or within large companies. As the progress is very rapid, the open software does not keep up. A large amount of time spent on this thesis was thus spent on debugging. Even the largest libraries have bugs and errors that are hard to detect. I had to, for example, manually modify a lot to make the models handle two channels as three is the norm (RGB). I found errors in the popular StyleGAN2 original code. A large battle was fought with the StylEx code. As the software matures and the computations get better, these methods will have a smaller learning curve and their implementation into existing systems will become reliable and cheaper. The implementation of a modern classifier such as the ConvNeXt can be done in under ten lines of code, while a GAN still requires very much manual effort even with the most used and well-developed code libraries. While the largest and most popular new models provide codebases, most papers never release the code, and the re-implementation might take months simply to verify the working of a single model.

Depth of the new subjects was a personal challenge during the thesis. First the DMS, and then the GAN. I had previous experience in training neural networks, but the generative side was new to me. Although most aspects are common, the finickiness of training a generative network proved a surprising challenge that took a lot of time to learn through experimentation and exploration, as I had to also compare different model families. The DMS is not something I had experience with earlier and it posed another learning experience. I was lucky to have a few talking sessions with the local experts who helped me start and cement my understanding of the subject. The substance behind DMS was important to understand to overcome the challenges of training a generative model with such small differences between images, and especially understanding what features are important and what to look for. For research question two, understanding the data from formation to representation was necessary to look for methods that would prove useful. Previous knowledge of these subjects would have likely driven me to a faster conclusion on what methods to use and I would have had more time to explore other interesting possibilities. Due to such a broad amount of new knowledge, it is very well possible that the results could be very different with more experience, and some errors may have propagated to the results.

Visualization is a large part of data science in general [136]. The challenge with DMS data is the large intensities in areas with low interest and low intensities in areas with high interest, as the total amount of reactant ions is limited. In similar situations with medical images, contrasting methods are used: a sigmoid-function or logarithm of the values are used to highlight also the lower intensities of the image. In the visualization for StylEx, I boosted the differences so that they appear visually to the viewer.

During the training of the networks, the generated images reached good visual quality very soon. It was then hard to tell how the rest of the training affected them by visual exploration. Visual quality was generally not used and quantitative metrics were used instead. Even when working with VAEs, that had very poor fine-grain details, my eye could not detect the low quality of the image, as the intensities were very low on the features that were of interest. Qualitative visual metrics are often used in GAN training [102]. During the work, I made a front-end website using Plotly-Dash that had 15 visualization functions to change the images live as I felt the visual inspection was important for my understanding. A similar library could be useful in general to explore features in DMS data. Besides visualization of the images, visualizing the 1200-dimensional data in statistically meaningful ways was tried. I applied UMAP, PCA, and t-SNE, but the dimensionality reduction was not very useful and probably not appropriate in this situation. Visualizing the latent spaces and their traversal was done using the images, such as traversing between classes. Visualizing the generated data distributions using frequency space tools could have been explored more. Especially visualizing the latent spaces and compressed versions of the image might provide insight.

Metrics and validation were a constant challenge. The results seen in this thesis reinforce this view that no single metric is the best and finding the usable metrics is an important effort, as they are the goal of the training process. While using multiple metrics is possible in a study like this, the realization of the models to real-world use must rely on metrics that are useful and robust. For downstream classification, the properly implemented GAN-train is a simple and working metric. However, it gives no information regarding the statistical or visual quality of the images. Studying their noise or frequency spectra and statistical distributions will make the synthetic dataset more reliable. There is still a lack of clear choice of metrics in this regard.

To assess the quality of the second task a visual interpretation was used. There are metrics used for quantification of disentanglement and completeness that relate to information theorems independence of variables. They are not generally yet used, however. An important metric for single feature generation is the measurement of change in local as compared to global editability. A good generatable feature has no global effect and only changes the local pixels. A good generatable DMS feature is a curve that is normally distributed in the C_V-axis with the intensity of the peak decreasing in the S_v-axis. This could also be metrified into an image quality metric. Similarity and distance metrics are useful, but human visual inspection is still the golden standard most often.

Validation of the results was done using averaging on multiple occasions. A common way of validation is done using cross-validation. In this method, the training data is typically partitioned into five or ten parts, such that one part is used for validation and the rest for training. The result is the average measured value between the folds. However, with generative data, a better solution is to average out the randomness effect with multiple trial runs as we can generate an infinite number of samples. One set of generated images even with cross-validation might have lucky sampling, while averaging over thousands of generated samples gives a realistic approximation of the average outcome of the generator. In a real-world setting, this can be applied using an ensemble of classifiers for the prediction, as inference is done in milliseconds, whether there is one classifier or multiple. In addition, six different models were compared to generally validate the GAN method. As they had quite different preprocessing approaches, it should give some general validation that the results were not specific to this dataset.

To further validate the robustness of the methods, similar-sized datasets from other domains could be utilized. The MedMNIST v2 [50] would fit this task well. Without validation, the methods might still only work due to large effort put into parameter search that does not generalize, or certain features in the data that are not general to the data but appear only in a specific subset. Especially as a lot of the analysis is qualitative, proper validation with other datasets should be done before employment into any real setting. Although it is probable that there is still some bias in the results, a lot of time was spent on considering fair and non-biased metrics and validation methods for the results in this thesis.

The limitations of this study arise from the above discussion and in addition some other subjects. The methods were only validated with a single dataset acquired with a single type of DMS sensor and may therefore not generalize to other datasets. Computational constraints and lack of software maturity caused iterations to take time, and this limited the exploration and comparison of different methods. Alternative methods that had ready code were tried but those that did not were ignored. The analysis of RQ2 (explainable AI) relied on visual human interpretation and the eye may see what it wants to see. Future ideas for more quantitative confirmation were discussed. Errors in code are very much possible, although critical operations such as test data leakage were very carefully explored to not exist.

Beyond the methodological limitations, some limitations on the research process and thesis writing also affected the results. The fact that a thesis must be written in a certain form guides the research process to follow a certain style and rhythm. A lot of alternative methods that were briefly explored were eventually ignored and forgotten, as they would not fit into the main story of the thesis. Research questions one and two guided the research.

The lack of experience, lack of research on the very specific topic, and including multiple distinct methods all caused potential for error. Although the theoretical foundations of GAN should work for the images of DMS, the research literature does not exist yet. Regarding the literature limitations, there is also the possibility that some of the research cited do not hold if reproduced for validation. Machine learning research moves very fast, and the methods might never be reproduced to validate the results. Most papers are only released as conference papers with minimal validation effort, and many are released only as preprints in the archive repository arXiv, including the original cGAN [59] paper with over 13 000 citations in Google Scholar. In the case of StylEx, all three reproductions I mentioned reported various challenges due to missing details. The results in this thesis are therefore also limited by the validity of the literature in general.

The lack of experience together with answering two somewhat distinct research questions and using new methods for both gives a high potential for errors. The results of this thesis should be understood through the limitations expressed in this section. In addition, in Chapter 3 I explained my motivations for the model choices to give more objective value to the study. The results should be repeatable with the methods and information provided in the main thesis report and the appendices.

7. CONCLUSIONS

In this thesis, I explored the use of deep generative AI to generate synthetic images that could be useful in data analysis of differential mobility spectrometry (DMS) images. The case of having a low amount of data motivated this approach. Two research questions were considered:

RQ1: *Could synthetic images be useful in training a classifier for DMS data?*

RQ2: *Can the generative AI be used to explain data or classifier decision making?*

The RQ1 had mixed results. With the LDA classifier, the synthetic images surpassed the baseline. With the more relevant CNN classifier, the synthetic images did not provide an advantage. With the LDA classifier, it was best to only use synthetic images, while for the CNN it was best to not use any synthetic images. This result aligns with the view in the literature that using GAN-based image augmentation may provide an advantage, but not necessarily. The synthetic images were mostly indistinguishable from the originals, and it is likely that with only small changes to the methods better improvements would have been found.

For RQ2 the StylEx method was applied. This method provided representations of the DMS images that explained what a classifier focuses on when deciding between two classes. These representations were distinct features of the DMS data, such as the alpha curves. The representations found in the model as attributes may be used for knowledge discovery about the data or for example controlling the generation to produce a dataset with desirable features. Validation of the found features requires human expertise.

For RQ1 the conclusions in the discussion were that the best approach of using GANs is a controlled generation, especially for class-imbalance. For RQ2 the conclusions were that this method both explained the black-box nature of neural network-based AI and also provided means for scientific discoveries. Hypotheses can be made by automatically finding features in the data that are semantically meaningful for a human to evaluate.

Overall, the generated images are promising and impressive. Their use in real-world settings will require more curated datasets and further validation. Future approaches should use unlabeled data and focus on quality metrics for the synthetic images.

REFERENCES

[1] A. Zhang, L. Xing, J. Zou, and J. C. Wu, “Shifting machine learning for healthcare from development to deployment and from models to data”, *Nature Biomedical Engineering*, vol. 6, no. 12, pp. 1330–1345, 12 Dec. 2022, ISSN: 2157-846X. DOI: 10.1038/s41551-022-00898-y.

[2] X. Gu, F. Deligianni, J. Han, *et al.*, “Beyond Supervised Learning for Pervasive Healthcare”, *IEEE Reviews in Biomedical Engineering*, pp. 1–21, 2023, ISSN: 1941-1189. DOI: 10.1109/RBME.2023.3296938.

[3] X. Yi, E. Walia, and P. Babyn, “Generative adversarial network in medical imaging: A review”, *Medical Image Analysis*, vol. 58, p. 101 552, Dec. 1, 2019, ISSN: 1361-8415. DOI: 10.1016/j.media.2019.101552.

[4] I. Haapala, A. Kondratev, A. Roine, *et al.*, “Method for the Intraoperative Detection of IDH Mutation in Gliomas with Differential Mobility Spectrometry”, *Current Oncology*, vol. 29, no. 5, pp. 3252–3258, May 2022, ISSN: 1718-7729. DOI: 10.3390/curroncol29050265.

[5] J. Haapasalo, M. Arola, S.-L. Lahtela, H. Haapasalo ja N. Kristiina, ”Lasten primaariset pahanlaatuiset aivokasvaimet: mitä olemme oppineet?”, *Duodecim*, vol. 137, nro 3, s. 271–279, 2021, ISSN: 0012-7183. url: https://www.duodecimlehti.fi/duo16037.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, Nov. 10, 2016, 801 pp., ISBN: 978-0-262-33737-3. URL: https://www.deeplearningbook.org/.

[7] I. Goodfellow, J. Pouget-Abadie, M. Mirza, *et al.*, “Generative Adversarial Nets”, *Advances in Neural Information Processing Systems*, vol. 27, Curran Associates, Inc., 2014. DOI: 10.48550/arXiv.1406.2661.

[8] B. B. Schneider, E. G. Nazarov, F. Londry, P. Vouros, and T. R. Covey, “Differential mobility spectrometry/mass spectrometry history, theory, design optimization, simulations, and applications”, *Mass Spectrometry Reviews*, vol. 35, no. 6, pp. 687–737, 2015, ISSN: 1098-2787. DOI: 10.1002/mas.21453.

[9] V. Gabelica, A. A. Shvartsburg, C. Afonso, *et al.*, “Recommendations for reporting ion mobility Mass Spectrometry measurements”, *Mass Spectrometry Reviews*, vol. 38, no. 3, pp. 291–320, 2019, ISSN: 1098-2787. DOI: 10.1002/mas.21585.

[10] J. Virtanen, A. Anttalainen, J. Ormiskangas, *et al.*, “Differentiation of aspirated nasal air from room air using analysis with a differential mobility spectrometry-

based electronic nose : A proof-of-concept study", 2021, ISSN: 1752-7155. DOI: 10.1088/1752-7163/ac3b39.

[11] F. Röck, N. Barsan, and U. Weimar, "Electronic Nose: Current Status and Future Trends", *Chemical Reviews*, vol. 108, no. 2, pp. 705–725, Feb. 1, 2008, ISSN: 0009-2665. DOI: 10.1021/cr068121q.

[12] J. L. Campbell, J. Y. Le Blanc, and R. G. Kibbey, "Differential mobility spectrometry: A valuable technology for analyzing challenging biological samples", *Bioanalysis*, vol. 7, no. 7, pp. 853–856, Apr. 2015, ISSN: 1757-6180. DOI: 10.4155/bio.15.14. pmid: 25932519.

[13] H. Borsdorf and G. A. Eiceman, "Ion Mobility Spectrometry: Principles and Applications", *Applied Spectroscopy Reviews*, vol. 41, no. 4, pp. 323–375, Aug. 1, 2006, ISSN: 0570-4928. DOI: 10.1080/05704920600663469.

[14] I. Haapala, M. Karjalainen, A. Kontunen, *et al.*, "Identifying brain tumors by differential mobility spectrometry analysis of diathermy smoke," *Journal of Neurosurgery*, vol. 133, no. 1, pp. 100–106, Jun. 14, 2019, ISSN: 1933-0693, 0022-3085. DOI: 10.3171/2019.3.JNS19274.

[15] A. Kontunen, *Tissue Identification by Differential Mobility Spectrometry*. Tampere University, 2022, ISBN: 978-952-03-2303-5. URL: https://trepo.tuni.fi/handle/10024/137428.

[16] M. Sutinen, A. Kontunen, M. Karjalainen, *et al.*, "Identification of breast tumors from diathermy smoke by differential ion mobility spectrometry", *European Journal of Surgical Oncology*, vol. 45, no. 2, pp. 141–146, Feb. 1, 2019, ISSN: 0748-7983. DOI: 10.1016/j.ejso.2018.09.005.

[17] A. Kontunen, J. Tuominen, M. Karjalainen, *et al.*, "Differential mobility spectrometry imaging for pathological applications", *Experimental and Molecular Pathology*, vol. 117, p. 104 526, Dec. 2020, ISSN: 00144800. DOI: 10.1016/j.yexmp.2020.104526.

[18] O. Anttalainen, J. Puton, A. Kontunen, *et al.*, "Possible strategy to use differential mobility spectrometry in real time applications", *International Journal for Ion Mobility Spectrometry*, vol. 23, no. 1, pp. 1–8, Apr. 1, 2020, ISSN: 1865-4584. DOI: 10.1007/s12127-019-00251-1.

[19] J. Jin, Y. Liu, S. Li, J. Hu, S. Liu, and C. Chen, "Identification of soy sauce using high-field asymmetric waveform ion mobility spectrometry combined with machine learning", *Sensors and Actuators B: Chemical*, vol. 365, p. 131 966, Aug. 15, 2022, ISSN: 0925-4005. DOI: 10.1016/j.snb.2022.131966.

[20] J. D. Zhang, M. J. Baker, Z. Liu, *et al.*, "Medical diagnosis at the point-of-care by portable high-field asymmetric waveform ion mobility spectrometry: A systematic review and meta-analysis", *Journal of Breath Research*, vol. 15, no. 4, p. 046 002, Jul. 2021, ISSN: 1752-7163. DOI: 10.1088/1752-7163/ac135e.

[21] S. Y. Park, Y. Kim, T. Kim, T. H. Eom, S. Y. Kim, and H. W. Jang, “Chemoresistive materials for electronic nose: Progress, perspectives, and challenges”, *InfoMat*, vol. 1, no. 3, pp. 289–316, 2019, ISSN: 2567-3165. DOI: 10.1002/inf2.12029.

[22] N. Oksala and A. Roine, “Toimenpiteenaikainen kudostunnistus”, *Duodecim*, vol. 134, no. 15, pp. 1433–1435, 2018. URL: https://www.duodecimlehti.fi/duo14436.

[23] A. Kontunen, U. Karhunen-Enckell, M. Karjalainen, *et al.*, “Tissue Identification From Surgical Smoke by Differential Mobility Spectrometry: An in Vivo Study”, *IEEE Access*, vol. 9, pp. 168 355–168 367, 2021, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3136719.

[24] M. V. Liberti and J. W. Locasale, “The Warburg Effect: How Does it Benefit Cancer Cells?”, *Trends in biochemical sciences*, vol. 41, no. 3, pp. 211–218, Mar. 2016, ISSN: 0968-0004. DOI: 10.1016/j.tibs.2015.12.001. pmid: 26778478.

[25] T. Saviauk, J. P. Kiiski, M. K. Nieminen, *et al.*, “Electronic Nose in the Detection of Wound Infection Bacteria from Bacterial Cultures: A Proof-of-Principle Study”, *European Surgical Research*, vol. 59, no. 1-2, pp. 1–11, Jan. 10, 2018, ISSN: 0014-312X. DOI: 10.1159/000485461.

[26] C. Ieritano and W. S. Hopkins, “The hitchhiker’s guide to dynamic ion–solvent clustering: Applications in differential ion mobility spectrometry”, *Physical Chemistry Chemical Physics*, vol. 24, no. 35, pp. 20 594–20 615, Sep. 14, 2022, ISSN: 1463-9084. DOI: 10.1039/D2CP02540J.

[27] *Discussion with Osmo Anttalainen*, Feb. 8, 2024.

[28] G. A. Eiceman, Z. Karpas, and H. H. Hill, *Ion Mobility Spectrometry*, Third edition, first issued in paperback. Boca Raton London New York: CRC Press, Taylor & Francis Group, 2016, 428 pp., ISBN: 978-1-138-19948-4. URL: https://www.routledge.com/Ion-Mobility-Spectrometry/Eiceman-Karpas-HillJr/p/book/9781138199484.

[29] A. Szczurek, M. Maziejuk, M. Maciejewska, T. Pietrucha, and T. Sikora, “BTX compounds recognition in humid air using differential ion mobility spectrometry combined with a classifier”, *Sensors and Actuators B: Chemical*, vol. 240, pp. 1237–1244, Mar. 1, 2017, ISSN: 0925-4005. DOI: 10.1016/j.snb.2016.08.164.

[30] R. P. Arasaradnam, E. Westenbrink, M. J. McFarlane, *et al.*, “Differentiating Coeliac Disease from Irritable Bowel Syndrome by Urinary Volatile Organic Compound Analysis – A Pilot Study”, *PLOS ONE*, vol. 9, no. 10, e107312, Oct. 16, 2014, ISSN: 1932-6203. DOI: 10.1371/journal.pone.0107312.

[31] M. K. Bomers, F. P. Menke, R. S. Savage, *et al.*, “Rapid, Accurate, and On-Site Detection of C. difficile in Stool Samples,” *Official journal of the American College of Gastroenterology | ACG*, vol. 110, no. 4, p. 588, Apr. 2015, ISSN: 0002-9270. DOI: 10.1038/ajg.2015.90.

[32] T. Sun, F. Tian, Y. Bi, *et al.*, “Local warning integrated with global feature based on dynamic spectra for FAIMS data analysis in detection of clinical wound infection”, *Sensors and Actuators B: Chemical*, vol. 298, p. 126 926, Nov. 1, 2019, ISSN: 0925-4005. DOI: 10.1016/j.snb.2019.126926.

[33] J. Virtanen, A. Roine, A. Kontunen, *et al.*, “The Detection of Bacteria in the Maxillary Sinus Secretion of Patients With Acute Rhinosinusitis Using an Electronic Nose: A Pilot Study”, *Annals of Otology, Rhinology & Laryngology*, Jan. 24, 2023, ISSN: 0003-4894. DOI: 10.1177/00034894231151301.

[34] D. Tingting, Z. Lei, W. Yifei, Z. Hao, H. Li, and D. Xiaoxia, “Integrated Lithium Battery-Powered High-Field Asymmetric Ion Mobility Spectrometer (FAIMS) for Molecular Structure Fingerprinting and Deep Learning”, *Analytical Letters*, vol. 0, no. 0, pp. 1–15, Mar. 6, 2023, ISSN: 0003-2719. DOI: 10.1080/00032719.2023.2185784.

[35] A. Anttalainen, “Cancer tissue classification from surgical smoke with convolutional neural networks”, May 6, 2019. URL: https://aaltodoc.aalto.fi:443/handle/123456789/37934.

[36] A. Wicaksono, “Diagnosis of human disease by odour analysis employing machine learning”, Ph.D. dissertation, University of Warwick, Sep. 2021. URL: http://webcat.warwick.ac.uk/record=b3781162.

[37] X. Ju, F. Lian, H. Ge, Y. Jiang, Y. Zhang, and D. Xu, “Identification of Rice Varieties and Adulteration Using Gas Chromatography-Ion Mobility Spectrometry”, *IEEE Access*, vol. 9, pp. 18 222–18 234, 2021, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3051685.

[38] A. Rauhameri, O. Anttalainen, J. Rantala, V. Surakka, A. Vehkaoja, and P. Muller, “Clustering of Alpha Curves in Differential Mobility Spectrometry Data”, *2022 IEEE International Symposium on Olfaction and Electronic Nose (ISOEN)*, May 2022, pp. 1–3. DOI: 10.1109/ISOEN54820.2022.9789640.

[39] A. Rauhameri, A. Robiños, O. Anttalainen, *et al.* “Classification of Volatile Organic Compounds by Differential Mobility Spectrometry Based on Continuity of Alpha Curves”. version 2. arXiv: 2401.07066 [cs]. (Mar. 13, 2024), pre-published.

[40] S. Balakrishnama, A. Ganapathiraju, and J. Picone, “Linear discriminant analysis for signal processing problems”, *Proceedings IEEE Southeastcon’99. Technology on the Brink of 2000 (Cat. No.99CH36300)*, Mar. 1999, pp. 78–81. DOI: 10.1109/SECON.1999.766096.

[41] C. M. Bishop, *Pattern Recognition and Machine Learning* (Information Science and Statistics). New York: Springer, 2006, 738 pp., ISBN: 978-0-387-31073-2.

[42] Y. LeCun. “What are some recent and potentially upcoming breakthroughs in deep learning?”, Quora. (Jul. 29, 2016), URL: https://www.quora.com/What-are-some-recent-and-potentially-upcoming-breakthroughs-in-deep-learning.

[43] A. Kazerouni, E. K. Aghdam, M. Heidari, *et al.*, “Diffusion models in medical imaging: A comprehensive survey”, *Medical Image Analysis*, vol. 88, p. 102 846, Aug. 1, 2023, ISSN: 1361-8415. DOI: 10.1016/j.media.2023.102846.

[44] F. Shamshad, S. Khan, S. W. Zamir, *et al.*, “Transformers in medical imaging: A survey”, *Medical Image Analysis*, vol. 88, p. 102 802, Aug. 1, 2023, ISSN: 1361-8415. DOI: 10.1016/j.media.2023.102802.

[45] M. U. Akbar, W. Wang, and A. Eklund. “Beware of diffusion models for synthesizing medical images – A comparison with GANs in terms of memorizing brain MRI and chest x-ray images”. arXiv: 2305.07644 [cs, eess]. (Jul. 7, 2024), prepublished.

[46] M. Frid-Adar, I. Diamant, E. Klang, M. Amitai, J. Goldberger, and H. Greenspan, “GAN-based synthetic medical image augmentation for increased CNN performance in liver lesion classification”, *Neurocomputing*, vol. 321, pp. 321–331, Dec. 10, 2018, ISSN: 0925-2312. DOI: 10.1016/j.neucom.2018.09.013.

[47] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and Improving the Image Quality of StyleGAN”, *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2020, pp. 8107–8116. DOI: 10.1109/CVPR42600.2020.00813.

[48] O. Lang, Y. Gandelsman, M. Yarom u. a., „Explaining in Style: Training a GAN to explain a classifier in StyleSpace“, in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, Okt. 2021, S. 673–682. DOI: 10.1109/ICCV48922.2021.00073.

[49] ImageNet. “ImageNet”. (2021), URL: https://www.image-net.org/about.php.

[50] J. Yang, R. Shi, D. Wei, *et al.*, “MedMNIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification”, *Scientific Data*, vol. 10, no. 1, p. 41, Jan. 19, 2023, ISSN: 2052-4463. DOI: 10.1038/s41597-022-01721-8.

[51] M. Roser, “The brief history of artificial intelligence: The world has changed fast – what might be next?”, *Our World in Data*, Jan. 29, 2024. URL: https://ourworldindata.org/brief-history-of-ai.

[52] Wikipedia, *Generative adversarial network*, *Wikipedia*, Dec. 30, 2023. URL: https://en.wikipedia.org/w/index.php?title=Generative_adversarial_network&oldid=1192688676.

[53] J. Gui, Z. Sun, Y. Wen, D. Tao, and J. Ye, “A Review on Generative Adversarial Networks: Algorithms, Theory, and Applications”, *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 3313–3332, Apr. 2023, ISSN: 1558-2191. DOI: 10.1109/TKDE.2021.3130191.

[54] S. Bond-Taylor, A. Leach, Y. Long, and C. G. Willcocks, “Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models”, *IEEE Transactions on Pattern Analysis and Machine*

Intelligence, vol. 44, no. 11, pp. 7327–7347, Nov. 2022, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2021.3116668.

[55] Y. Chen, X.-H. Yang, Z. Wei, *et al.*, "Generative Adversarial Networks in Medical Image augmentation: A review", *Computers in Biology and Medicine*, vol. 144, p. 105 382, May 1, 2022, ISSN: 0010-4825. DOI: 10.1016/j.compbiomed.2022.105382.

[56] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive Growing of GANs for Improved Quality, Stability, and Variation", presented at the International Conference on Learning Representations, Feb. 15, 2018. URL: https://openreview.net/forum?id=Hk99zCeAb.

[57] T. Karras, M. Aittala, J. Hellsten, S. Laine, J. Lehtinen, and T. Aila, "Training Generative Adversarial Networks with Limited Data", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 12 104–12 114. URL: https://proceedings.neurips.cc/paper/2020/hash/8d30aa96e72440759f74bd2306c1fa3d-Abstract.html.

[58] Z. Wu, D. Lischinski, and E. Shechtman, "StyleSpace Analysis: Disentangled Controls for StyleGAN Image Generation", presented at the 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Nashville, TN, USA: IEEE, Jun. 2021, pp. 12 858–12 867, ISBN: 978-1-66544-509-2. DOI: 10.1109/CVPR46437.2021.01267.

[59] M. Mirza and S. Osindero. "Conditional Generative Adversarial Nets". version 1. arXiv: 1411.1784 [cs, stat]. (Nov. 6, 2014), pre-published.

[60] Q. Yang, Z. Wang, K. Guo, C. Cai, and X. Qu, "Physics-Driven Synthetic Data Learning for Biomedical Magnetic Resonance: The imaging physics-based data synthesis paradigm for artificial intelligence", *IEEE Signal Processing Magazine*, vol. 40, no. 2, pp. 129–140, Mar. 2023, ISSN: 1558-0792. DOI: 10.1109/MSP.2022.3183809.

[61] T. Zhou, Q. Li, H. Lu, Q. Cheng, and X. Zhang, "GAN review: Models and medical image fusion applications", *Information Fusion*, vol. 91, pp. 134–148, Mar. 1, 2023, ISSN: 1566-2535. DOI: 10.1016/j.inffus.2022.10.017.

[62] M. Kang, J. Shin, and J. Park, "StudioGAN: A Taxonomy and Benchmark of GANs for Image Synthesis", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15 725–15 742, Dec. 2023, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2023.3306436.

[63] A. Bermano, R. Gal, Y. Alaluf, *et al.*, "State-of-the-Art in the Architecture, Methods and Applications of StyleGAN", *Computer Graphics Forum*, vol. 41, no. 2, pp. 591–611, 2022, ISSN: 1467-8659. DOI: 10.1111/cgf.14503.

[64] M. Cossio. "Augmenting Medical Imaging: A Comprehensive Catalogue of 65 Techniques for Enhanced Data Analysis". arXiv: 2303.01178 [cs, eess]. (Mar. 2, 2023), pre-published.

[65] F. Garcea, A. Serra, F. Lamberti und L. Morra, „Data augmentation for medical imaging: A systematic literature review“, *Computers in Biology and Medicine*, Jg. 152, S. 106 391, 1. Jan. 2023, ISSN: 0010-4825. DOI: 10.1016/j.compbiomed.2022.106391.

[66] D. Schaudt, C. Späte, R. von Schwerin, *et al.*, “A Critical Assessment of Generative Models for Synthetic Data Augmentation on Limited Pneumonia X-ray Data”, *Bioengineering*, vol. 10, no. 12, p. 1421, 12 Dec. 2023, ISSN: 2306-5354. DOI: 10.3390/bioengineering10121421.

[67] A. Mumuni and F. Mumuni, “Data augmentation: A comprehensive survey of modern approaches”, *Array*, vol. 16, p. 100 258, Dec. 1, 2022, ISSN: 2590-0056. DOI: 10.1016/j.array.2022.100258.

[68] Z. Yang, F. Zhan, K. Liu, M. Xu, and S. Lu. “AI-Generated Images as Data Source: The Dawn of Synthetic Era”. arXiv: 2310.01830 [cs]. (Oct. 23, 2023), pre-published.

[69] A. Kebaili, J. Lapuyade-Lahorgue, and S. Ruan, “Deep Learning Approaches for Data Augmentation in Medical Imaging: A Review”, *Journal of Imaging*, vol. 9, no. 4, p. 81, 4 Apr. 2023, ISSN: 2313-433X. DOI: 10.3390/jimaging9040081.

[70] E. Goceri, “Medical image data augmentation: Techniques, comparisons and interpretations”, *Artificial Intelligence Review*, vol. 56, no. 11, pp. 12 561–12 605, Nov. 1, 2023, ISSN: 1573-7462. DOI: 10.1007/s10462-023-10453-z.

[71] I. Joshi, M. Grimmer, C. Rathgeb, C. Busch, F. Bremond, and A. Dantcheva, “Synthetic Data in Human Analysis: A Survey”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 7, pp. 4957–4976, Jul. 2024, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2024.3362821.

[72] R. J. Chen, M. Y. Lu, T. Y. Chen, D. F. K. Williamson, and F. Mahmood, “Synthetic data in machine learning for medicine and healthcare”, *Nature Biomedical Engineering*, vol. 5, no. 6, pp. 493–497, 6 Jun. 2021, ISSN: 2157-846X. DOI: 10.1038/s41551-021-00751-8.

[73] Y. Chen and P. Esmaeilzadeh, “Generative AI in Medical Practice: In-Depth Exploration of Privacy and Security Challenges”, *Journal of Medical Internet Research*, vol. 26, e53008, Mar. 8, 2024, ISSN: 1439-4456. DOI: 10.2196/53008. pmid: 38457208.

[74] M. Abdollahzadeh, T. Malekzadeh, C. T. H. Teo, K. Chandrasegaran, G. Liu, and N.-M. Cheung. “A Survey on Generative Modeling with Limited Data, Few Shots, and Zero Shot”. arXiv: 2307.14397 [cs]. (Jul. 26, 2023), pre-published.

[75] M. Yang and Z. Wang. “Image Synthesis under Limited Data: A Survey and Taxonomy”. arXiv: 2307.16879 [cs]. (Jul. 31, 2023), pre-published.

[76] J. Röglin, K. Ziegeler, J. Kube, F. König, K.-G. Hermann, and S. Ortmann, “Improving classification results on a small medical dataset using a GAN; An outlook for

dealing with rare disease datasets", *Frontiers in Computer Science*, vol. 4, 2022, ISSN: 2624-9898. DOI: 10.3389/fcomp.2022.858874.

[77] W. Xia, Y. Zhang, Y. Yang, J.-H. Xue, B. Zhou, and M.-H. Yang, "GAN Inversion: A Survey", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3121–3138, 2022, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2022.3181070.

[78] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 12, pp. 4217–4228, 2021. DOI: 10.1109/TPAMI.2020.2970919.

[79] M.-A. Carbonneau, J. Zaïdi, J. Boilard, and G. Gagnon, "Measuring Disentanglement: A Review of Metrics", *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–15, 2022, ISSN: 2162-2388. DOI: 10.1109/TNNLS.2022.3218982.

[80] X. Chen, Y. Duan, R. Houthooft, J. Schulman, I. Sutskever, and P. Abbeel, "InfoGAN: Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets", *Advances in Neural Information Processing Systems*, vol. 29, Curran Associates, Inc., 2016. URL: https://papers.nips.cc/paper_files/paper/2016/hash/7c9d0b1f96aebd7b5eca8c3edaa19ebb-Abstract.html.

[81] I. Cetin, M. Stephens, O. Camara, and M. A. González Ballester, "Attri-VAE: Attribute-based interpretable representations of medical images with variational autoencoders", *Computerized Medical Imaging and Graphics*, vol. 104, p. 102 158, Mar. 1, 2023, ISSN: 0895-6111. DOI: 10.1016/j.compmedimag.2022.102158.

[82] E. Härkönen, A. Hertzmann, J. Lehtinen, and S. Paris, "GANSpace: Discovering Interpretable GAN Controls", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9841–9850. URL: https://proceedings.neurips.cc/paper/2020/hash/6fe43269967adbb64ec6149852b5cc3e-Abstract.html.

[83] Y. L. E. YLE. "Mäntän kuvataideviikoilla voi antaa itsensä tekoälyn käyttöön – 'Ihmisellä on samanlainen suhde tekoälyyn kuin jumalaan'", Yle Uutiset. (28. heinäkuuta 2021), url: https://yle.fi/a/3-12027129.

[84] Q. Su, H. N. A. Hamed, M. A. Isa, X. Hao und X. Dai, „A GAN-Based Data Augmentation Method for Imbalanced Multi-Class Skin Lesion Classification", *IEEE Access*, Jg. 12, S. 16 498–16 513, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3360215.

[85] J. M. Dolezal, R. Wolk, H. M. Hieromnimon, *et al.*, "Deep learning generates synthetic cancer histology for explainability and education", *npj Precision Oncology*, vol. 7, no. 1, pp. 1–13, May 29, 2023, ISSN: 2397-768X. DOI: 10.1038/s41698-023-00399-4.

[86] N. Akhtar. “A Survey of Explainable AI in Deep Visual Modeling: Methods and Metrics”. arXiv: 2301.13445 [cs]. (Jan. 31, 2023), pre-published.

[87] F. Bodria, F. Giannotti, R. Guidotti, F. Naretto, D. Pedreschi, and S. Rinzivillo, “Benchmarking and survey of explanation methods for black box models”, *Data Mining and Knowledge Discovery*, vol. 37, no. 5, pp. 1719–1778, Sep. 1, 2023, ISSN: 1573-756X. DOI: 10.1007/s10618-023-00933-9.

[88] G. Müller-Franzes, J. Niehues, F. Khader, *et al.*, “Diffusion Probabilistic Models beat GANs on Medical Images”, Dec. 14, 2022. URL: https://arxiv.org/abs/2212.07501.

[89] P. Dhariwal and A. Nichol, “Diffusion Models Beat GANs on Image Synthesis”, *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 8780–8794. URL: https://papers.nips.cc/paper_files/paper/2021/hash/49ad23d1ec9fa4bd8d77d02681df5cfa-Abstract.html.

[90] L. X. Nguyen, P. Sone Aung, H. Q. Le, S.-B. Park, and C. S. Hong, “A New Chapter for Medical Image Generation: The Stable Diffusion Method”, *2023 International Conference on Information Networking (ICOIN)*, Jan. 2023, pp. 483–486. DOI: 10.1109/ICOIN56518.2023.10049010.

[91] H. Cao, C. Tan, Z. Gao, *et al.*, “A Survey on Generative Diffusion Models”, *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–20, 2024, ISSN: 1558-2191. DOI: 10.1109/TKDE.2024.3361474.

[92] Y. Song, T. Wang, P. Cai, S. K. Mondal, and J. P. Sahoo, “A Comprehensive Survey of Few-shot Learning: Evolution, Applications, Challenges, and Opportunities”, *ACM Computing Surveys*, vol. 55, 271:1–271:40, 13s Jul. 13, 2023, ISSN: 0360-0300. DOI: 10.1145/3582688.

[93] C. Chadebec, L. J. Vincent, and S. Allassonniere, “Pythae: Unifying Generative Autoencoders in Python - A Benchmarking Use Case”, presented at the Thirty-Sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2023. URL: https://openreview.net/forum?id=w7VPQWgnn3s.

[94] C. Chadebec, E. Thibeau-Sutre, N. Burgos, and S. Allassonnière, “Data Augmentation in High Dimensional Low Sample Size Setting Using a Geometry-Based Variational Autoencoder”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–18, 2022, ISSN: 0162-8828, 2160-9292, 1939-3539. DOI: 10.1109/TPAMI.2022.3185773. arXiv: 2105.00026 [cs, stat].

[95] W. H. L. Pinaya, M. S. Graham, E. Kerfoot, *et al.* “Generative AI for Medical Imaging: Extending the MONAI Framework”. arXiv: 2307.15208 [cs, eess]. (Jul. 27, 2023), pre-published.

[96] jbetker. “The “it” in AI models is the dataset. – Non_Interactive – Software & ML.” (Jun. 10, 2023), URL: https://nonint.com/2023/06/10/the-it-in-ai-models-is-the-dataset/.

[97] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019. URL: https://arxiv.org/abs/1912.01703.

[98] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. URL: http://jmlr.org/papers/v12/pedregosa11a.html.

[99] J. Virtanen, "Detecting Rhinosinusitis With an Electronic Nose Based on Differential Mobility Spectrometry", Tampere University, 2023, 190 pp. URL: https://trepo.tuni.fi/handle/10024/144303.

[100] S. Woo, S. Debnath, R. Hu, *et al.*, "ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders", *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2023, pp. 16 133–16 142. DOI: 10.1109/CVPR52729.2023.01548.

[101] Wiktionary, *Graduate student descent*, *Wiktionary, the Free Dictionary*, Sep. 1, 2022. URL: https://en.wiktionary.org/w/index.php?title=graduate_student_descent&oldid=68997468.

[102] A. Borji, "Pros and cons of GAN evaluation measures: New developments", *Computer Vision and Image Understanding*, vol. 215, p. 103 329, Jan. 1, 2022, ISSN: 1077-3142. DOI: 10.1016/j.cviu.2021.103329.

[103] G. Stein, J. Cresswell, R. Hosseinzadeh, *et al.*, "Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models", *Advances in Neural Information Processing Systems*, vol. 36, pp. 3732–3784, Dec. 15, 2023. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/0bc795afae289ed465a65a3b4b1f4eb7-Abstract-Conference.html.

[104] K. Shmelkov, C. Schmid, and K. Alahari, "How Good Is My GAN?", *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds., Cham: Springer International Publishing, 2018, pp. 218–234, ISBN: 978-3-030-01216-8. DOI: 10.1007/978-3-030-01216-8_14.

[105] S. Ravuri and O. Vinyals, "Classification Accuracy Score for Conditional Generative Models", *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/fcf55a303b71b84d326fb1d06e332a26-Abstract.html.

[106] L. Biewald, *Experiment Tracking with Weights and Biases*, 2020. URL: https://www.wandb.com/.

[107] Wikipedia, *Vision transformer*, *Wikipedia*, Jun. 10, 2023. URL: https://en.wikipedia.org/w/index.php?title=Vision_transformer&oldid=1159424413.

[108] O. Lang, D. Yaya-Stupp, I. Traynis, *et al.*, "Using generative AI to investigate medical imagery models and datasets", *eBioMedicine*, vol. 102, Apr. 1, 2024, ISSN: 2352-3964. DOI: 10.1016/j.ebiom.2024.105075. pmid: 38565004.

[109] N. V. der Vleuten, T. Radusinović, R. Akkerman, and M. Reksoprodjo, “[Re] Explaining in Style: Training a GAN to explain a classifier in StyleSpace”, presented at the ML Reproducibility Challenge 2021 (Fall Edition), Feb. 5, 2022. URL: https://openreview.net/forum?id=SYUxyazQh0Y.

[110] D. Korporaal, E. Bosch, R. Ettes, and G. V. Meer, “Replication study of ’Explaining in Style: Training a GAN to explain a classifier in StyleSpace’”, presented at the ML Reproducibility Challenge 2021 (Fall Edition), Feb. 5, 2022. URL: https://openreview.net/forum?id=BtIz0nz7hRY.

[111] C. M. van de Geijn, V. Kyriacou, I. Papadopoulou, and V. Vasileiou, “Reproducibility study for ’Explaining in Style: Training a GAN to explain a classifier in StyleSpace’”, presented at the ML Reproducibility Challenge 2021 (Fall Edition), Feb. 5, 2022. URL: https://openreview.net/forum?id=SK8gAhfX2AK.

[112] K. Alomar, H. I. Aysel, and X. Cai, “Data Augmentation in Classification and Segmentation: A Survey and New Strategies”, *Journal of Imaging*, vol. 9, no. 2, p. 46, 2 Feb. 2023, ISSN: 2313-433X. DOI: 10.3390/jimaging9020046.

[113] P. Chlap, H. Min, N. Vandenberg, J. Dowling, L. Holloway, and A. Haworth, “A review of medical image data augmentation techniques for deep learning applications”, *Journal of Medical Imaging and Radiation Oncology*, vol. 65, no. 5, pp. 545–563, 2021, ISSN: 1754-9485. DOI: 10.1111/1754-9485.13261.

[114] P. Virtanen, R. Gommers, T. E. Oliphant, *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python”, *Nature Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2, 2020, ISSN: 1548-7091, 1548-7105. DOI: 10.1038/s41592-019-0686-2.

[115] M. Karjalainen, A. Kontunen, A. Anttalainen, *et al.*, “Characterization of signal kinetics in real time surgical tissue classification system”, *Sensors and Actuators B: Chemical*, vol. 365, p. 131 902, Aug. 15, 2022, ISSN: 0925-4005. DOI: 10.1016/j.snb.2022.131902.

[116] P. Celard, E. L. Iglesias, J. M. Sorribes-Fdez, R. Romero, A. S. Vieira, and L. Borrajo, “A survey on deep learning applied to medical images: From simple artificial neural networks to generative models”, *Neural Computing and Applications*, vol. 35, no. 3, pp. 2291–2323, Jan. 1, 2023, ISSN: 1433-3058. DOI: 10.1007/s00521-022-07953-4.

[117] C. E. Karakas, A. Dirik, E. Yalçınkaya, and P. Yanardag, “FairStyle: Debiasing StyleGAN2 with Style Channel Manipulations”, *Computer Vision – ECCV 2022*, S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., Cham: Springer Nature Switzerland, 2022, pp. 570–586, ISBN: 978-3-031-19778-9. DOI: 10.1007/978-3-031-19778-9_33.

[118] I. Ktena, O. Wiles, I. Albuquerque, *et al.*, “Generative models improve fairness of medical classifiers under distribution shifts”, *Nature Medicine*, vol. 30, no. 4, pp. 1166–1173, Apr. 2024, ISSN: 1546-170X. DOI: 10.1038/s41591-024-02838-6.

[119] F. Emmert-Streib and M. Dehmer, “Taxonomy of machine learning paradigms : A data-centric perspective”, 2022, ISSN: 1942-4787. DOI: 10.1002/widm.1470.

[120] M. Usman, T. Zia, and A. Tariq, “Analyzing Transfer Learning of Vision Transformers for Interpreting Chest Radiography”, *Journal of Digital Imaging*, vol. 35, no. 6, pp. 1445–1462, Dec. 2022, ISSN: 0897-1889. DOI: 10.1007/s10278-022-00666-z. pmid: 35819537.

[121] A. Abbasi, S. Bahrami, T. Hemmati, and S. A. Mirroshandel, “Transfer-GAN: Data augmentation using a fine-tuned GAN for sperm morphology classification”, *Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization*, vol. 0, no. 0, pp. 1–17, 2023, ISSN: 2168-1163. DOI: 10.1080/21681163.2023.2238846.

[122] S.-W. Park, J.-Y. Kim, J. Park, S.-H. Jung, and C.-B. Sim, “How to train your pre-trained GAN models”, *Applied Intelligence*, vol. 53, no. 22, pp. 27 001–27 026, Nov. 1, 2023, ISSN: 1573-7497. DOI: 10.1007/s10489-023-04807-x.

[123] K. Saab, T. Tu, W.-H. Weng, *et al.* “Capabilities of Gemini Models in Medicine”. arXiv: 2404.18416 [cs]. (May 1, 2024), pre-published.

[124] Y. Zhao, H. Ding, H. Huang, and N.-M. Cheung, “A Closer Look at Few-shot Image Generation”, *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, New Orleans, LA, USA: IEEE, Jun. 2022, pp. 9130–9140, ISBN: 978-1-66546-946-3. DOI: 10.1109/CVPR52688.2022.00893.

[125] H. Li, J. Pan, H. Zeng, Z. Chen, X. Du, and W. Xiao, “Identification of Specific Substances in the FAIMS Spectra of Complex Mixtures Using Deep Learning”, *Sensors*, vol. 21, no. 18, p. 6160, 18 Jan. 2021, ISSN: 1424-8220. DOI: 10.3390/s21186160.

[126] Y. Jiang, S. Chang, and Z. Wang, “TransGAN: Two Pure Transformers Can Make One Strong GAN, and That Can Scale Up”, *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 14 745–14 758. URL: https://proceedings.neurips.cc/paper/2021/hash/7c220a2091c26a7f5e9f1cfb099511e3-Abstract.html.

[127] J. Zhao, X. Hou, M. Pan, and H. Zhang, “Attention-based generative adversarial network in medical imaging: A narrative review”, *Computers in Biology and Medicine*, vol. 149, p. 105 948, Oct. 1, 2022, ISSN: 0010-4825. DOI: 10.1016/j.compbiomed.2022.105948.

[128] X. I. Chen, N. Mishra, M. Rohaninejad, and P. Abbeel, “PixelSNAIL: An Improved Autoregressive Generative Model”, *Proceedings of the 35th International Conference on Machine Learning*, PMLR, 2017, pp. 864–872. URL: https://arxiv.org/abs/1712.09763.

[129] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman, “MaskGIT: Masked Generative Image Transformer”, presented at the Proceedings of the IEEE/CVF Con-

ference on Computer Vision and Pattern Recognition, 2022, pp. 11 315–11 325. URL: https://arxiv.org/abs/2202.04200.

[130] D. M. Anstine and O. Isayev, “Generative Models as an Emerging Paradigm in the Chemical Sciences”, *Journal of the American Chemical Society*, vol. 145, no. 16, pp. 8736–8750, Apr. 26, 2023, ISSN: 0002-7863. DOI: 10.1021/jacs.2c13467.

[131] A. Kontunen, M. Karjalainen, A. Anttalainen, *et al.*, “Real Time Tissue Identification From Diathermy Smoke by Differential Mobility Spectrometry”, *IEEE Sensors Journal*, vol. 21, no. 1, pp. 717–724, Jan. 2021, ISSN: 1558-1748. DOI: 10.1109/JSEN.2020.3012965.

[132] A. Barragán-Montero, U. Javaid, G. Valdés, *et al.*, “Artificial intelligence and machine learning for medical imaging: A technology review”, *Physica Medica*, vol. 83, pp. 242–256, Mar. 1, 2021, ISSN: 1120-1797. DOI: 10.1016/j.ejmp.2021.04.016.

[133] V. Cheng, V. M. Suriyakumar, N. Dullerud, S. Joshi, and M. Ghassemi, “Can You Fake It Until You Make It? Impacts of Differentially Private Synthetic Data on Downstream Classification Fairness”, *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT ’21, New York, NY, USA: Association for Computing Machinery, Mar. 1, 2021, pp. 149–160, ISBN: 978-1-4503-8309-7. DOI: 10.1145/3442188.3445879.

[134] S. Ghalebikesabi, L. Berrada, S. Gowal, *et al.* “Differentially Private Diffusion Models Generate Useful Synthetic Images”. arXiv: 2302.13861 [cs, stat]. (Feb. 27, 2023), pre-published.

[135] Z. Jia, J. Chen, X. Xu, *et al.*, “The importance of resource awareness in artificial intelligence for healthcare”, *Nature Machine Intelligence*, vol. 5, no. 7, pp. 687–698, Jul. 2023, ISSN: 2522-5839. DOI: 10.1038/s42256-023-00670-0.

[136] F. Cady, *The Data Science Handbook*. John Wiley & Sons, Jan. 20, 2017, 417 pp., ISBN: 978-1-119-09293-3. DOI: 10.1002/9781119092919.

APPENDIX A: STYLEX FIGURES

In this section, some StylEx-found features are displayed with captions. The intensities are boosted so their values are not displayed. These features together with the dashboard presented in Chapter 5 are the outcome of the method. Only some features are shown.

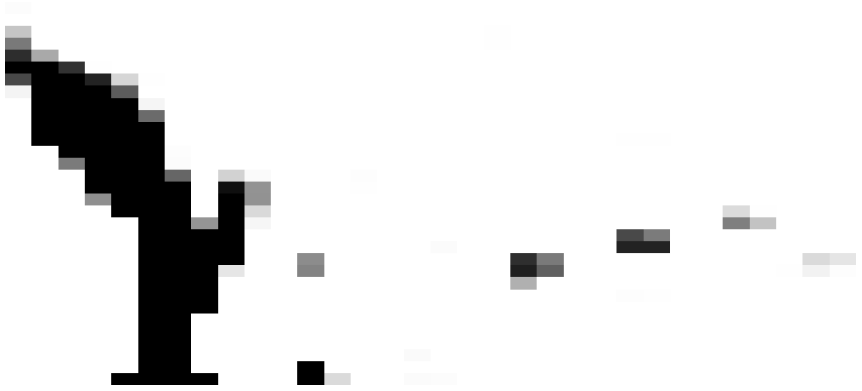

Figure A.1. *Image of the main dimer that appeared in multiple attributes. It was often accompanied by some pixels on the water curve as well as a common stump to the right.*

Figure A.2. *The main dimer together with faint curves for other features. To the right are the pixels for the monomer, water, and near the bottom what is likely ammonium.*

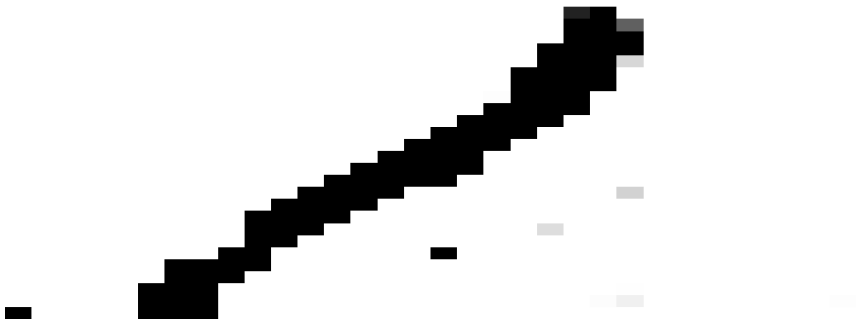

Figure A.3. *Image of the main monomer with some pixels for the water peak and the* $(0,0)$ *artefact.*

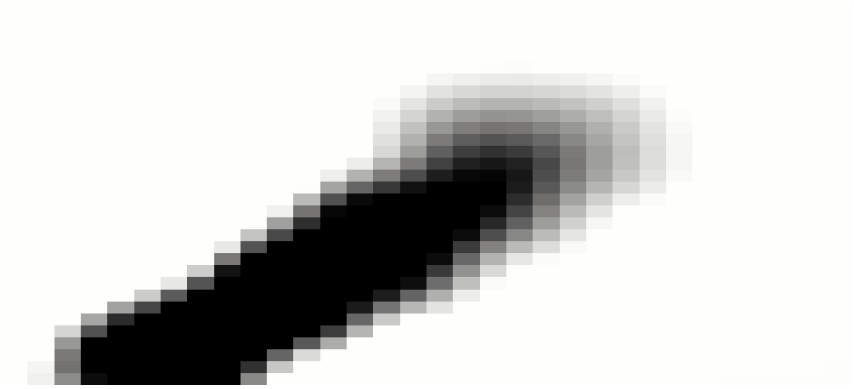

Figure A.4. *A more crude attribute of the main feature to the right. In lower layers of the network, the resolution is lower, leading to more crude features. They still represent the areas of interest and direction of the curve. In some attributes there are only 2 blobs, the starting point and the endpoint. The final image is generated using a large combination of these features.*

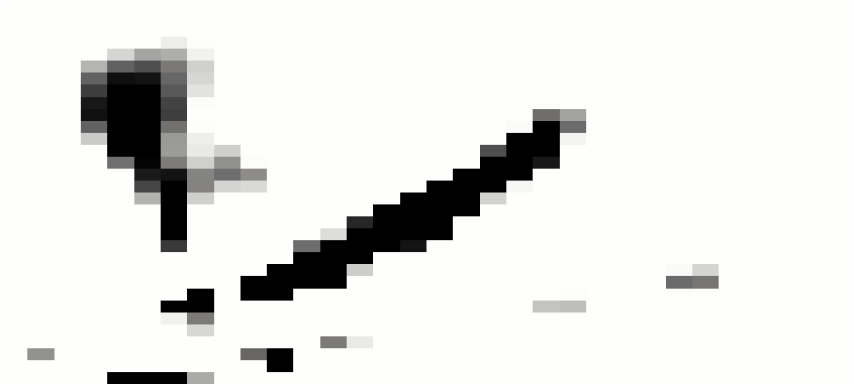

Figure A.5. *Main monomer going to the right. It appeared often. Either alone, but also commonly with the unknown feature on the left that was often a curve that changed its direction. Although the main features were common to both classes, they were more heavily indicated to belong to one of the classes. With longer training, the network might have focused on different features to differentiate the classes.*

Figure A.6. *Another image of the main monomer with more noise or partial curves. Here it starts to curve to the left in the upper part. In some attributes, the curve completed to the path indicated by the faint pixels. Below the monomer is the water peak with some tangential distribution. Other pixels might be correlative or simply relics due to the non-locality of convolutional layers.*

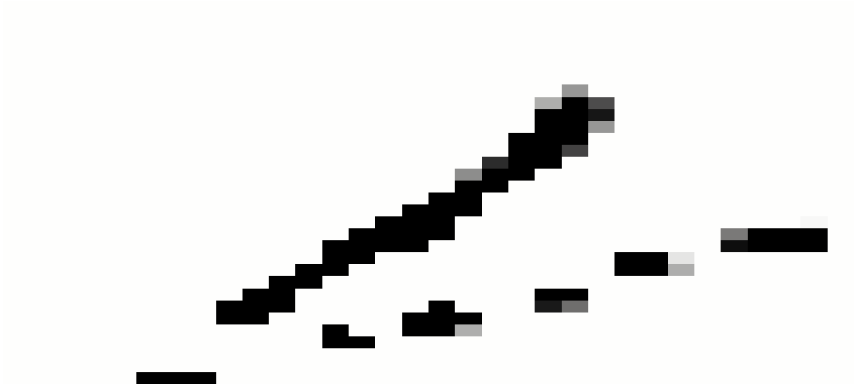

Figure A.7. *Cleaner version of two of the main features going to the right.*

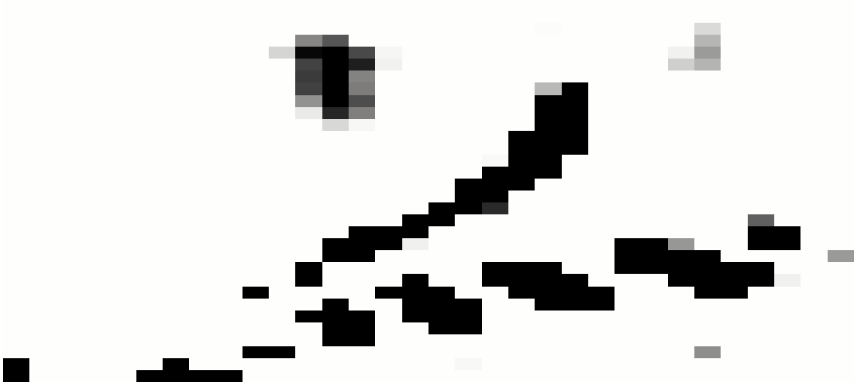

Figure A.8. *Another attribute of the main monomer with water. This one had fewer other features, although the one in the upper middle appears in both and was a common feature that was not explained properly. It could be the result of fragmentation or there could be multiple features there together. The general pixel area was indicated to be of importance. Note also the* $(0,0)$ *artefact.*

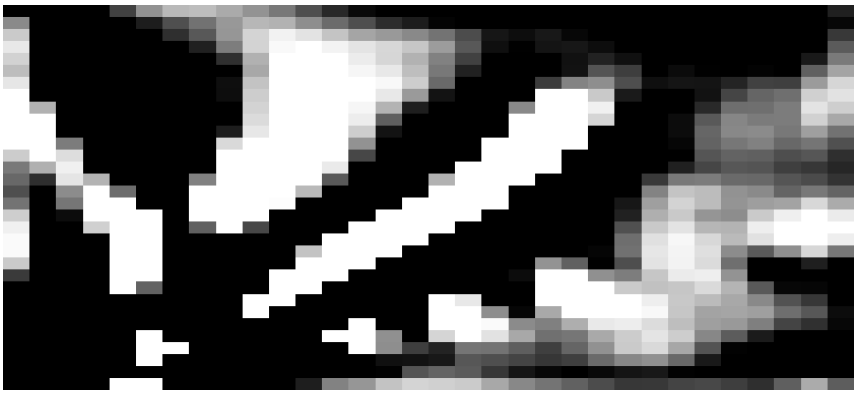

Figure A.9. *Main features with less clearly defined lines in negative color for clarity. Such multi-feature attributes are hard to interpret although they do reveal areas of significance (or non-significance) and might explain correlative features.*

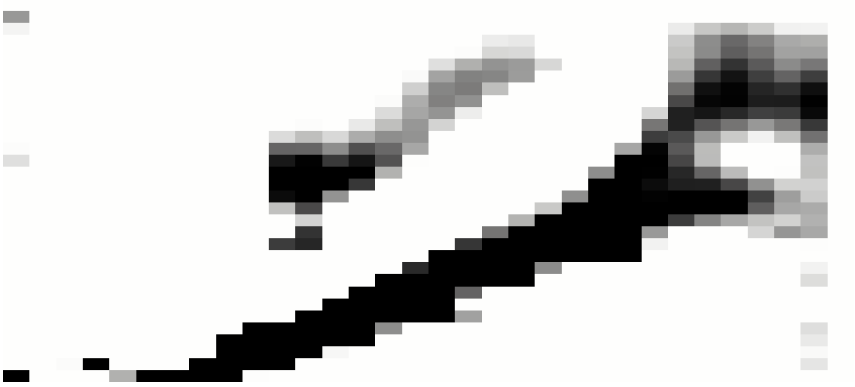

Figure A.10. *One of the main features to the right. It is accompanied by the 2 'tunnels' to the right side of the image in some cases. In others, the top part curves harder to the left and 'spreads' to the top side of the image. Another curve is seen above it. This could also be a feature that is 'around' a curve, as usually the area around the curve has the opposite polarity of ion intensity.*

Figure A.11. *One of the less common features. Notice the feint curvature to the left. While present in a couple of attributes that had solid linearity, this attribute was often very faint. It could indicate a less common alpha curve or separation of two. The area where it possibly splits was often also indicated with a blob. From the perspective of the classifier, it means that it uses this area to classify between classes, giving hints to important areas for class definitions.*

Figure A.12. *Another image of a feint curve to the left. Some weird extra action is included, as not all features are well-defined or sensical.*

Figure A.13. *Multiple features together. Some curves to the right, with also one curve to the left. Notice also the* $(0,0)$ *error feature. It is correlative with other features as it always represents one class due to unideal preprocessing methods.*

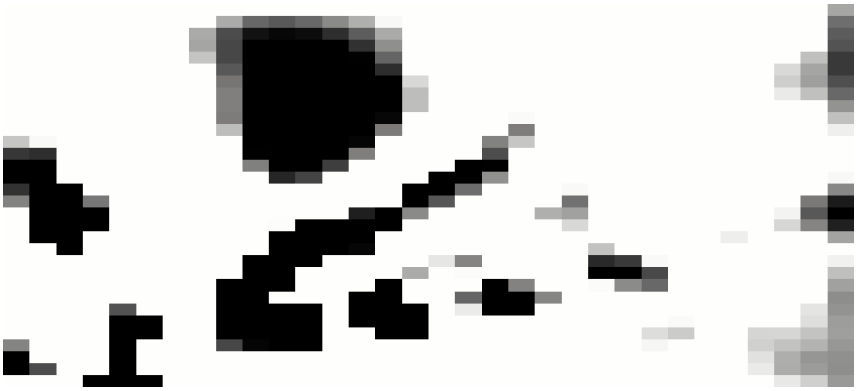

Figure A.14. *On the right side of the image there are three different normally distributed areas. These are common end-points of the curves going to the right. These could indicate a normally distributed ending point of the curves, with slightly different curvature due to the different sizes of the water cluster. The one in the bottom below water might be ammonium or artefact. The large blob in the upper-middle area was a commonplace of unclear activity.*

Figure A.15. *Multiple features again together. The one in the bottom-middle is the typical starting point for what might be the ammonium curve. The split before the dimer going to the left is also apparent in many attributes, where the end-point of the curve going to the right is possibly indicated in the image in the middle-top area.*

APPENDIX B: TRAINING METRIC GRAPHS

Some more metric graphs are provided in this section.

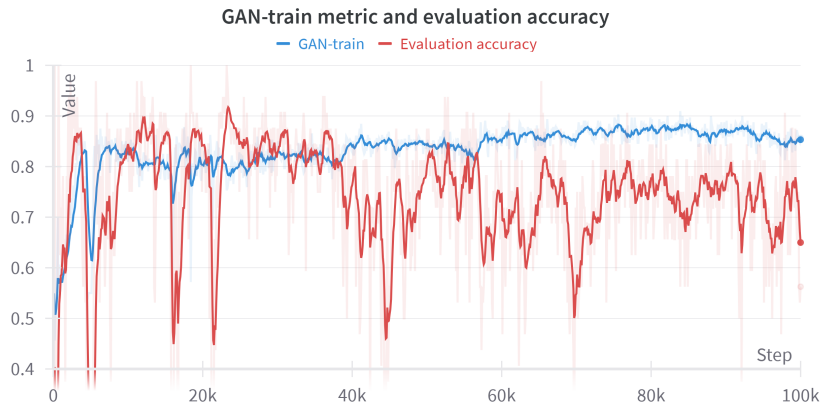

Figure B.1. *An example of a typical training run with the GAN-train metric. The training was stopped at 100k steps. The step with the best value for GAN-train was used to save the model disk, and that model generated the images for further evaluation. The other less stable line shows how the evaluation accuracy varies very highly during the training. This seems to correlate highly with how the network learned something that was more pronounced in one class over the other.*

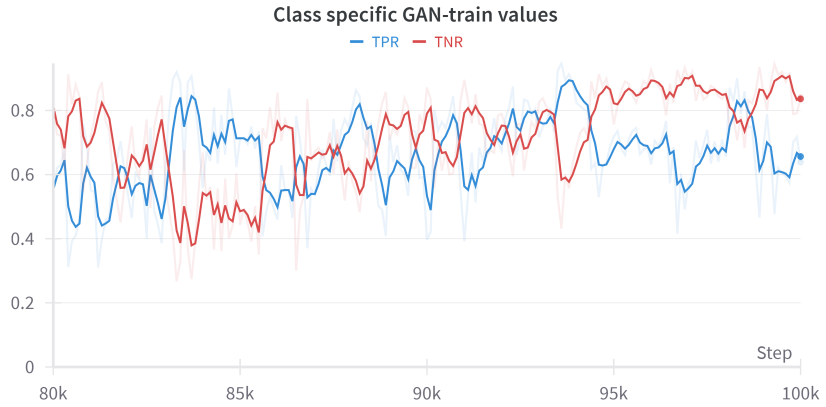

Figure B.2. *Typical graph for the evolution of the class-specific GAN-train metric values. It is easy to see the high negative correlation of the features. Although both improve from the initial condition of 0.5, they oscillate around each other throughout the whole training. Generally, class 2 (TNR) had a better value which motivated the search for balancing methods. In an ideal situation, the generator learns to make features that are unique to each class, raising this value for both of them. The negative correlation is likely due to shared features being more pronounced in either of the two classes. If the model is saved at an unbalanced point, the latents may be controlled to produce a more balanced dataset.*

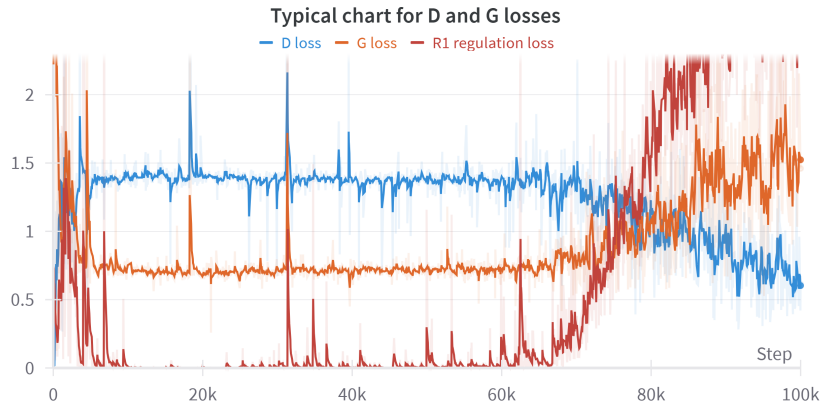

Figure B.3. *Typical D and G losses for a training run. The r1 regularization loss is also included. The equilibrium is reached here very early, at around 5k steps. The D will generally at some point in training start to be very confident, as seen after 60k steps. At this point, some control is required to keep the D 'dumber'. R1 regulation will force it to focus less on certain areas. The adaptive augmentation method ADA is activated similarly and would show rising activation similar to the R1 loss in this image, by more strongly augmenting images that go to the D. Both methods and many more have been developed to keep the D dumber, as the G will not receive good guidance if the D is overly confident. An arrogant D will breed an ignorant G. If the training were continued longer a new plateau would have been reached. In the very long stable zone from around 5k to 60k the metrics of the network improved. Although not visible on the losses, the generation capability improved, highlighting how the losses do not alone serve as guidance to the training.*

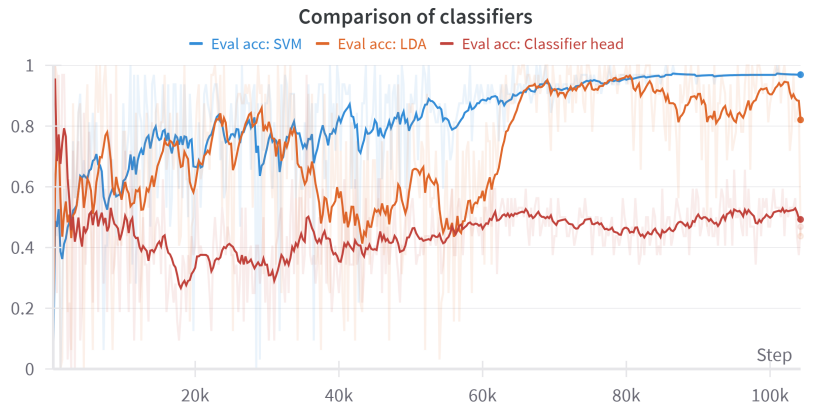

Figure B.4. *Comparison of classifiers on evaluation accuracy during StyleGAN training. Initially, multiple classifiers were compared including SVM, LDA, KNN, RFC, a classifier head, and more. LDA was chosen due to being a common choice in literature and also from my experience as it was the most robust in different settings. The classifier head is a linear layer built into the StyleGAN2 network and trained during the GAN training. The head can be also used to condition the network as is done in AC-GAN.*

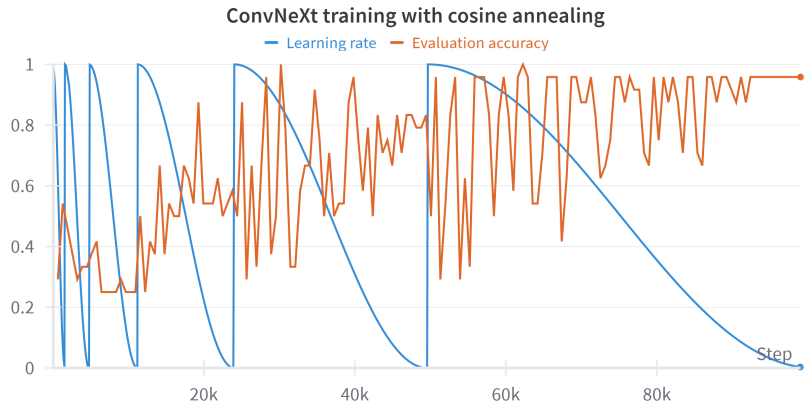

Figure B.5. *Typical ConvNeXt training. Validation data to end the training is not required as the training is halted before a restart in the cosine annealing scheduler. Because the learning rate is minimally low before a restart, the value for the evaluation accuracy is very stable. The jumps in learning rate help the network escape local minima. The proper number of steps was chosen by experience and the same value was used for all the results. Another possible method is to use cross-validation and for example 20% split for val-train data and stop the training at the best value of the validation loss.*

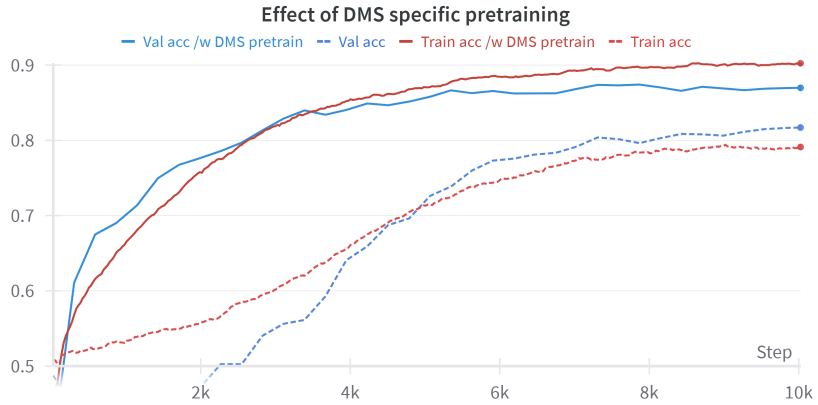

Figure B.6. *Effect of DMS image pre-training on CNN accuracy. This dataset was not the IDH dataset but a completely different DMS dataset. The base validation accuracy was* 81.7% *and it improved to* 86.9% *when the CNN classifier was first trained using GAN-made DMS images and later trained again on a completely different set. For an unknown reason, the IDH-GAN images were better than the original IDH images in this pre-training scheme. It could be that the high amount of fake images gave more variability in the very long pre-training compared to the limited training set. The DMS pre-trained version was also trained to similar accuracy within one-fourth of the training time. The DMS pre-trained version 'knew what to look for' in the images and learned the new set faster and with better accuracy.*

APPENDIX C: CODE SNIPPETS

Some training code snippets are provided in this section.

StylEx training in generator loop:

```
# Lpips is prepared before the loop.
# This version of the loss was done in alternating steps during training as was suggested. Every
other step was normal StyleGAN2 training and every other was this.
# Some lines that swap dimension axes etc. are omitted. More details are in the StudioGAN code
library for eg. generator functions.
# This loss calculation is for the generator. For the discriminator, the loss for real images is as
done originally, while the loss for fake images is also done in alternating steps to be the recon
images loss instead of fake image loss every alternating step.

# 1) get real imgs and classifier values for them
real_images, real_labels = self.sample_data_basket()
with torch.no_grad():
        c_real = self.classifier(real_images)
# 2) run imgs thru Encoder, then run output concatanated W thru Generator to get reconstructed imgs
and get classifier value for them.
real_dict = self.Encoder(real_images, real_labels)
w = torch.concat((real_dict["h"], c), dim=1)
recon_images = self.Gen.synthesis(w)
with torch.no_grad():
        c_rec = self.classifier(recon_images)
# 3) run reconstructed images through the encoder as well
recon_encode_dict = self.Encoder_(recon_images, real_labels)
encoder_output_real = real_dict["h"]
encoder_output_recon = recon_encode_dict["h"]
# 4a) Calculate Lx = difference btw. imgs
L_x = torch.nn.L1Loss(reduction='mean')(real_images, recon_images)
# 4b) Calculate L_w, the loss btw. real and reconstrcuted image after encoding
L_w = torch.nn.L1Loss(reduction='mean')(encoder_output_real, encoder_output_recon)
# 4c) Calculatte lpips, advanced difference btw. imgs
# needs 3 channels and normalized to -1 to 1
real_for_lpips, recon_for_lpips = prepare_for_lpips(real_images, recon_images)
L_lpips = self.lpips(real_for_lpips, recon_for_lpips).mean()
# 4) add recon losses together
recon_loss = 0.1*L_w + 1.0*L_x + 0.1*L_lpips
# 5) adversarial loss using recon img
dis_dict_recon = self.Dis(recon_images, real_labels)
recon_adv_loss = self.LOSS.g_loss(dis_dict_recon["adv_output"], real_labels)
# 6) classifier loss using a classifier
classify_real = torch.nn.functional.log_softmax(c_real, dim=0)
classify_recon = torch.nn.functional.log_softmax(c_rec, dim=0)
kl_loss = torch.nn.KLDivLoss(reduction='batchmean',log_target = True)(classify_real,
classify_recon)
# 7) summing the 3 losses
g_loss = 2*recon_loss+1*recon_adv_loss+1*kl_loss
```

Computation of the classifier's change when changes to the network are applied:

```
# The work to get changed classifier values is done in batches. Essentially we only need 1) CNN
classifiers and 2) LDA classifier's new opinion after a style has been changed. This is saved over
very many values for each dimension of S to get a discrete mapping of change over eg. (-1000,
1000).
# The main body of work is done inside stylex_synth function where the image is generated with the
change in style.
# Although seemingly simple, changing a parameter of a network in-run takes quite a lot of effort
due to the inner workings of PyTorch. I found it best to simply save all the network parameters on
disk and do the operations with changed parameters outside of the network parameters and replace
the values during the generation of the image at the correct step of the generator.
changed_imgs = self.Gen.synthesis.stylex_synth(style_vector, style_to_change, change_value)
s_preds = self.classifier(changed_imgs)
lda_preds = torch.tensor(lda_classifier.predict(np.array(imgs_new.view(-1, 2048).cpu(),
dtype='float64')), dtype=torch.float32)
```

The linearity search for ideal s-attributes:

```
# in here, x is changed values for s-attribute eg [-1.0, -0.5, 0, 0.5, 1.0]
# y is then the average classification rate over the whole dataset.
# when the attribute is changed, the classification rate changes.
# this find the best linear area around s-delta=0 where the change to classification rate is linear
and nice.
for FIT_LEN in range(30, 100):
        for i in range(len(x) - FIT_LEN-1):
                x_subset = x[i:i+FIT_LEN]
                y_subset = y[i:i+FIT_LEN]
                if y_subset.min()<-15: # lowest MEAN value within windows should not be below 15
                        continue
                if y_subset.max()>15: # .. or highest over 15
                        continue
                if abs(len(x_subset[x_subset<0])-len(x_subset[x_subset>0])) > 2: # the area should
                be balanced to both sides of 0, ~equal amounts in - and + sides.
                        continue
                if abs(y_subset.max()-y_subset.min()) < 8: # the mean value should change
                significantly within the window.
                        continue
                if y_subset.min()>-2: # want mean line to enter negatives
                        continue
                if y_subset.max()<2: # want mean line to enter positives
                        continue
                if max(abs(x_subset)) > 5: # want lower s values, as they have better features.
                        continue
                coefficients = np.polyfit(x_subset, y_subset, 1)
                poly = np.poly1d(coefficients)
                y_pred = poly(x_subset)
                ss_res = np.sum((y_subset - y_pred) ** 2)
                ss_tot = np.sum((y_subset - np.mean(y_subset)) ** 2)
                r_squared = 1 - (ss_res / ss_tot)
                if r_squared > best_fit_r_squared:
                        best_fit_r_squared = r_squared
                        best_fit_indices = (i, i+FIT_LEN)
                        best_fit_params = coefficients
# After this new images are generated only for the best found attributes in the best linear fit
area. These images are then displayed using the Plotly-dash script.
```

Competitive augmentation strategy for ConvNeXt training:

```
# Competitive augmentation strategy was formed using Albumentation library and search for the best
set of augmentations. All of the random-based methods use 3-dimensional methods, so I had to make
one myself that does not use color-based methods. This set was chosen as it had very robust
training in experimentation.

# All preprocessing is included except the invertion of the negative channel and flipping of the
channels to follow albumentations channel order. The values of the negative ion channel were
inverted to be positive.

# For evaluation data only the normalization transforms and resizing are included, not the
augmentation.

compose_list = []
compose_list.append(A.Normalize(mean=(0.5, 0.5), std=(0.5, 0.5), max_pixel_value=1,
always_apply=True, p=1.0))
compose_list.append(A.OneOf([
        A.Sharpen(p=0.5),
        A.Affine(p=0.5),
        A.RandomCropFromBorders(p=0.5),
        A.ZoomBlur(p=0.5),
        A.PixelDropout(p=0.5),
        A.Downscale(p=0.5),
],p=0.5))
compose_list.append(A.OneOf([
        A.Sharpen(p=0.5),
        A.Affine(p=0.5),
        A.RandomCropFromBorders(p=0.5),
        A.ZoomBlur(p=0.5),
        A.PixelDropout(p=0.5),
        A.Downscale(p=0.5),
],p=0.5))
compose_list.append(ToTensorV2(always_apply=True))
compose_list.append(A.Resize(224, 224, always_apply=True, interpolation=cv2.INTER_LINEAR)) #
ConvNeXt size
train_transforms = A.Compose(compose_list, p=1.00)
```

ConvNeXt V2 model initiation and training parameters:

```
# The model is instantiated using timm and pre-training. A typical training loop follows where the
model is always trained for 100 000 seen images. This means that if the dataset consists of 10 000
images, all of them are seen 10 times. If it is 100 images, they are all seen 1 000 times.

# Optimizer, scheduler, dropout for convnext, batch size, and other parameters were explored but
these are the values used for all models in this thesis.

model = timm.create_model(convnextv2_atto, pretrained=True, in_chans=2, num_classes=1,
drop_rate=0.5)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-5, weight_decay=1e-2)
scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer, T_0=1600, T_mult=2)
criterion = nn.BCEWithLogitsLoss()
batch_size = 32
```